STUDENTHUB PORTAL

WEB DEVELOPMENT FRAMEWORKS

PRACTICAL FILE: PRACTICALS 1-15

Student ID: D26DCE157

Practical	Title	PDF pages
1	Project Initiation, Requirements, Sitemap, Wireframes and GitHub Setup	2-12
2	Semantic HTML5 Pages and Accessibility	13-25
3	Responsive UI Design	26-42
4	JavaScript DOM Manipulation and UI Interactivity	43-54
5	Registration Form Validation	55-63
6	Fetch API: Rendering, Search and Filter	64-70
7	PHP Form Processing and File Storage	71-76
8	MySQL Schema Design, ER Model and PDO	77-82
9	Secure User Registration	83-88
10	Secure Login, Sessions, Roles and Remember-Me	89-95
11	Student Management CRUD with Search and Filter	96-103
12	Event Management CRUD and Poster Upload	104-111
13	PHP JSON API and Fetch-Based CRUD	112-116
14	Admin Dashboard, Role-Based Access and Audit Log	117-121
15	Final Integration, Testing, Documentation and Deployment	122-126

Each practical report follows in numerical order with its original report pages and included output evidence.

PRACTICAL – 1

Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

AIM:

To plan the StudentHub portal by identifying its requirements, user roles and pages, creating a sitemap and wireframes, and organizing the project using Git and GitHub.

Introduction:

StudentHub is a proposed web portal that brings academic and campus information together in one place. It helps students access notices, view events, find useful information and provide feedback.

Planned User Roles:

Role	Responsibility
Student	View academic information, events and profile details, and submit feedback.
Faculty	Provide academic notices and study materials.
Admin	Manage users and portal content.

Main Functional Requirements:

- Provide login and registration interfaces.
- Display academic notices and information on a dashboard.
- Show upcoming campus events.
- Provide student profile information.
- Include FAQ, contact and feedback pages.

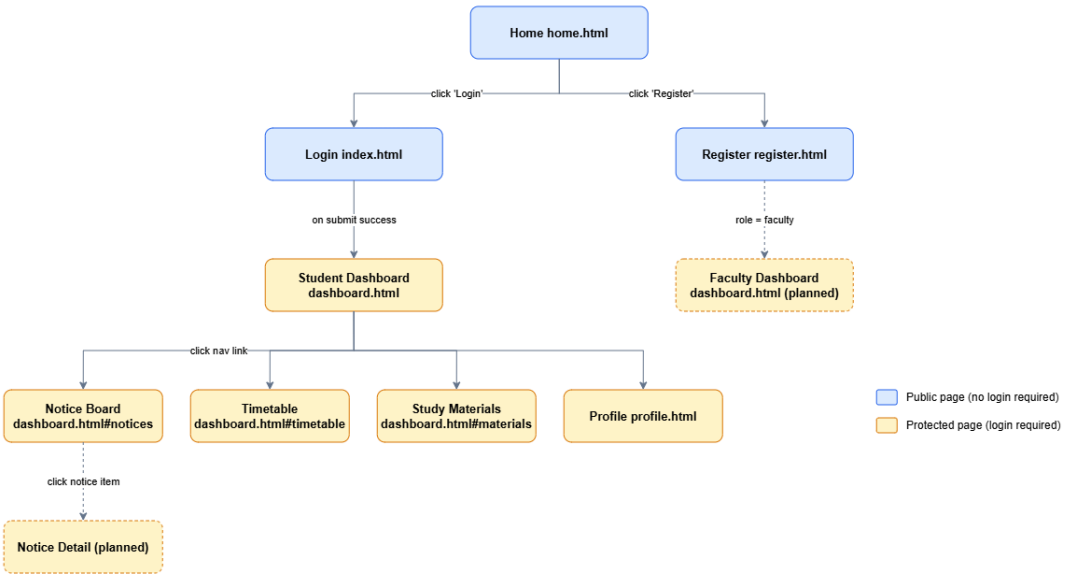
Pages of the Portal:

Page	Purpose
Home	Introduces the portal and its main features.
About	Explains the purpose of StudentHub.
Login	Provides an interface for existing users to sign in.
Register	Provides an interface for new users to register.
Dashboard	Organizes academic notices and useful information.
Events	Displays campus events and their details.
Profile	Displays student information.
FAQ	Answers common questions.
Contact	Provides contact information and an enquiry interface.
Feedback	Provides a form for student feedback.

These features describe the planned scope of the portal. Working authentication and data storage require further implementation.

SITEMAP

StudentHub — Sitemap



WIREFRAMES

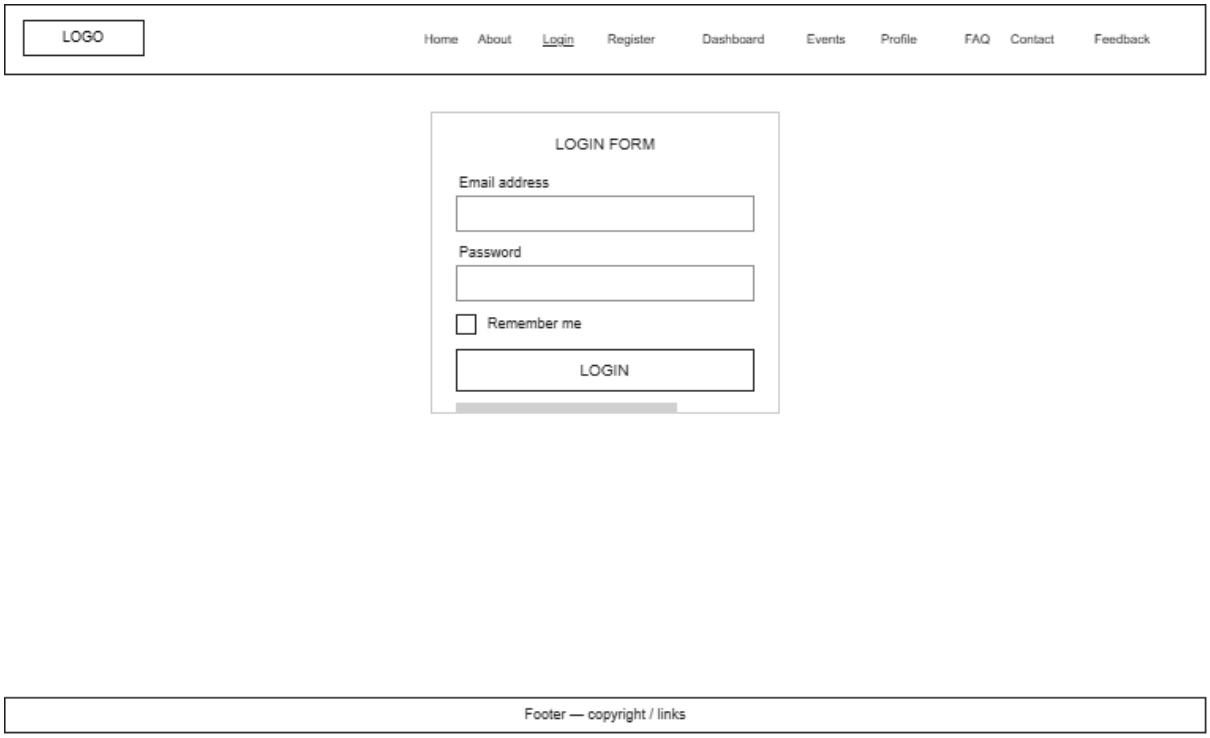
1. Home



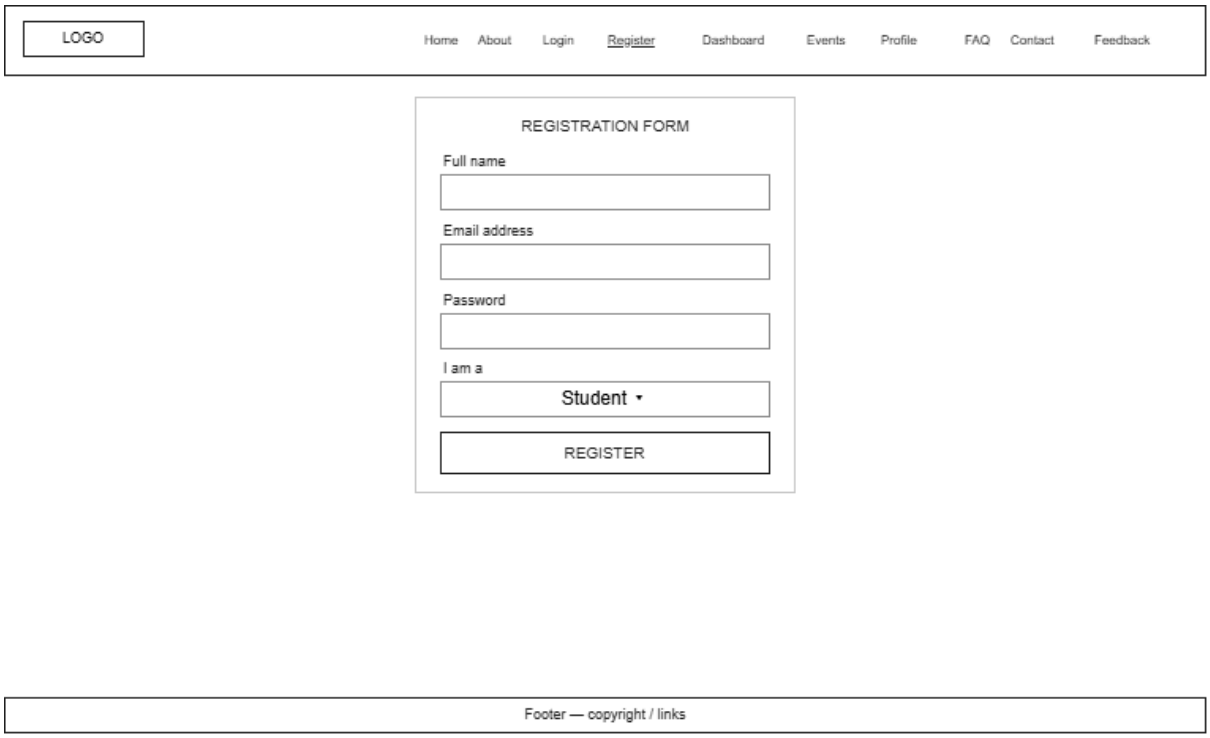
2. About



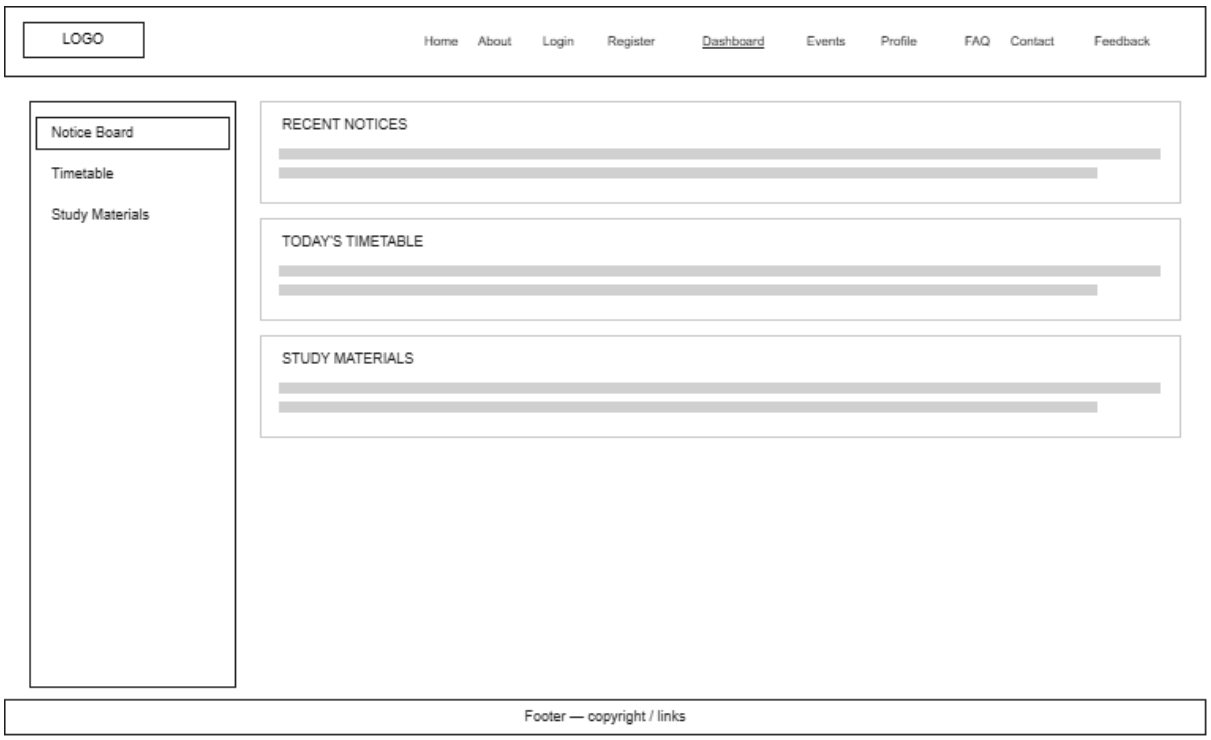
3. Login



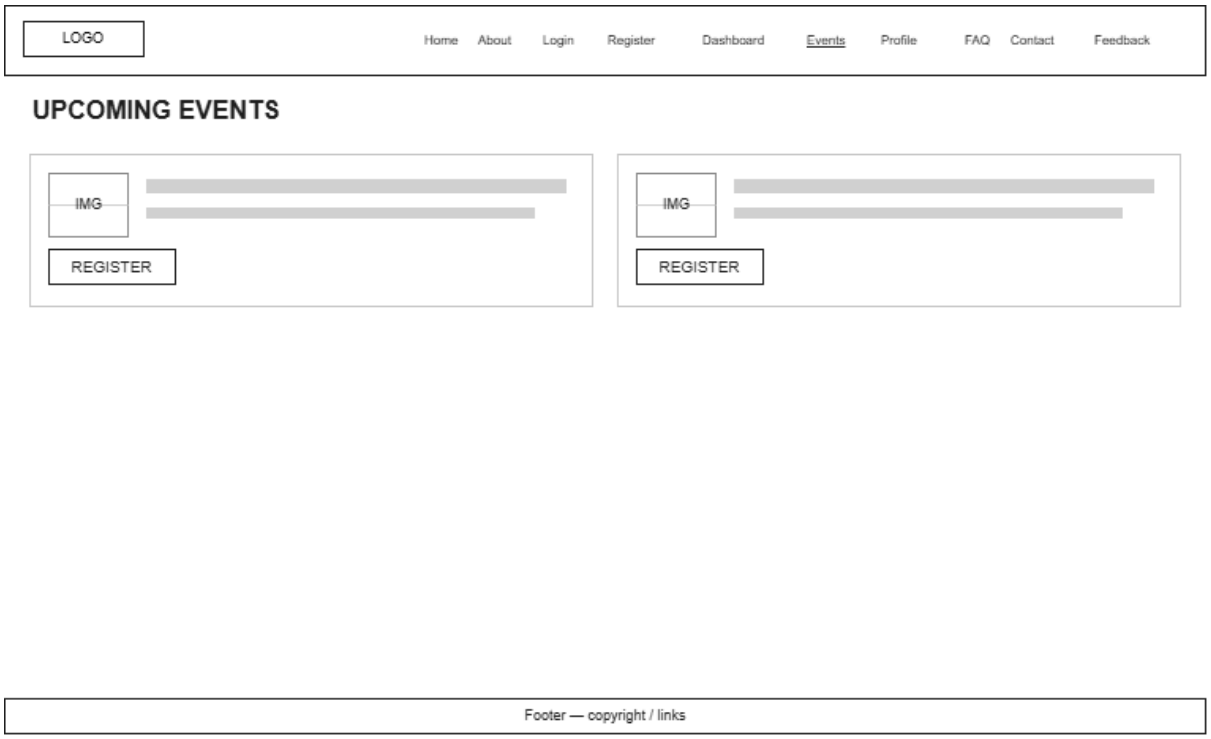
4. Register



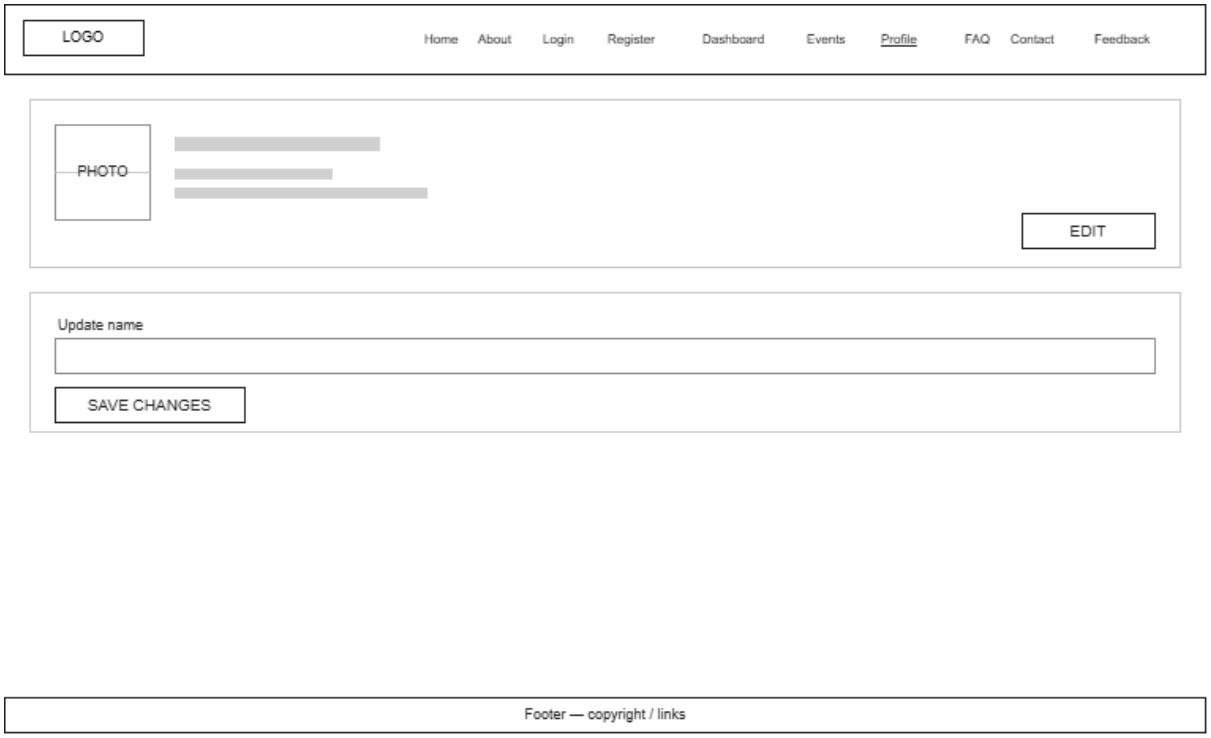
5. Dashboard



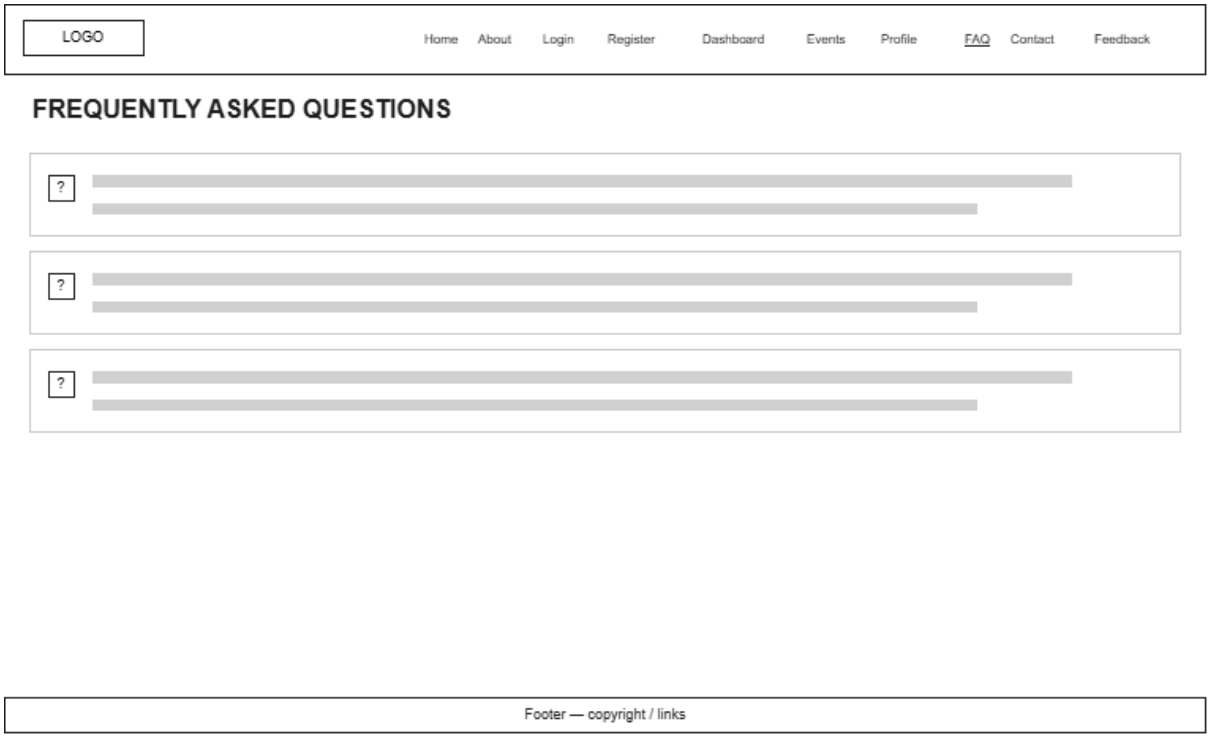
6. Events



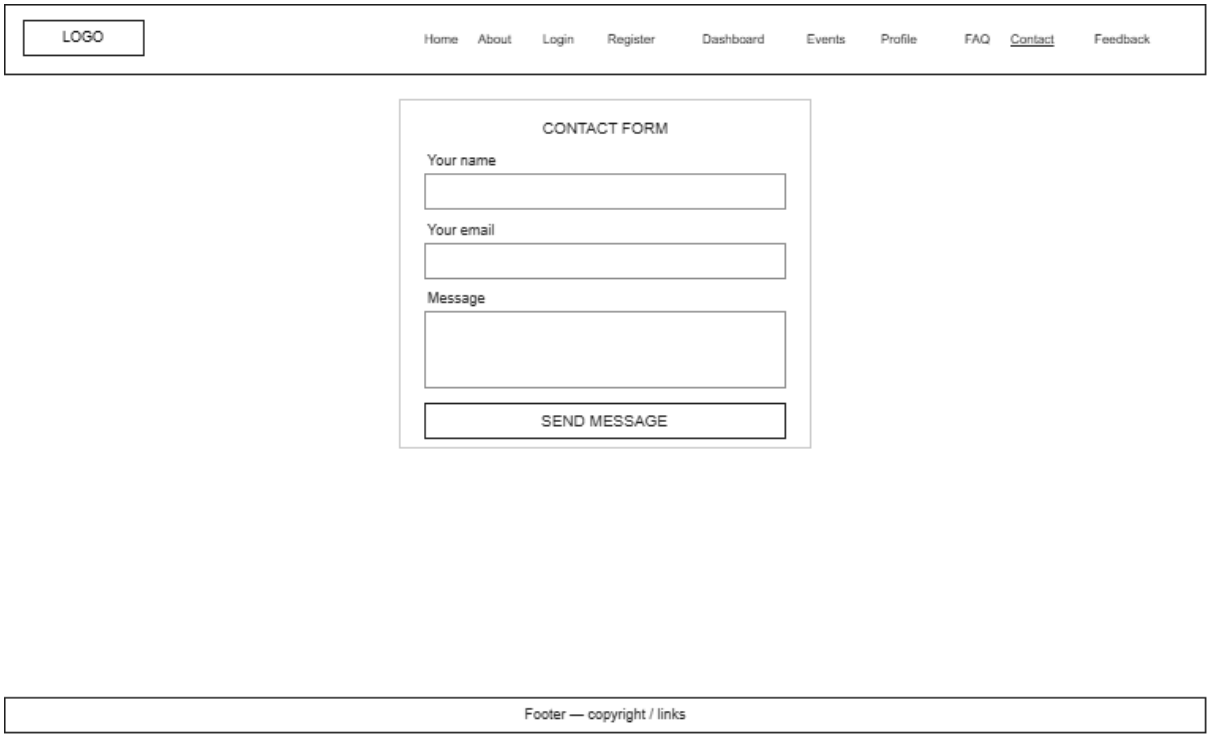
7. Profile



8. FAQ



9. Contact



10. Feedback



Key Questions and Answers

Q1. What is a URL, and what are its parts?

A URL, or Uniform Resource Locator, is the address of a resource on the internet.

Example:
`https://example.com:443/events.html?id=1#details`

Q2. How does an HTML file get processed in a web browser?

The browser reads the HTML file and builds the Document Object Model (DOM), which represents the page structure. It loads linked resources such as CSS, JavaScript and images. CSS controls the appearance, and JavaScript can add behaviour. The browser then calculates the layout and displays the page.

Q3. How will page navigation be managed among all HTML pages?

Navigation is managed using hyperlinks created with the `<a>` tag. Its `href` attribute specifies the destination page. For example, `<a href="about.html">About</a>` opens the About page. A common navigation menu helps users move consistently between the portal pages.

Q4. How will GitHub commits be maintained after each practical?

After each practical:

1. Save the completed changes.
2. Check modified files using `git status`.
3. Stage the required files using `git add`.
4. Record the changes using `git commit` with a meaningful message.
5. Upload the commits using `git push origin main`.

This maintains a history of the work completed during each practical.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

README.md

```
# StudentHub Portal

StudentHub is a web portal designed for college students to manage their
academic and campus life digitally. It reduces dependency on physical notice
boards, scattered WhatsApp groups, and manual processes by providing a
centralized platform for notices, timetables, study materials, and student
profiles.

## User Roles
- **Student** – View notices, check timetable, download study material, update
profile
- **Faculty** – Post notices, upload study materials, manage class schedule
- **Admin** – Manage users, approve/moderate content, oversee all modules

## Key Modules
1. Authentication (Login / Register)
2. Notice Board
3. Timetable / Schedule
4. Study Material Repository
5. Profile & Settings

## Pages
1. Home / Landing Page
2. Login Page
3. Register Page
4. Student Dashboard
5. Faculty Dashboard
6. Notice Board
7. Notice Detail Page
8. Timetable Page
9. Study Materials Page
10. Profile Page
11. About / Contact Page

## Navigation Flow
Landing Page → Login/Register → Role-based Dashboard → Module Pages (Notice
Board / Timetable / Study Materials) → Profile

## Tech Stack
HTML, CSS, JavaScript, GitHub

## Folder Structure
```
studenthub-portal/
├── docs/
│   ├── sitemap/
│   └── wireframes/
```
```

```
├── css/
├── js/
├── images/
└── README.md
```

PRACTICAL - 2

Semantic HTML5 Pages with Accessibility-Ready Structure

AIM:

To develop static HTML5 pages for the StudentHub portal using semantic tags, linked navigation and an accessibility-ready structure.

Introduction:

StudentHub is a web portal for organizing academic and campus information. This practical focuses on creating the HTML structure of at least ten pages and connecting them through navigation links. Semantic HTML and accessibility features help make the content clear and easy to use.

Main Requirements:

- Develop at least ten static StudentHub pages.
- Use semantic elements according to the purpose of the content.
- Link the pages through a consistent navigation menu.
- Provide suitable headings, form labels and image alternative text.
- Maintain a consistent page structure and prepare an accessibility checklist.

Tools and Technology:

VS Code, HTML5, a web browser, browser accessibility inspection tools, Git and GitHub.

Scope of the Practical:

The focus is on static page structure, page linking and accessibility readiness. Login and registration pages represent form interfaces; working authentication and data storage belong to later implementation.

Pages of the Portal:

Page	Purpose
Home	Introduces the portal and its main features.
About	Explains the purpose of StudentHub.
Login	Provides the sign-in form interface.
Register	Provides the new-user registration form.
Dashboard	Organizes academic notices and useful information.
Events	Displays campus events and their details.
Profile	Displays student profile information.
FAQ	Answers common questions about the portal.
Feedback	Provides a form for student feedback.

Page Navigation:

The portal pages are connected using HTML hyperlinks. Each link uses an href attribute to identify its destination. A common navigation menu gives users a consistent way to move between pages such as Home, About, Login, Register, Dashboard and Events.

HTML Source Code:

The StudentHub HTML source files are provided through the project repository linked below. The page screenshots in this report show the browser output of the portal pages.

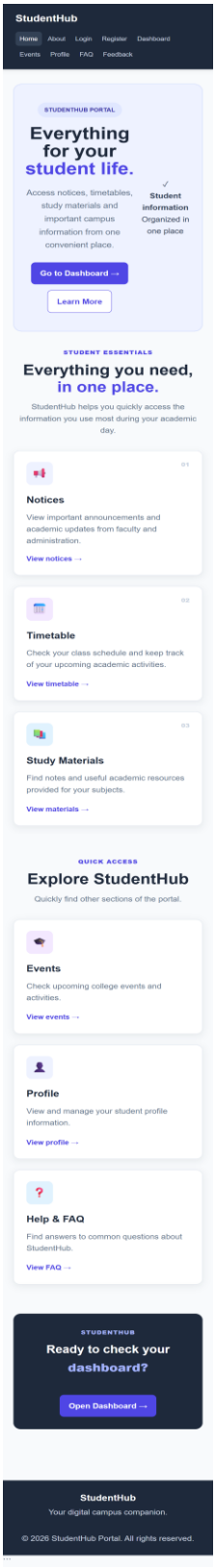
GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

(FOR SOURCE CODE)

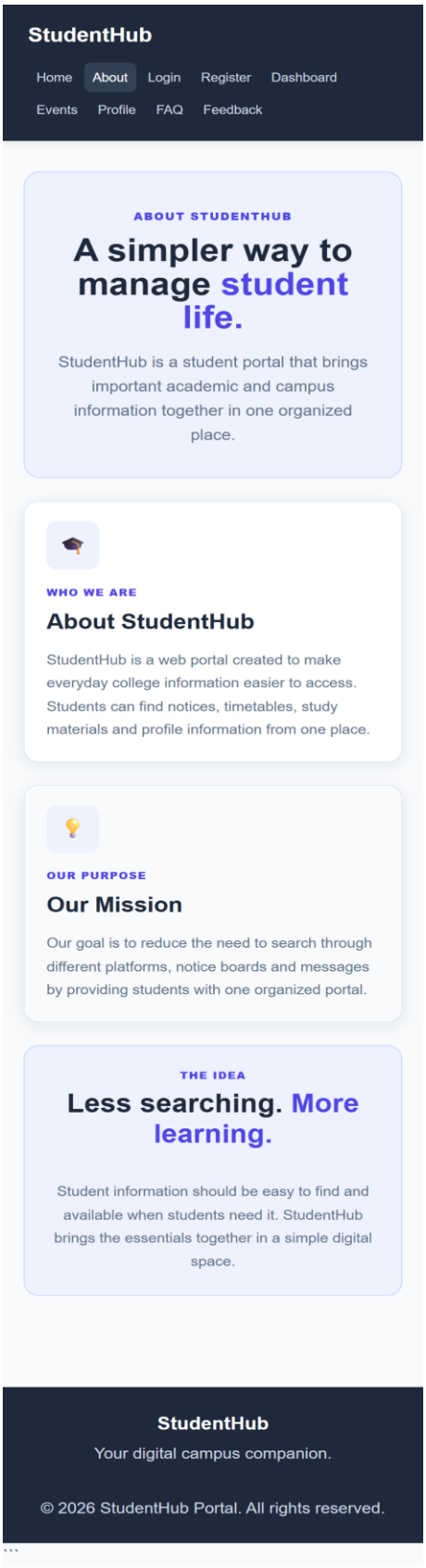
WEBSITE SCREENSHOTS

1. Home



Description: Introduces the portal and its main features.

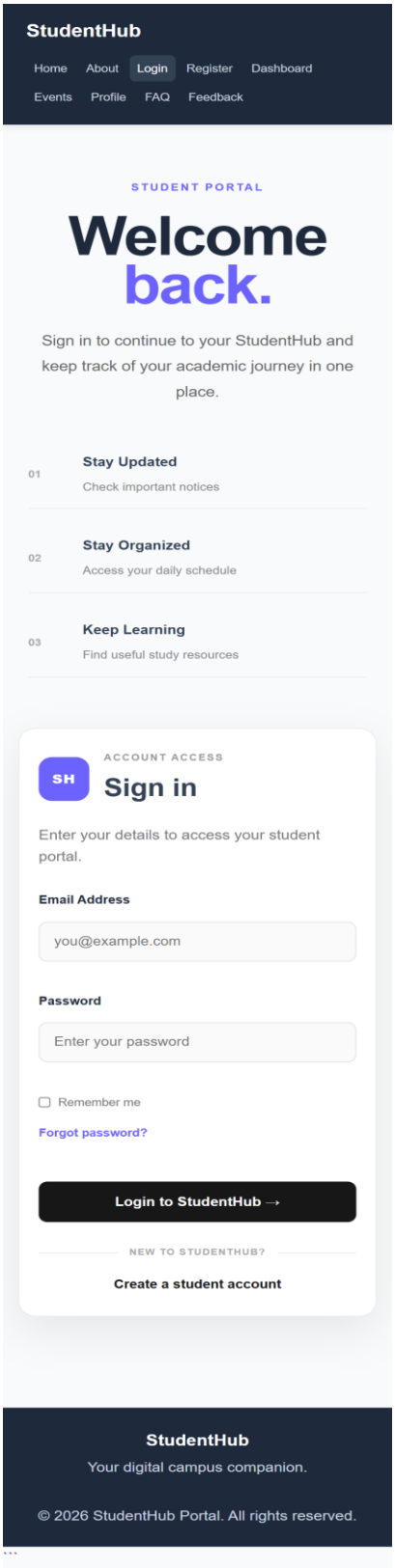
2. About



Description: Explains the purpose of StudentHub.

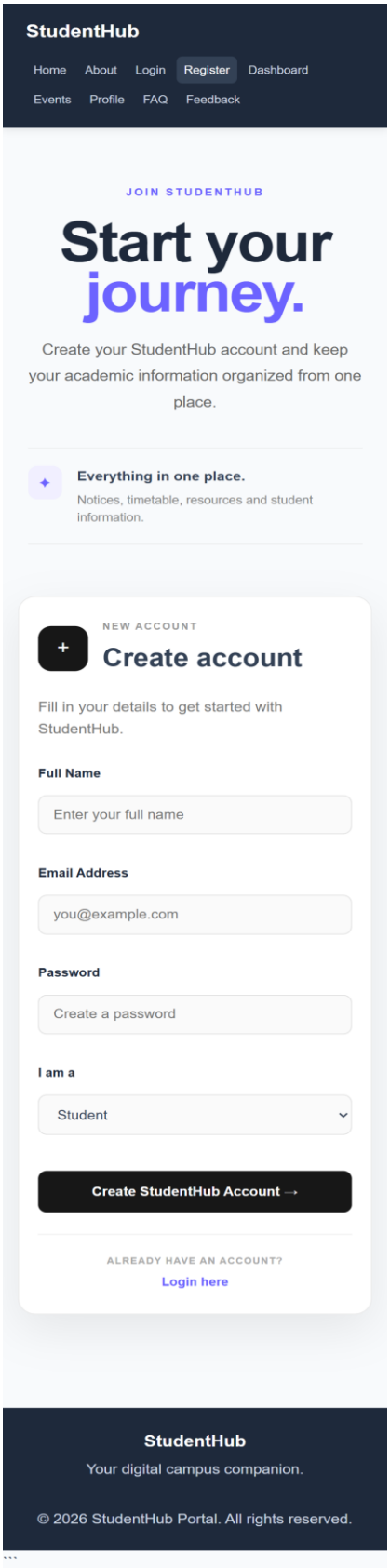
WEBSITE SCREENSHOTS

3. Login



Description: Provides the sign-in form interface.

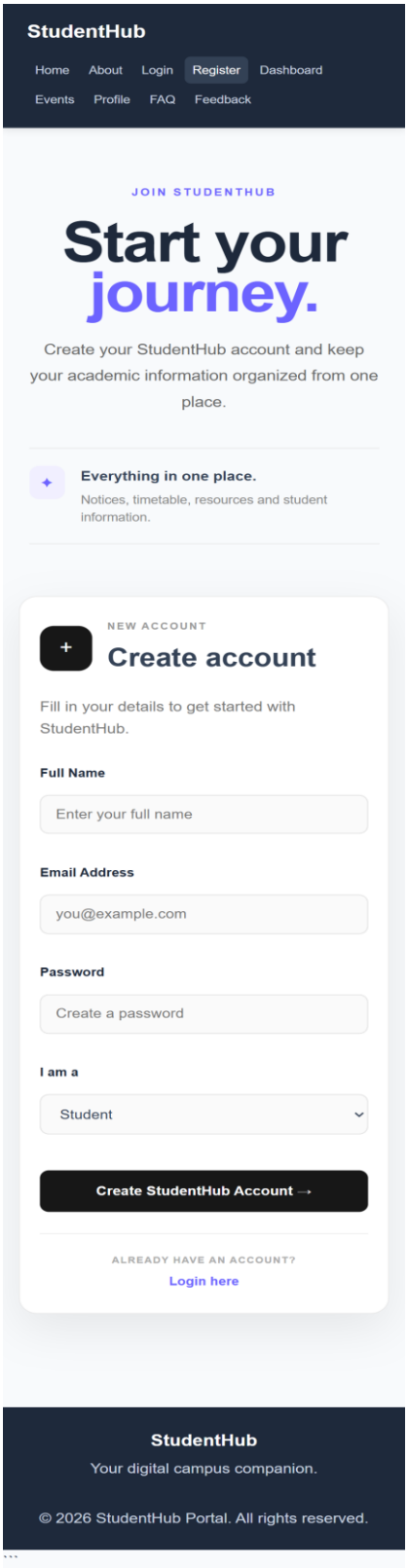
4. Register



Description: Provides the new-user registration form.

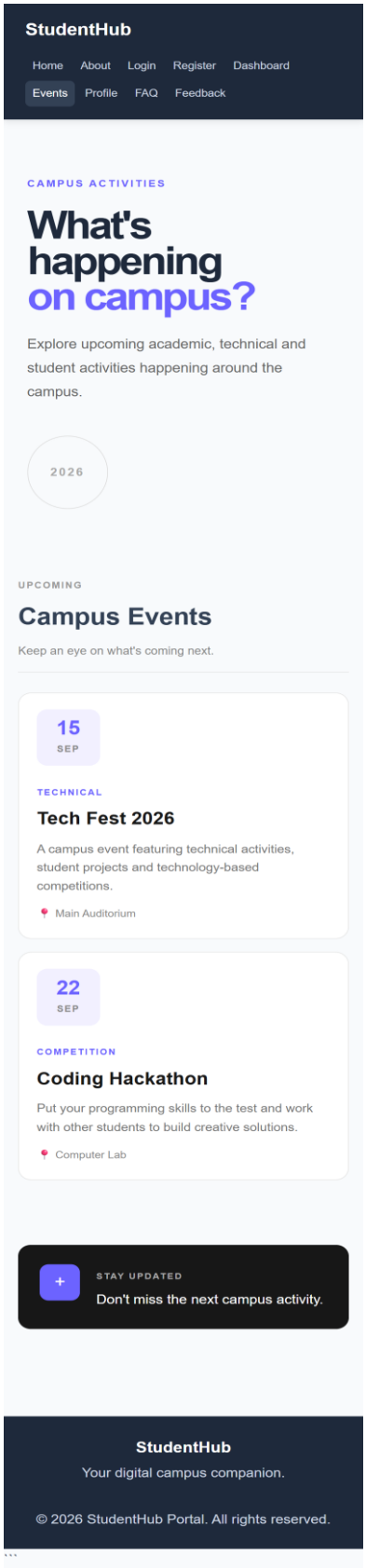
WEBSITE SCREENSHOTS

5. Dashboard



Description: Organizes academic notices and useful information.

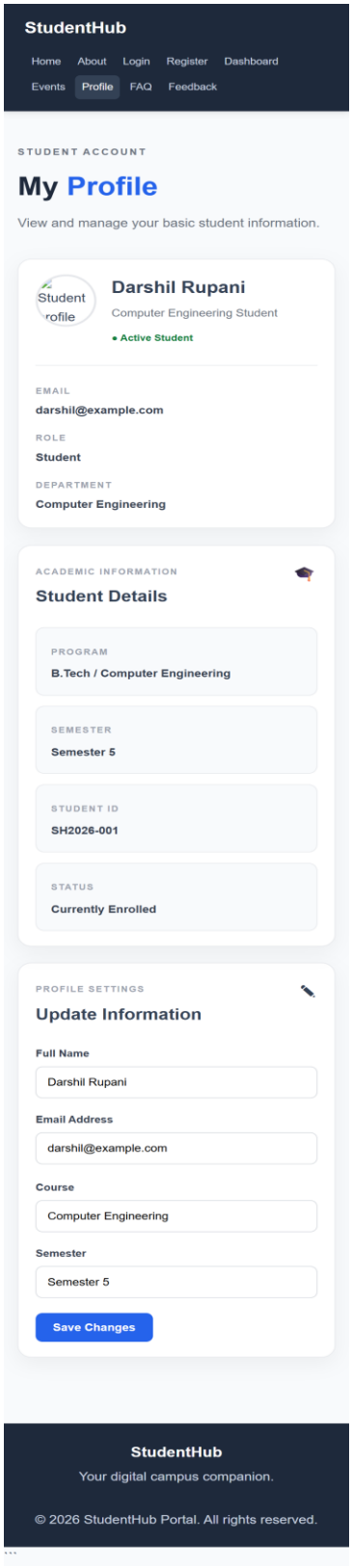
6. Events



Description: Displays campus events and their details.

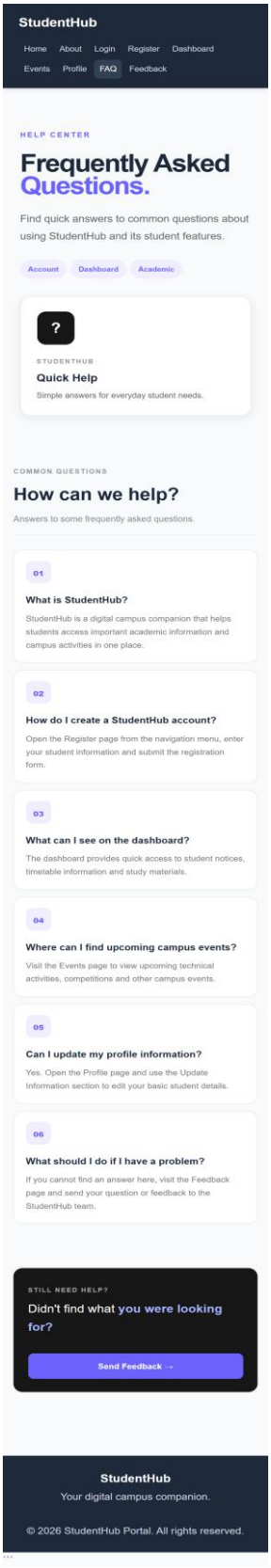
WEBSITE SCREENSHOTS

7. Profile



Description: Displays student profile information.

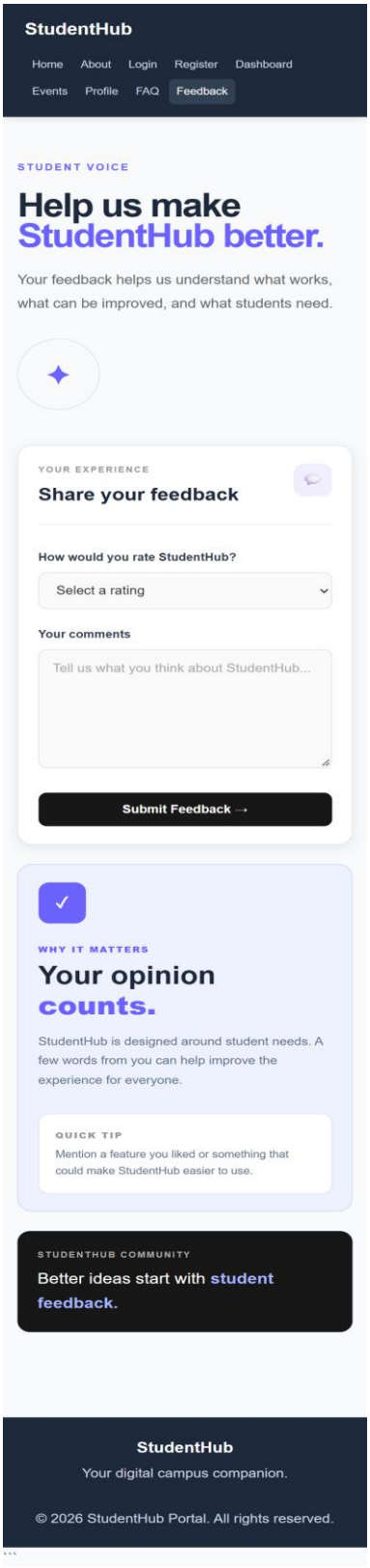
8. FAQ



Description: Answers common questions about the portal.

WEBSITE SCREENSHOTS

9. Feedback



Description: Provides a form for student feedback.

Accessibility Checklist

The following checklist defines the accessibility checks for the StudentHub HTML pages.

Item	Accessibility check
Semantic structure	Use header, nav, main and footer according to their purpose. Use section, article and aside only where they suit the content.
Headings	Use a clear main heading for each page and arrange section headings in a logical hierarchy.
Form labels	Give each form field a descriptive label. Associate the label with its input using matching for and id values or an enclosing label.
Image alternative text	Give informative images meaningful alt text. Use an empty alt attribute for purely decorative images.
Navigation links	Use descriptive link text and ensure links open the intended pages.
Consistent layout	Maintain a consistent header, navigation and footer across the pages.
Keyboard access	Make links and form controls reachable and usable with the keyboard. Keep keyboard focus visible.
Document information	Give each page a meaningful title and specify the document language using the lang attribute.

Checking Method:

Review the HTML source for semantic elements, labels and alt attributes. Open each page in the browser, follow its navigation links and use the Tab key to move between controls. Use browser accessibility tools to inspect headings and accessible names.

Key Questions and Answers

Q1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?

Proper use means choosing tags according to the meaning of the content. The header contains introductory content, nav contains navigation links, and main contains the main content. A section groups a related topic, an article contains self-contained content, an aside contains supporting information, and the footer contains closing information. Every page does not need every semantic tag.

Q2. Are form labels, headings, image alt text, and navigation links accessible?

Accessible forms use descriptive labels associated with their inputs. Headings follow a logical order. Informative images have meaningful alt text, while decorative images use empty alt text. Navigation links clearly describe their destinations and can be reached using the keyboard. These features are checked through source review and browser inspection.

Q3. Is the page structure consistent across all pages?

A consistent structure uses the same main layout across the portal: header, navigation, main content and footer. Menu names and link destinations remain consistent, while each page has its own title, heading and content. Comparing these elements across all ten pages helps verify consistency.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

Conclusion:

Semantic HTML5 provides a clear structure for the StudentHub portal. Linked navigation connects the pages, while meaningful headings, form labels and image alternatives support accessibility. The page screenshots, repository source code and accessibility checklist form the documentation for this practical.

PRACTICAL – 3

Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

AIM:

To design responsive layouts for the Home, About, Registration, Dashboard and Events pages of StudentHub using CSS Grid and Flexbox, so that the portal adapts correctly to mobile, tablet and desktop screens, following a mobile-first approach.

Introduction:

The pages built in Practical 2 used a single fixed layout. In this practical, CSS Grid, Flexbox and media queries were added so that the same HTML rearranges itself for different screen widths. The base CSS rules (written without any media query) target mobile screens first, and larger layouts are added afterwards using min-width media queries. No CSS framework such as Bootstrap or Tailwind was used; Grid and Flexbox alone were enough to build the layouts.

Tools and Technology:

VS Code, HTML5, CSS3 (Grid, Flexbox, media queries), Chrome DevTools responsive/device mode, Git and GitHub.

Layout Technique Used on Each Page:

Page	Layout technique	What changes across screen sizes
Home	CSS Grid (.features-grid) and Flexbox (.feature-card, header)	1 column on mobile → 2 on tablet → 3 on desktop; header switches from a stacked block to a single row
About	CSS Grid (.about-layout)	Single column on mobile and tablet; a 2fr / 1fr two-column grid from tablet width up
Registration	CSS Grid (.register-layout, .form-row)	Form and intro text stack in one column below 900px; form fields such as name/email sit two per row above that
Dashboard	CSS Grid (.dashboard-layout, .stats-grid)	Sidebar and content stack in one column on mobile; a 220px sidebar + content grid from tablet width, with a 4-column stats grid on tablet and up
Events	CSS Grid (.events-grid)	Single column of event cards on mobile and tablet, matching the available width at each size

Screenshots:

Each page below is shown at Desktop (about 1588px), Tablet (about 768px) and Mobile (about 375px) width.

Home Page

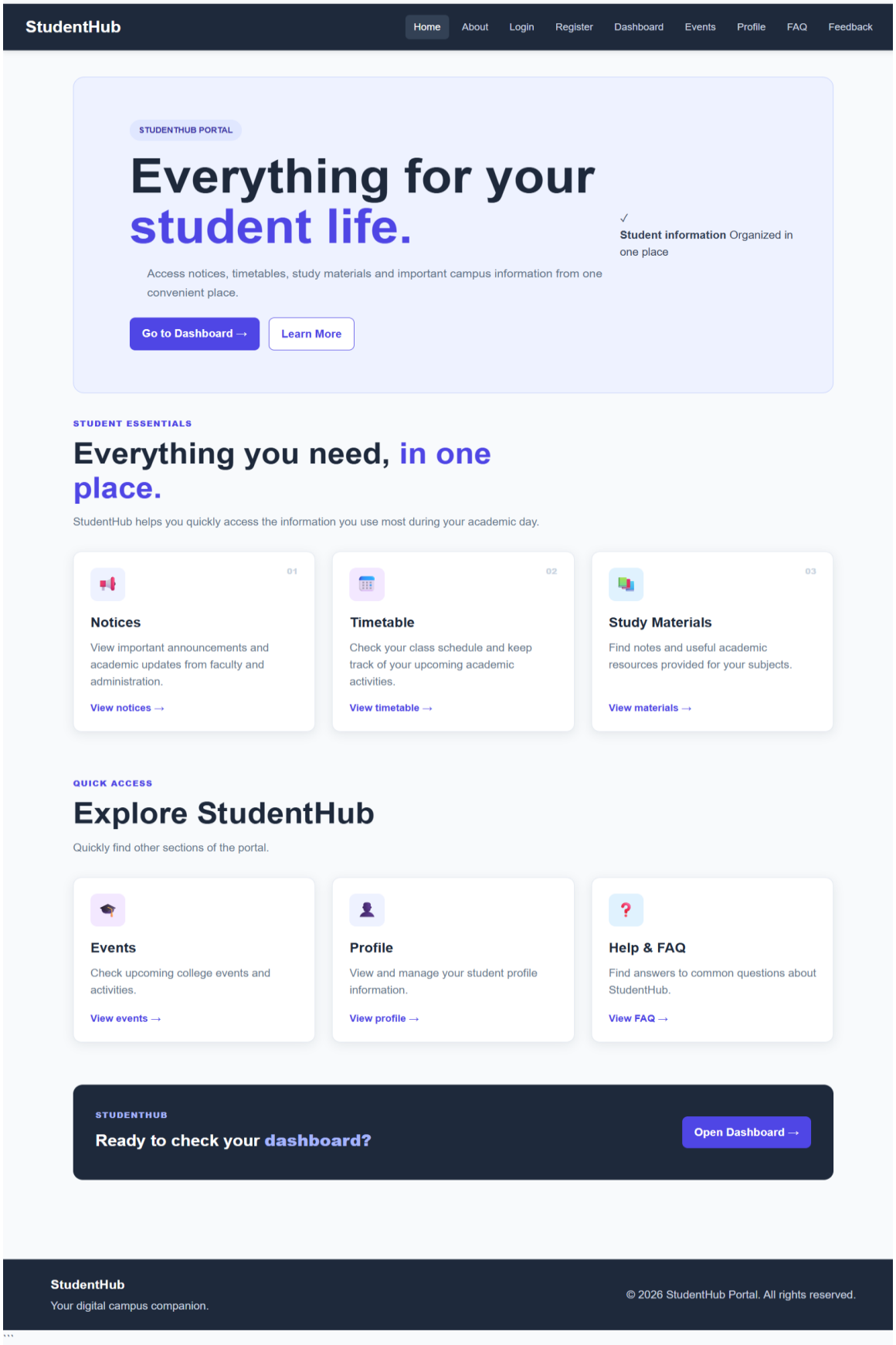


*Figure: Home page, **desktop** width*

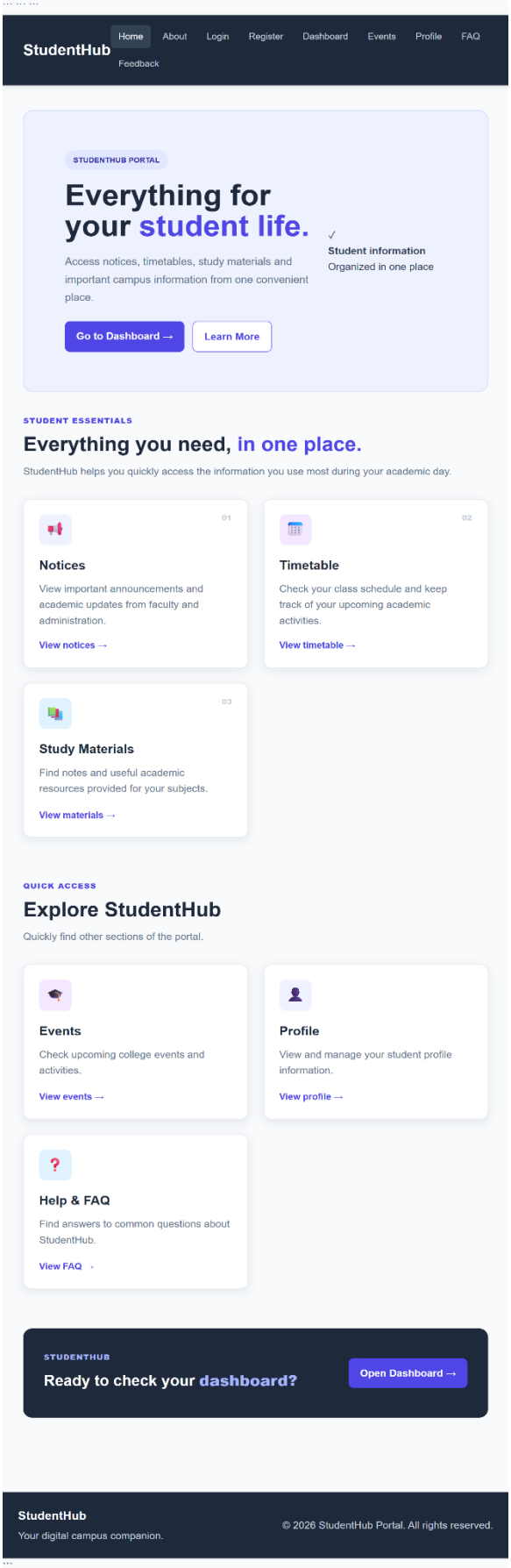


*Figure: Home page, **tablet** width*

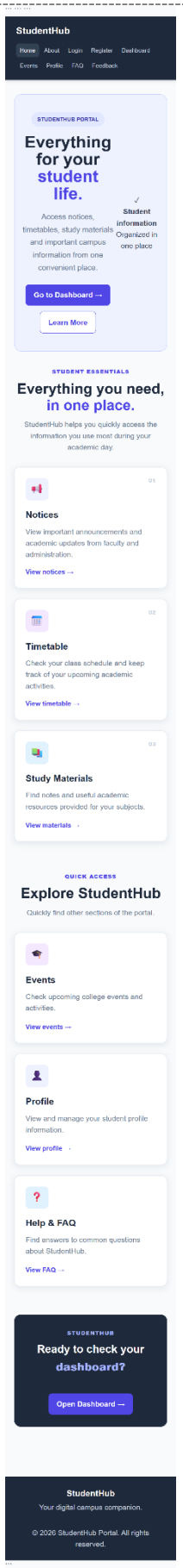


*Figure: Home page, **mobile** width*

About Page

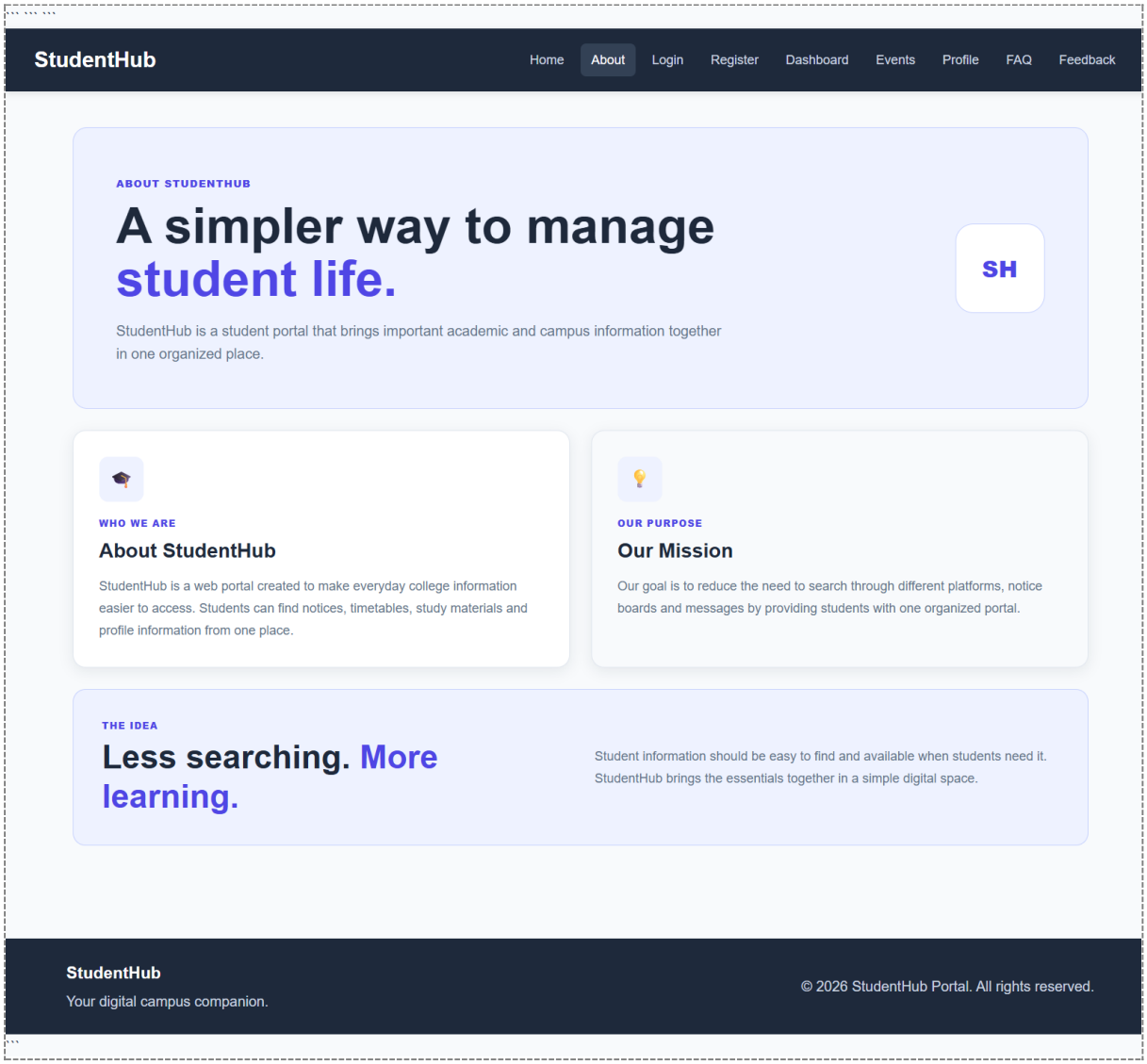


*Figure: About page, **desktop** width*

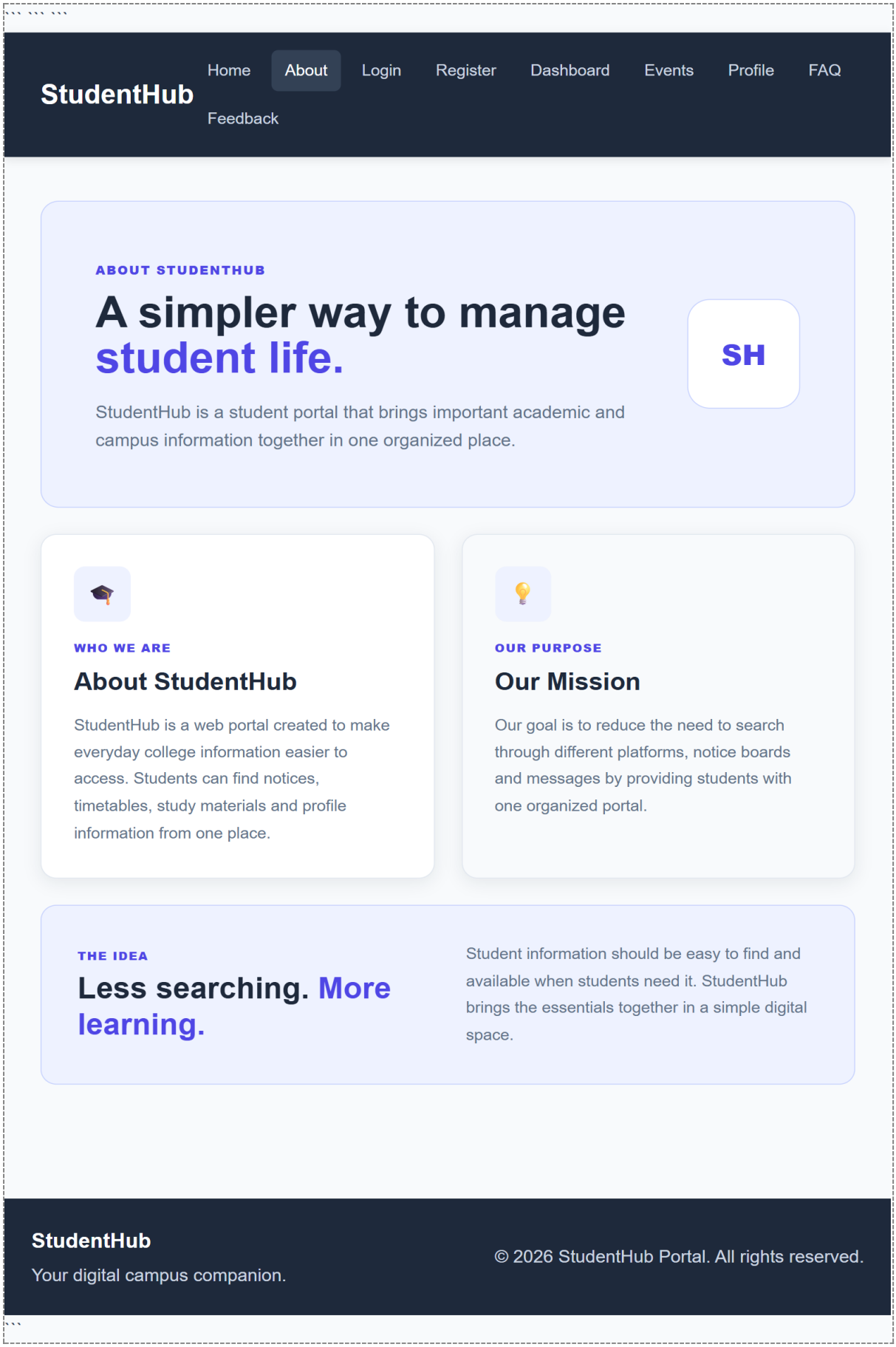


*Figure: About page, **tablet** width*

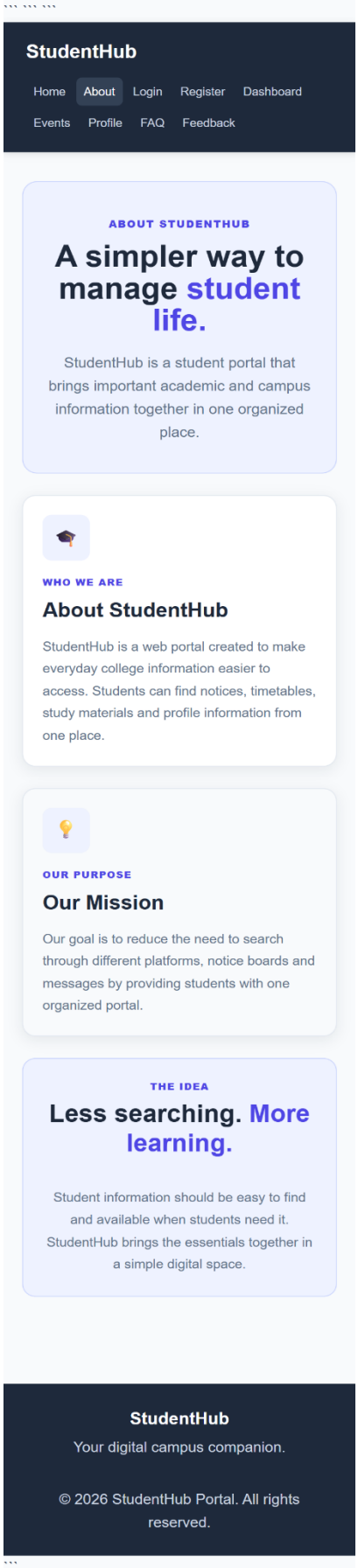


*Figure: About page, **mobile** width*

Registration Page

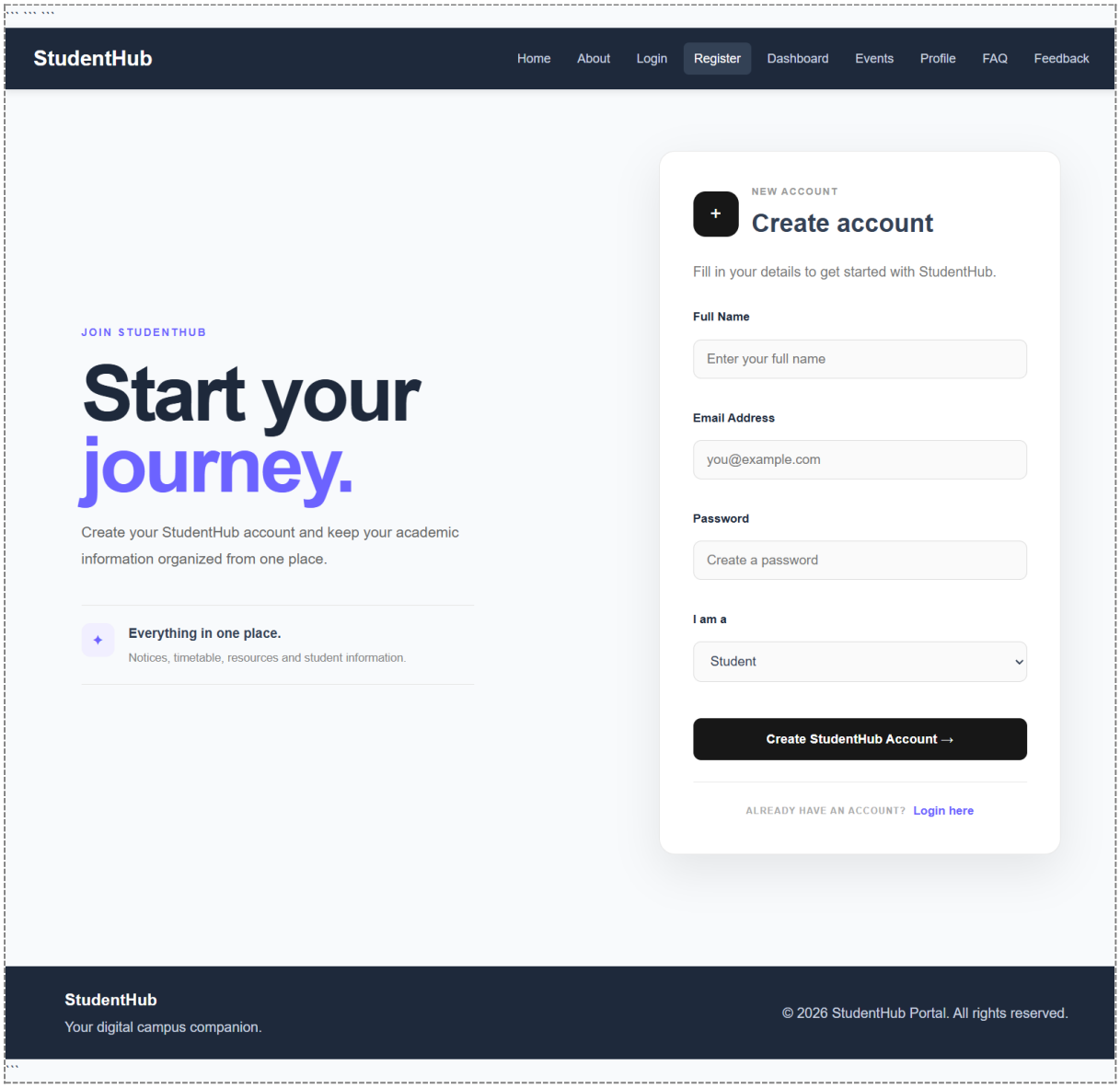


*Figure: Registration page, **desktop** width*

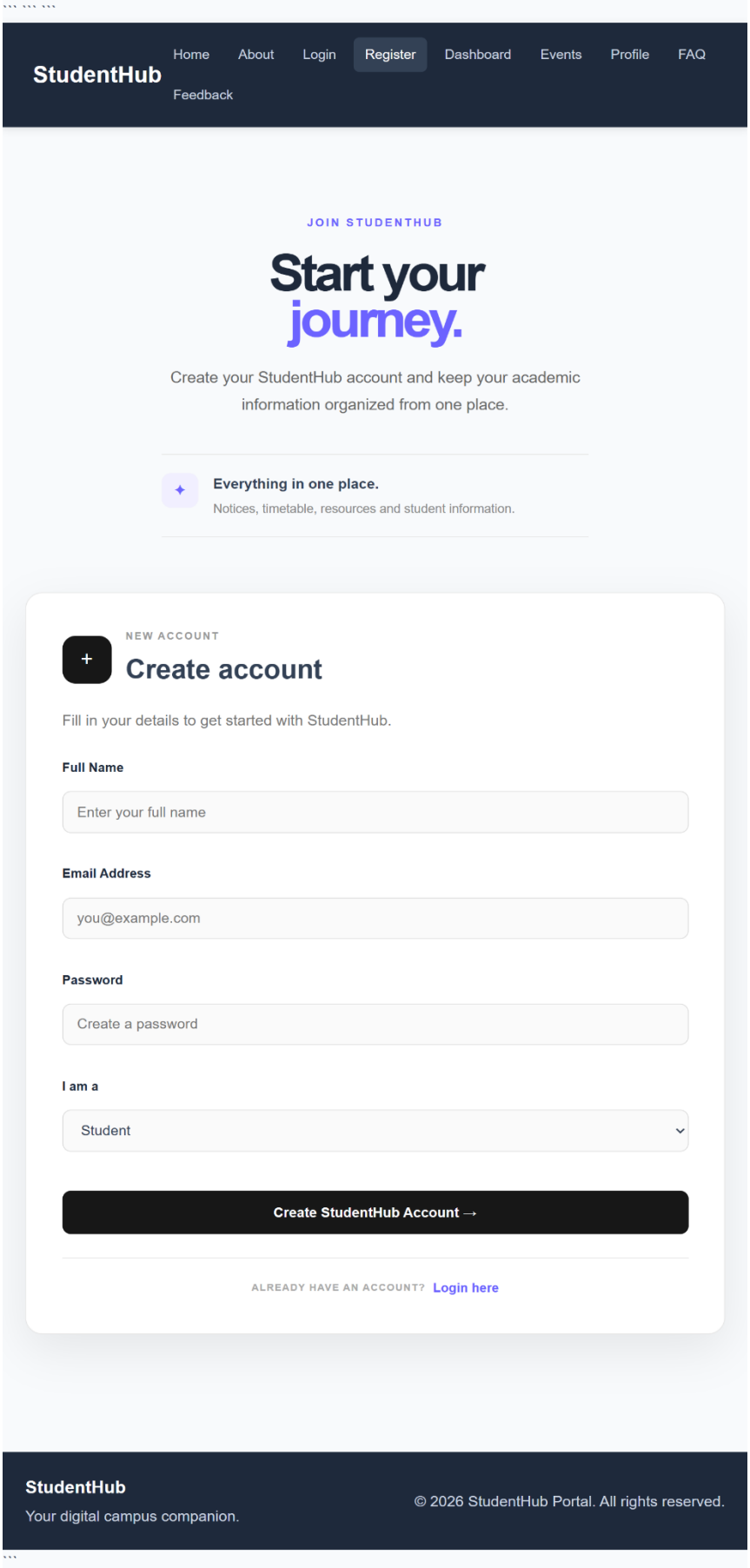


*Figure: Registration page, **tablet** width*

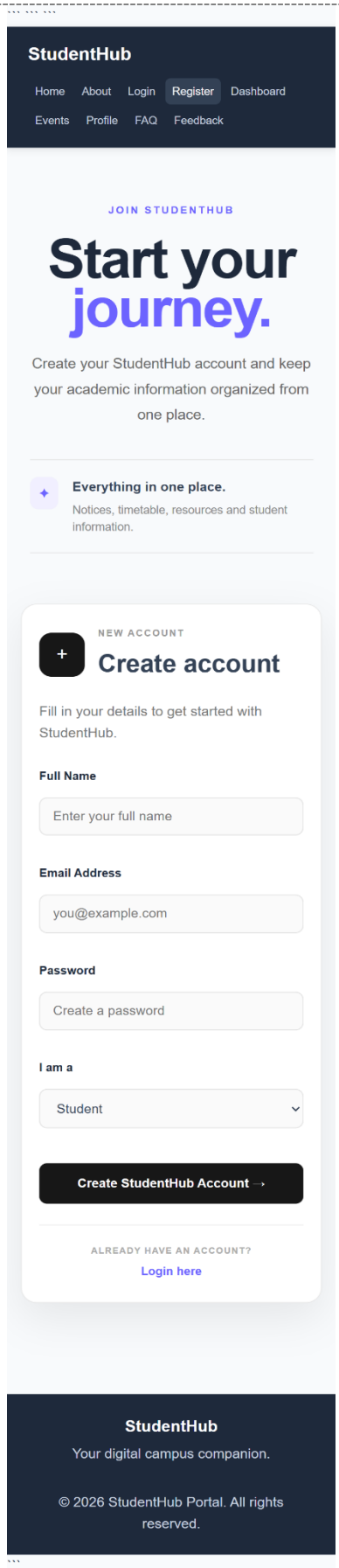


*Figure: Registration page, **mobile** width*

Dashboard Page

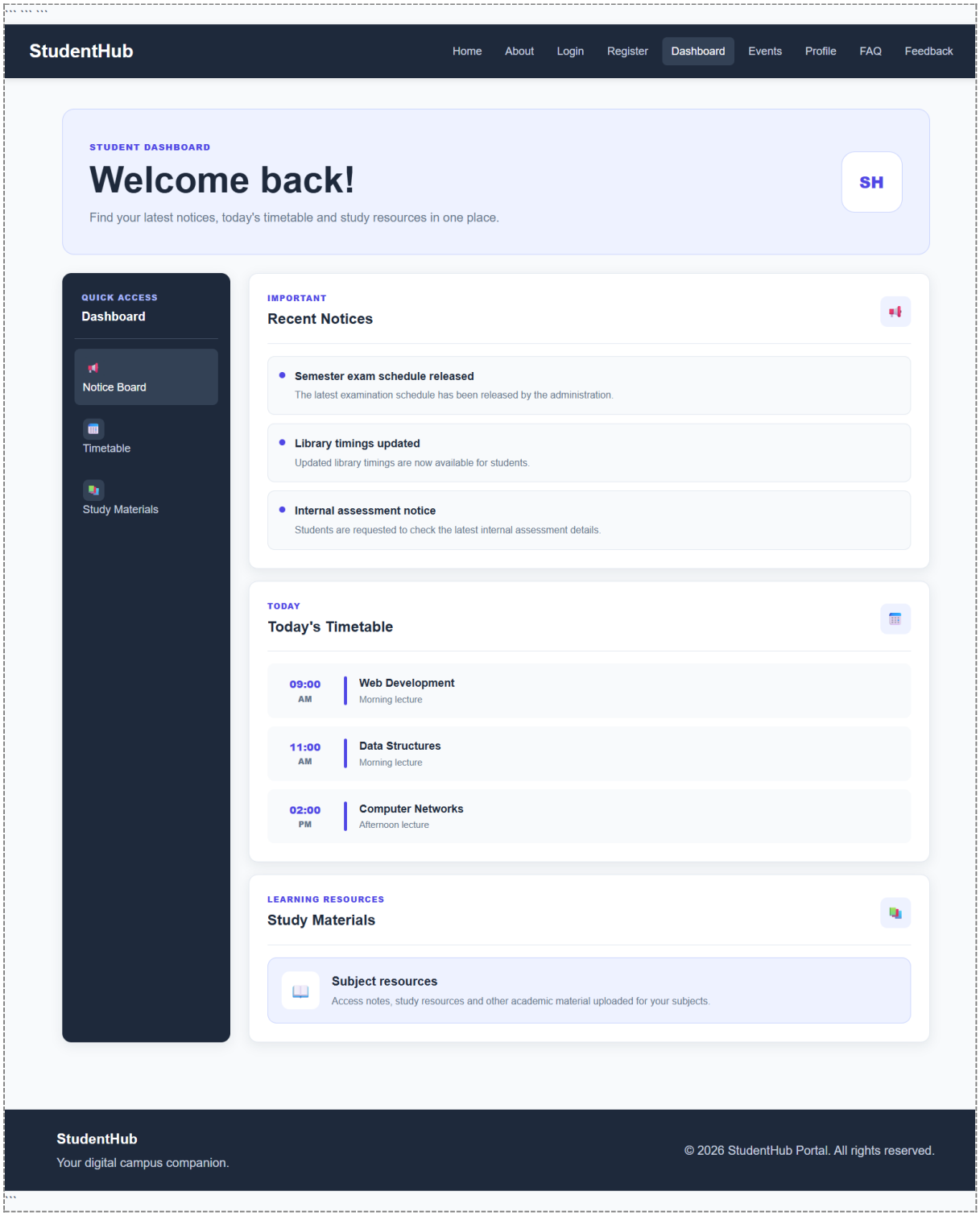


*Figure: Dashboard page, **desktop** width*

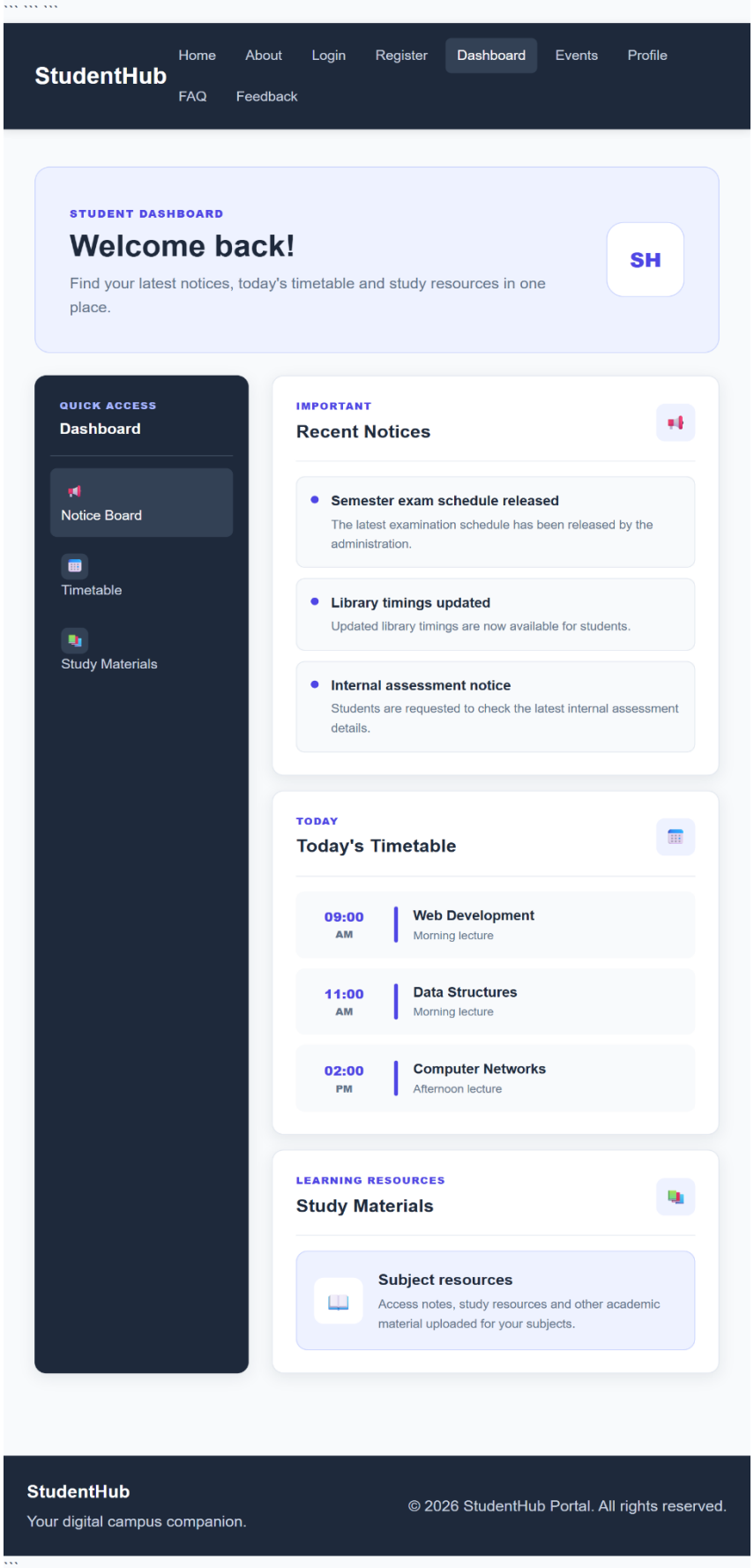


*Figure: Dashboard page, **tablet** width*

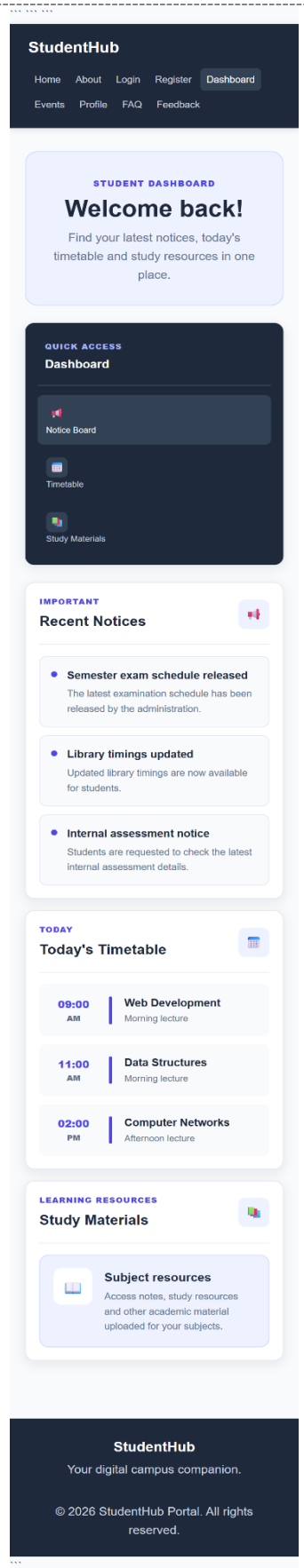


*Figure: Dashboard page, **mobile** width*

Events Page

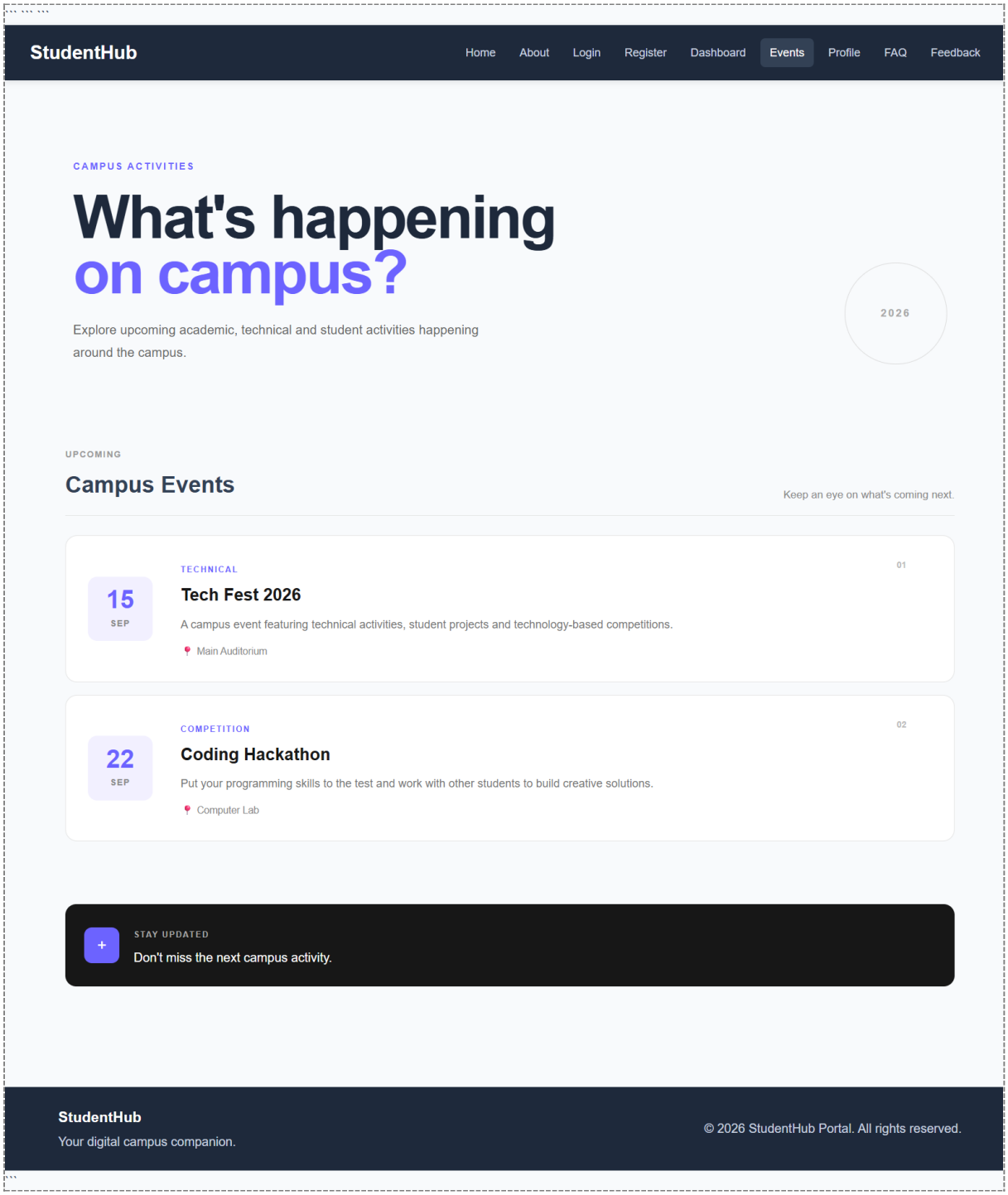


*Figure: Events page, **desktop** width*

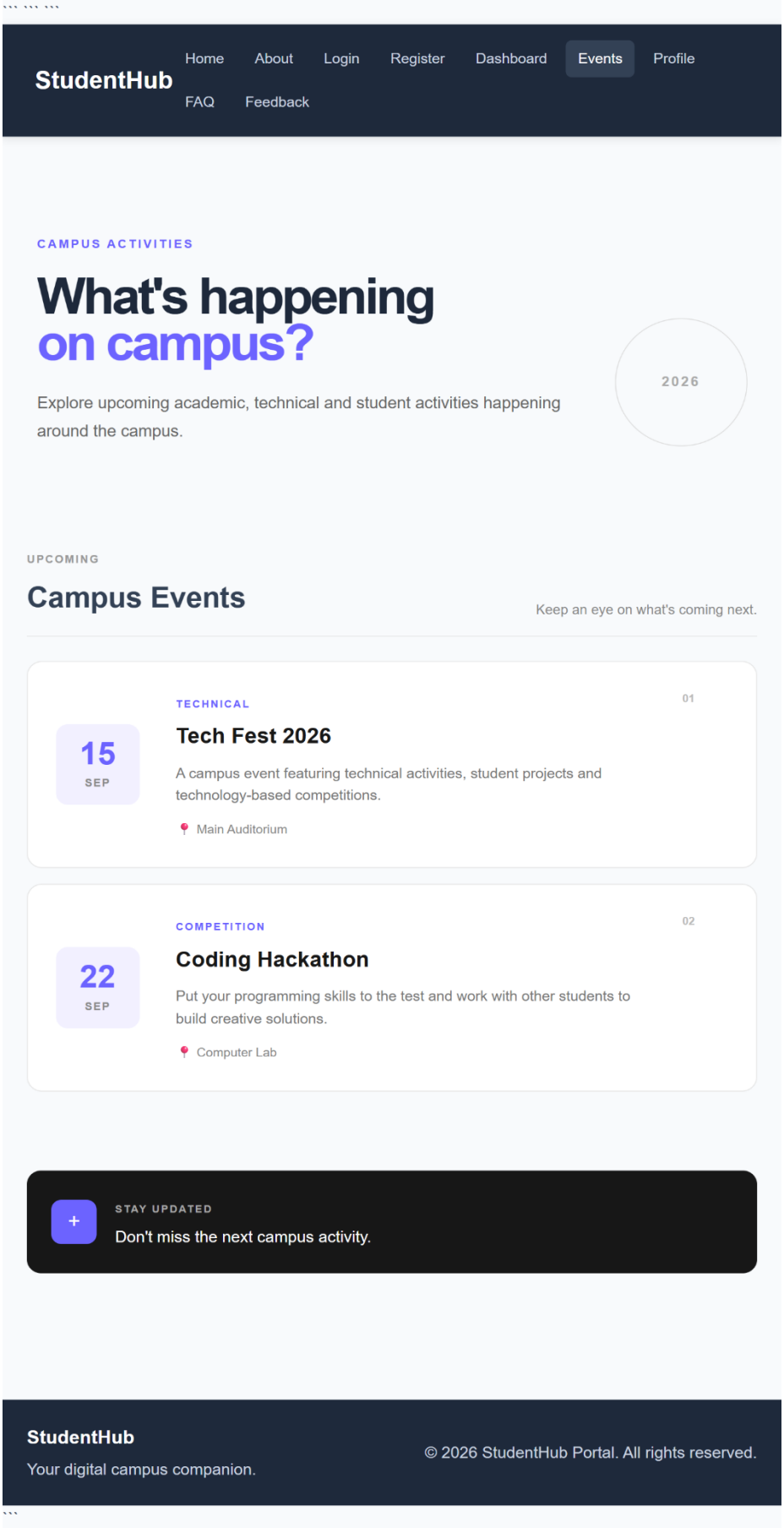


*Figure: Events page, **tablet** width*

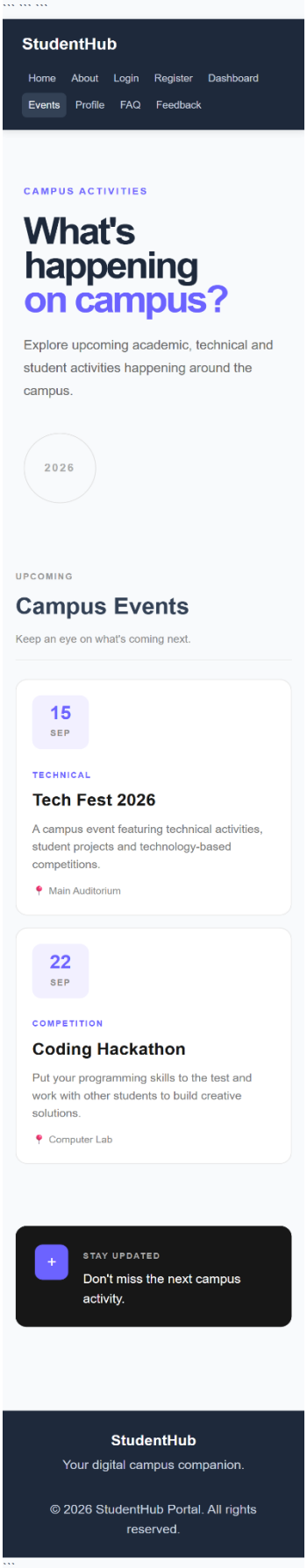


*Figure: Events page, **mobile** width*

Key Questions and Answers

Q1. How does the layout adapt for mobile, tablet and desktop screens?

The base CSS (outside any media query) targets mobile screens with single-column grids and stacked flex items. A media query at min-width: 768px changes selected grid-template-columns values to add a second column or a sidebar, for example turning .features-grid into two columns and .dashboard-layout into a 220px sidebar next to the content. A further media query at min-width: 1024px adds a third column to the features grid for desktop. Two max-width queries, at 900px and 767px, simplify layouts such as the registration form back to one column and adjust spacing once the screen is no longer wide enough for the tablet layout.

Q2. Which layout technique is used, and why?

CSS Grid is used for page-level sections that need rows and columns together, such as the feature cards, the about layout, the dashboard sidebar and the events list, because Grid can control both dimensions at once and change the column count with a single property. Flexbox is used for smaller, one-dimensional groupings, such as arranging the icon, heading, text and link inside a single feature card, and lining up the logo and navigation links in the header, because Flexbox handles alignment along one axis and vertical centring more simply than Grid.

Q3. Are typography, spacing, colours and contrast readable?

Font sizes and spacing use rem units, so they scale with the rest of the page instead of staying fixed in pixels. Padding increases at the tablet breakpoint so that sections do not look cramped on larger screens. Text and background colours are taken from the same CSS variables used across every StudentHub page, which keeps contrast consistent between pages and screen sizes.

Q4. Is the framework used consistently, without unnecessary inline CSS?

No CSS framework such as Bootstrap or Tailwind was used; the responsive behaviour was built directly with CSS Grid, Flexbox and media queries. All layout and spacing rules are kept in css/style.css, organised into numbered sections by page or component. No inline style attributes are used in the HTML files, which keeps the layout rules in one place and easy to update.

Post Laboratory Work:

Screenshots of the mobile, tablet and desktop views were captured for each page and are included above.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal
(FOR SOURCE CODE)

Conclusion:

CSS Grid and Flexbox were used to make the Home, About, Registration, Dashboard and Events pages responsive across mobile, tablet and desktop widths, following a mobile-first approach with min-width and max-width media queries. Typography and spacing were kept consistent using rem units and shared CSS variables, and no inline styles or CSS framework were needed. Contact and Feedback pages were added as the intermediate extension, using the same responsive patterns as the rest of the site.

PRACTICAL – 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

AIM:

To add dynamic user-interface components to the StudentHub portal using JavaScript: a collapsible FAQ, a modal popup, a content slider, a notification banner, a hamburger menu and a light/dark theme switcher, with the theme choice remembered using localStorage.

Introduction:

In the earlier practicals, StudentHub was built as a set of static HTML pages styled with CSS. This practical makes the pages interactive. JavaScript selects elements from the Document Object Model (DOM), listens for user actions such as clicks and key presses, and then changes the page by updating text, attributes and visibility. All ten pages load one shared script, js/script.js, so the same behaviour is available across the portal.

Tools and Technology:

VS Code, JavaScript (ES6+), HTML5, CSS3, a web browser with developer tools (Console), Git and GitHub.

Features Implemented:

Feature	What it does	Where it appears
Light/dark theme switcher	Switches the colour theme and remembers the choice using localStorage.	Header of all ten pages
Hamburger menu	Shows or hides the navigation links on narrow (mobile) screens.	Header of all ten pages
Collapsible FAQ	Expands and collapses each answer when its question is clicked.	FAQ page
Modal popup	Opens a “Quick guide” dialog over the page and closes it again.	Home page
Notification banner	Shows a welcome notice that the user can dismiss.	Home page
Content slider	Moves through three “Campus highlights” slides using Previous and Next.	Home page

All styling for these components is kept in the existing css/style.css file, in the section marked “Practical 4”. The script and style rules only act on elements that exist on a given page, so no page produces errors for features that belong to another page.

Screenshots:

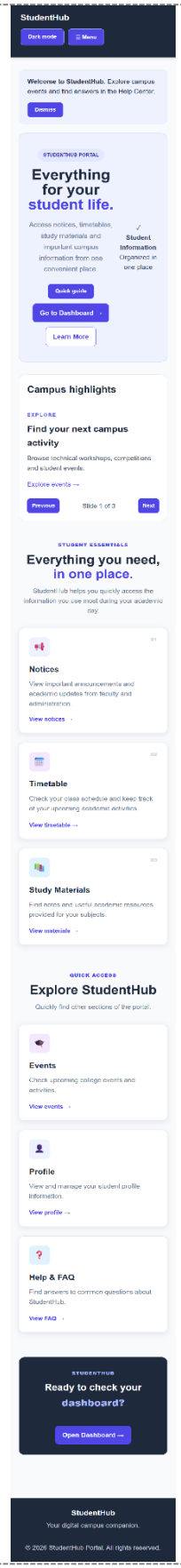


Figure 1(a): Home page in light mode

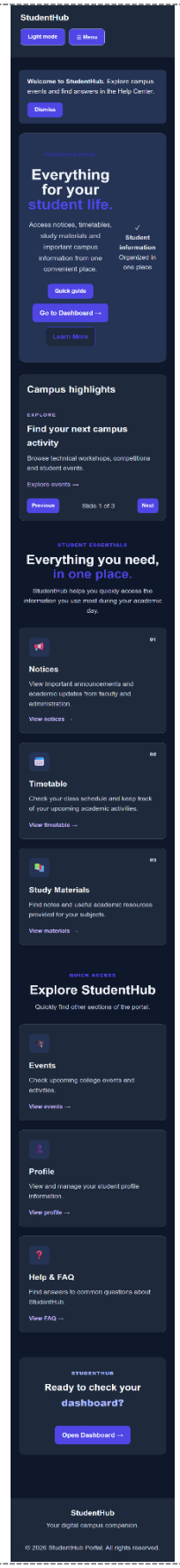


Figure 1(b): Home page in dark mode, still dark after refresh

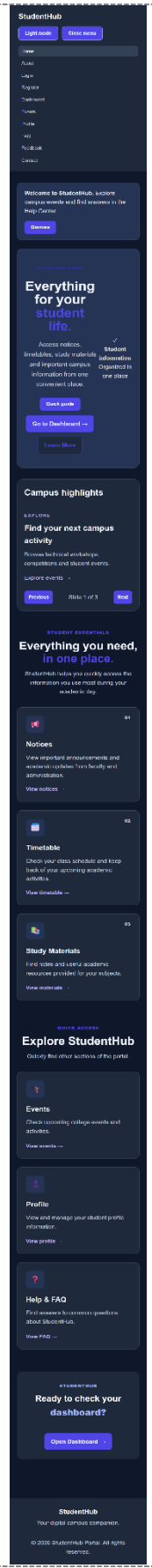


Figure 2(a): Mobile view with the menu closed

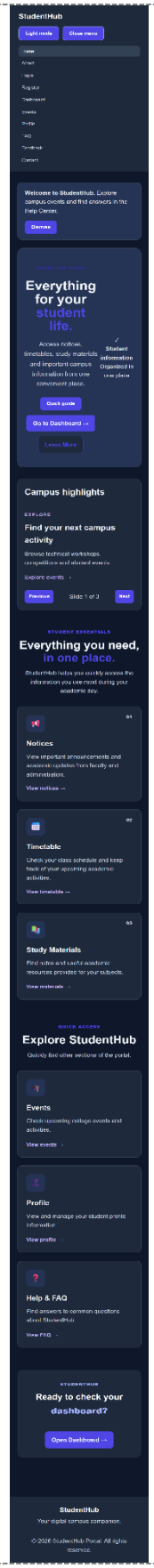


Figure 2(b): Mobile view with the menu open

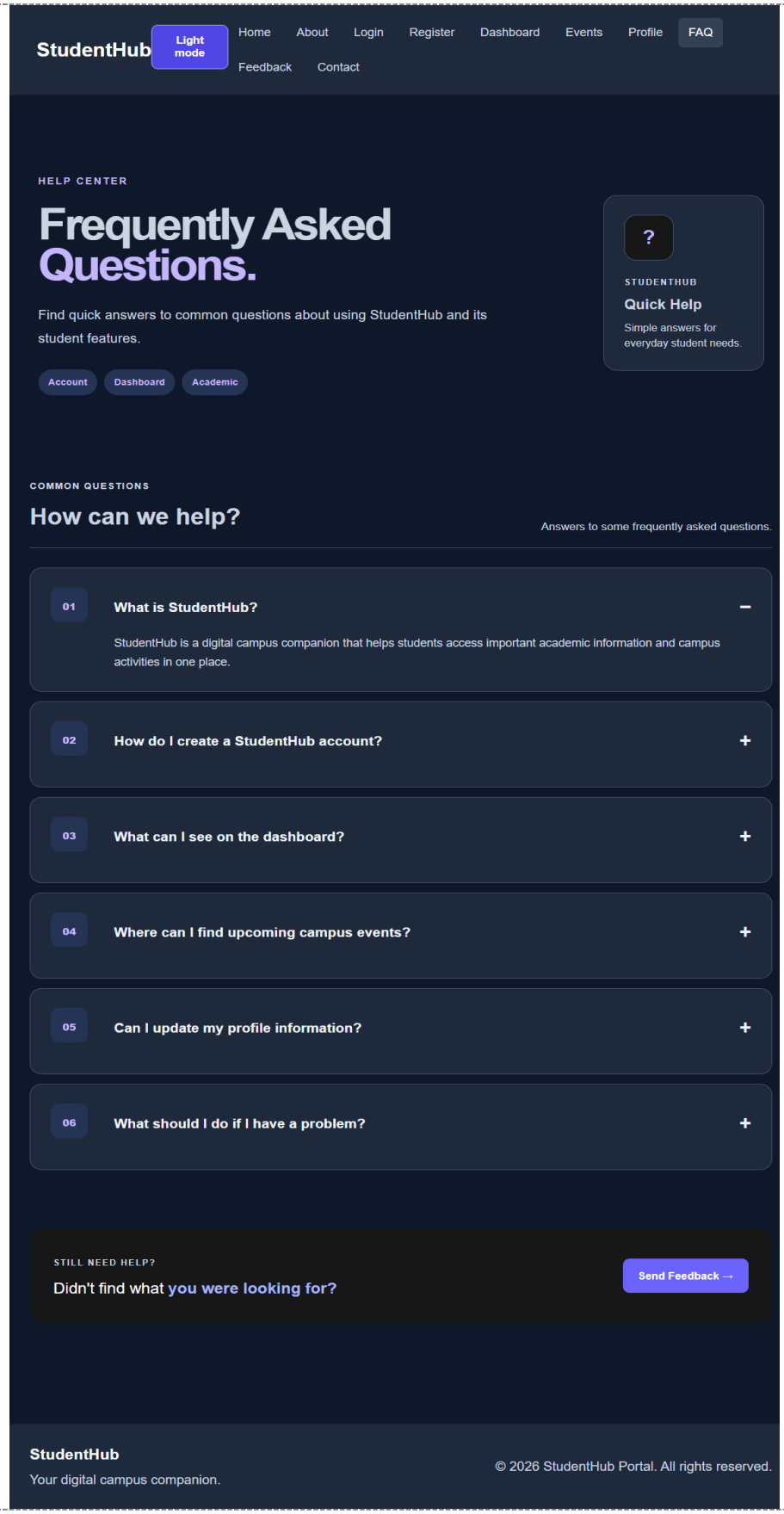


Figure 3: FAQ page showing an expanded answer

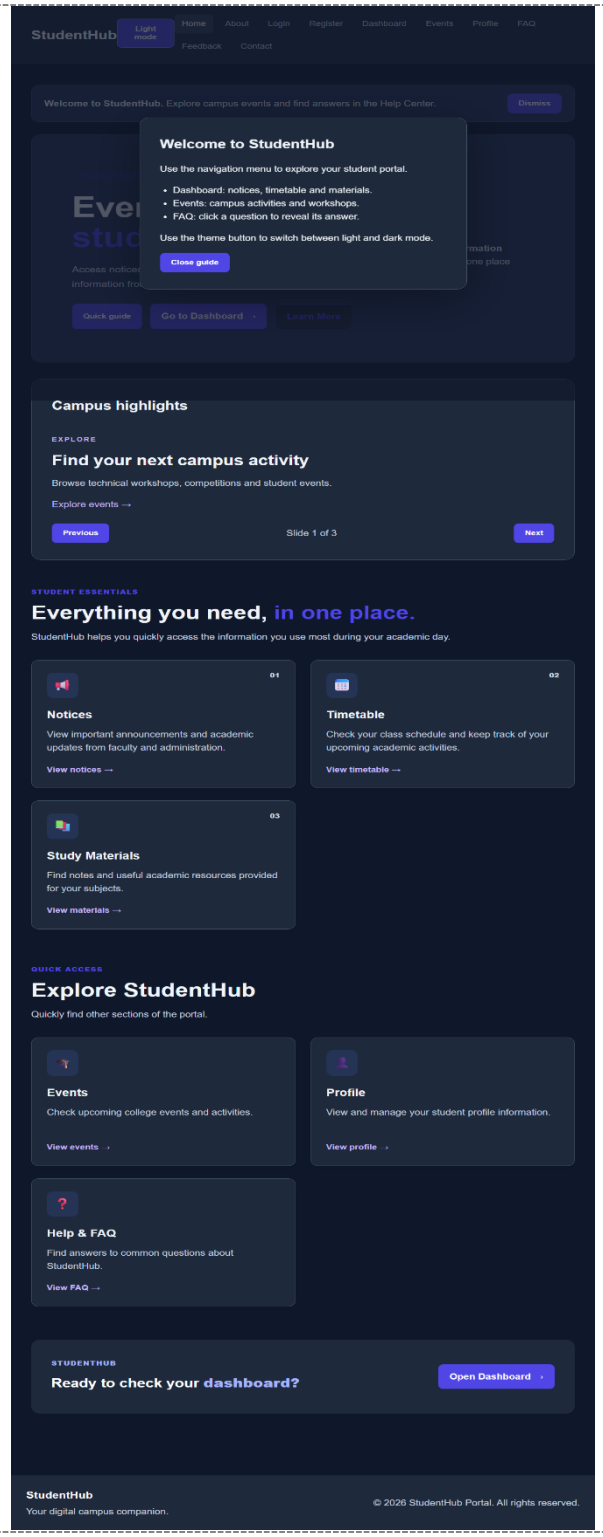


Figure 4: Modal popup displayed over the Home page

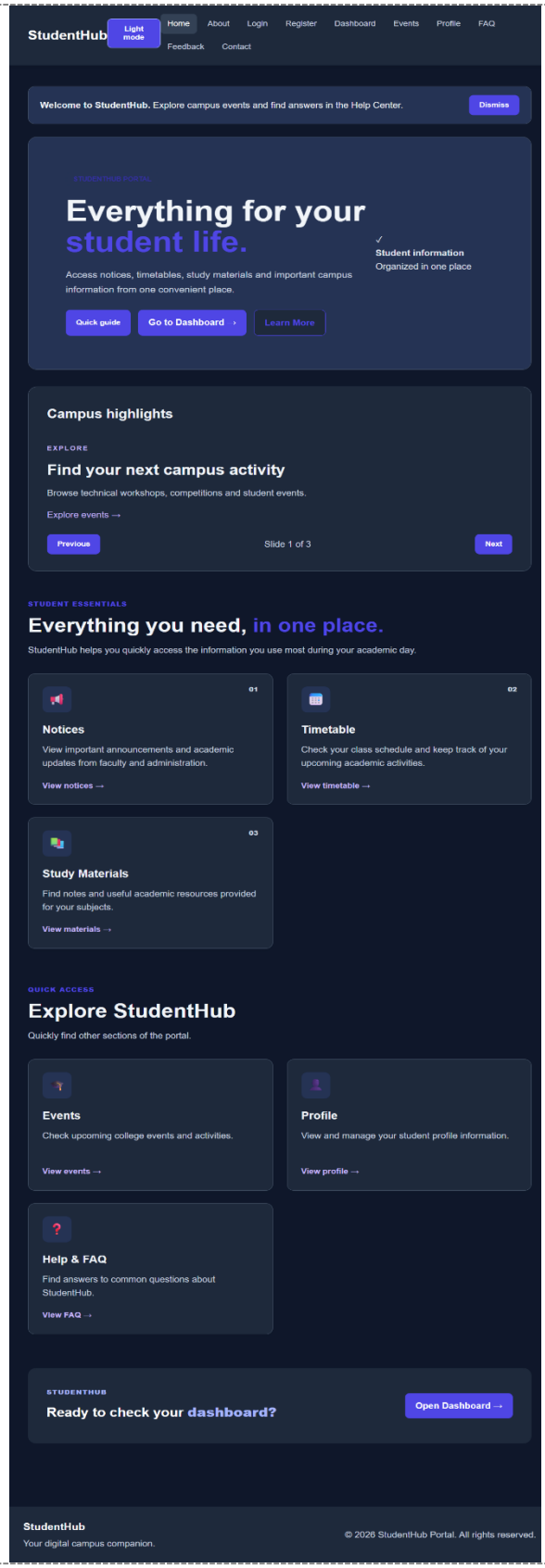


Figure 5(a): Notification banner visible

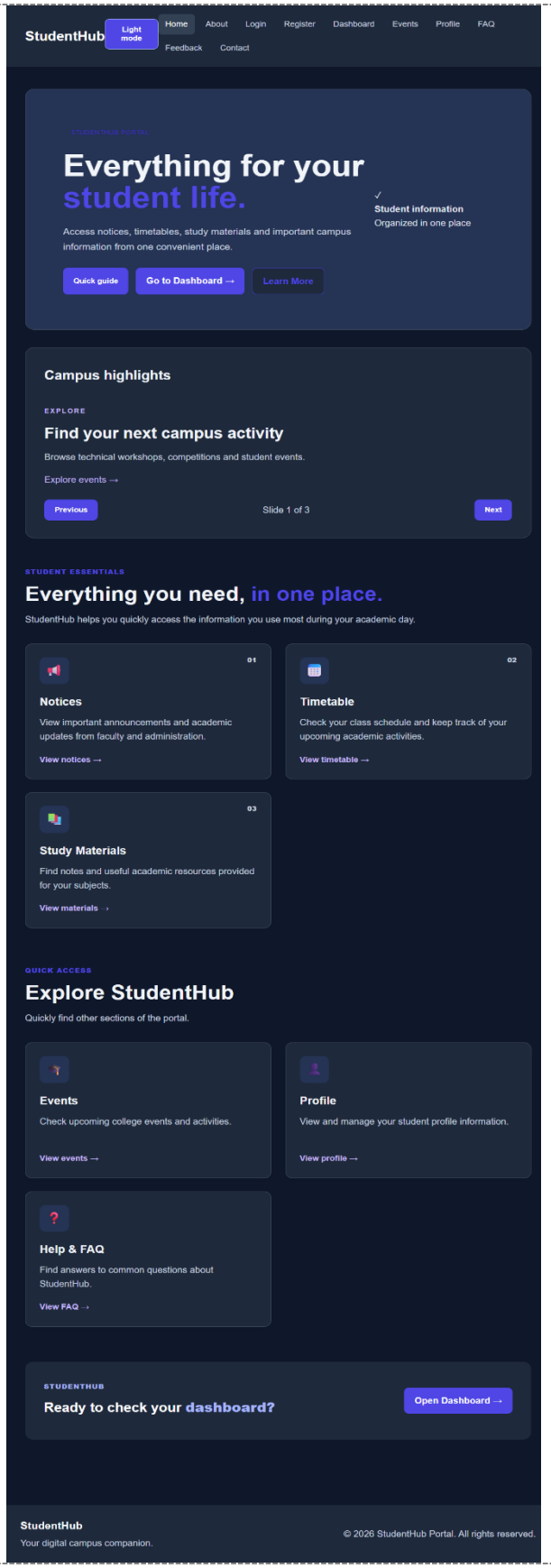


Figure 5(b): Notification banner dismissed

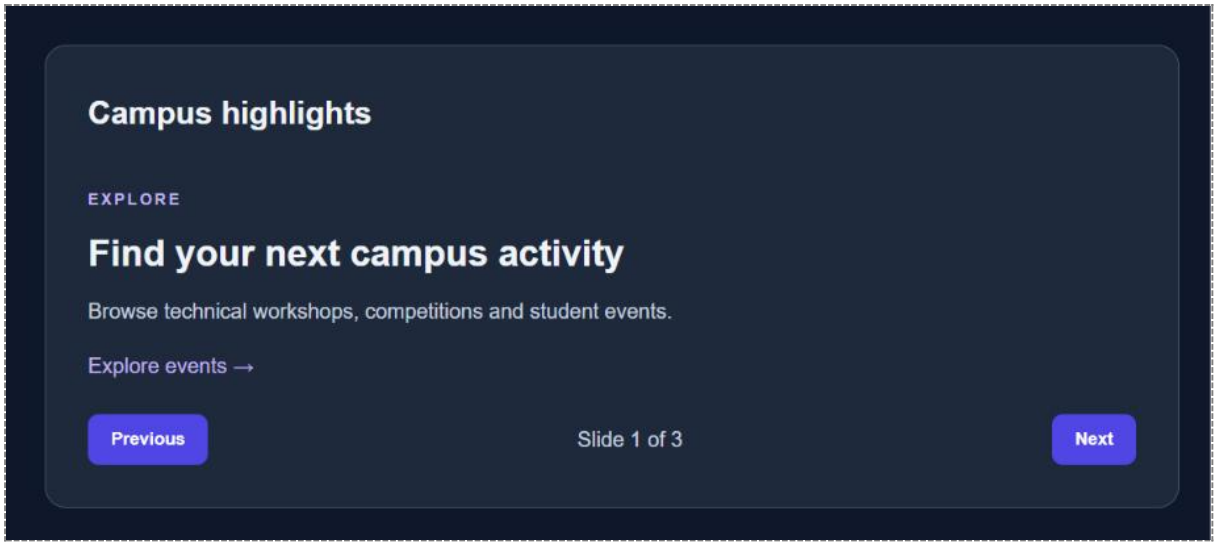


Figure 6(a): Slider showing the first slide

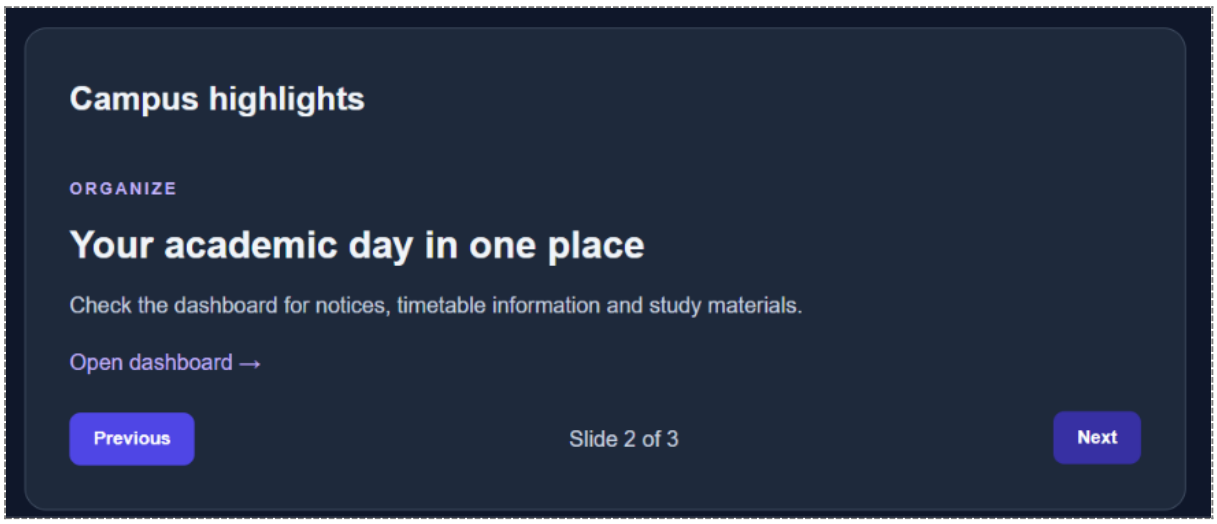


Figure 6(b): Slider after clicking Next

Key Questions and Answers

Q1. How are DOM elements selected and modified?

Elements are selected using document.querySelector() for a single element, document.querySelectorAll() for several, and document.getElementById() for a known id. Examples in the project are querySelector('#theme-toggle') and querySelectorAll('.faq-toggle'). Once selected, elements are modified by changing their textContent, the hidden property, attributes with setAttribute(), and data attributes with dataset. For example, the theme switcher changes document.documentElement.dataset.theme, and the FAQ changes an answer's hidden property.

Q2. Are event listeners attached correctly?

Yes. Listeners are attached with addEventListener() for click, keydown, change and close events. The script is loaded with the defer attribute, so the HTML is fully parsed before the script runs and every element already exists when a listener is attached. Each section first checks that its elements exist (for example if (guide && openGuide && closeGuide)), so the same script runs safely on all ten pages. Testing in the browser and Console showed that each control responds once per action.

Q3. Is localStorage used for remembering the theme choice?

Yes. When the user switches theme, localStorage.setItem('studenthub-theme', theme) saves it. On the next page load, localStorage.getItem('studenthub-theme') restores it before the theme is applied. Because localStorage belongs to the browser and stays after the tab is closed, the choice is kept across refreshes and across all StudentHub pages opened from the same address. The calls are placed inside try/catch so that the other components keep working if storage is blocked.

Q4. Does interactivity improve usability without breaking accessibility?

The features make the portal easier to use: dark mode reduces glare, the FAQ shortens long pages, the menu saves space on phones, and the slider shows more content in less space. Accessibility was preserved by:

- using native <button> and <dialog> elements, which support the keyboard and screen readers automatically;
- updating aria-expanded, aria-pressed and aria-controls whenever a control changes state;
- announcing slide changes with an aria-live status message;
- returning keyboard focus to a sensible control after the popup or notice closes;
- showing a clear focus outline on controls, and turning off animation for users who request reduced motion.

Post Laboratory Work:

Live demonstration of the StudentHub portal and explanation of the source code (js/script.js). The complete script (js/script.js) is available in the GitHub repository below.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

(FOR SOURCE CODE)



Figure 8: Practical 4 commit on GitHub (add after you push)

Conclusion:

JavaScript was used to add six interactive components to StudentHub: a theme switcher that remembers its setting, a hamburger menu, a collapsible FAQ, a modal popup, a dismissible notification and a content slider. DOM selection, event handling, class and attribute changes, localStorage and basic transitions were applied, and the components were built with native elements and ARIA attributes to stay accessible. The features were tested in the browser and the results are recorded above.

PRACTICAL – 5

Registration Form with Frontend Validation and User-Friendly Error Handling

AIM:

To build the StudentHub registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender and terms acceptance, using regular expressions to check each field and a password strength meter, while keeping the form accessible with the keyboard and with screen readers.

Introduction:

The registration form on register.html was extended with the fields required by the sheet and a new script, js/validation.js, that checks each field as the user types and again when the form is submitted. The browser's default validation pop-ups were turned off with the novalidate attribute so that custom error messages could be shown next to each field instead.

Tools and Technology:

VS Code, JavaScript (ES5/ES6), HTML5, CSS3, regular expressions, browser developer tools (Console), Git and GitHub.

Validation Rules Used:

Field	Rule	Regular expression / check
Full name	2–50 letters, spaces, dot, apostrophe or hyphen	/^[A-Za-z][A-Za-z .'-]{1,49}$/
Email	Standard text@domain.tld shape	/^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$/
Mobile number	10 digits, starting with 6, 7, 8 or 9	/^[6-9][0-9]{9}$/
Password	At least 8 characters, with uppercase, lowercase, a number and a symbol	checked with /[A-Z]/, /[a-z]/, /[0-9]/ and /[^A-Za-z0-9\s]/
Confirm password	Must match the password field exactly	string comparison
Course / Year / Gender	Must be selected, none left blank	required check
CAPTCHA	Typed text must match the code shown on the canvas	string comparison, not case sensitive
Terms	Checkbox must be ticked	required check

Every field has its own error message shown directly beneath it, and the field is marked with aria-invalid so screen readers announce the problem. Validation runs on keyup, input, change and blur so mistakes are caught immediately, and again on submit so nothing can be skipped.

Screenshots:

The empty form, each invalid case, and a fully valid submission were tested in the browser.

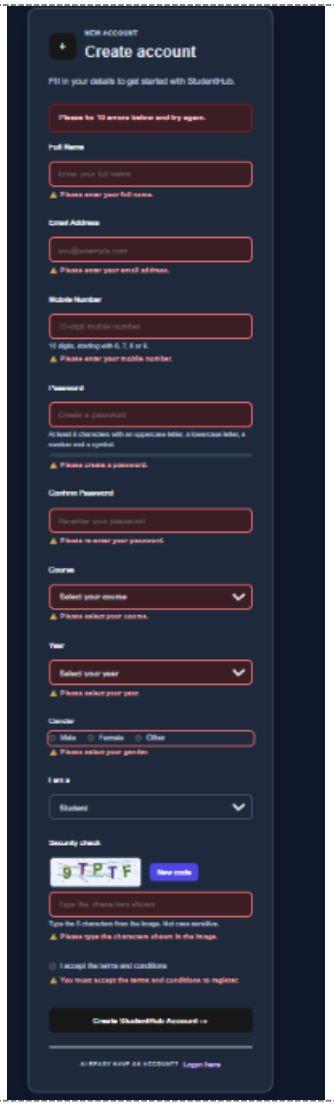


Figure 1: Empty submission showing every required-field error

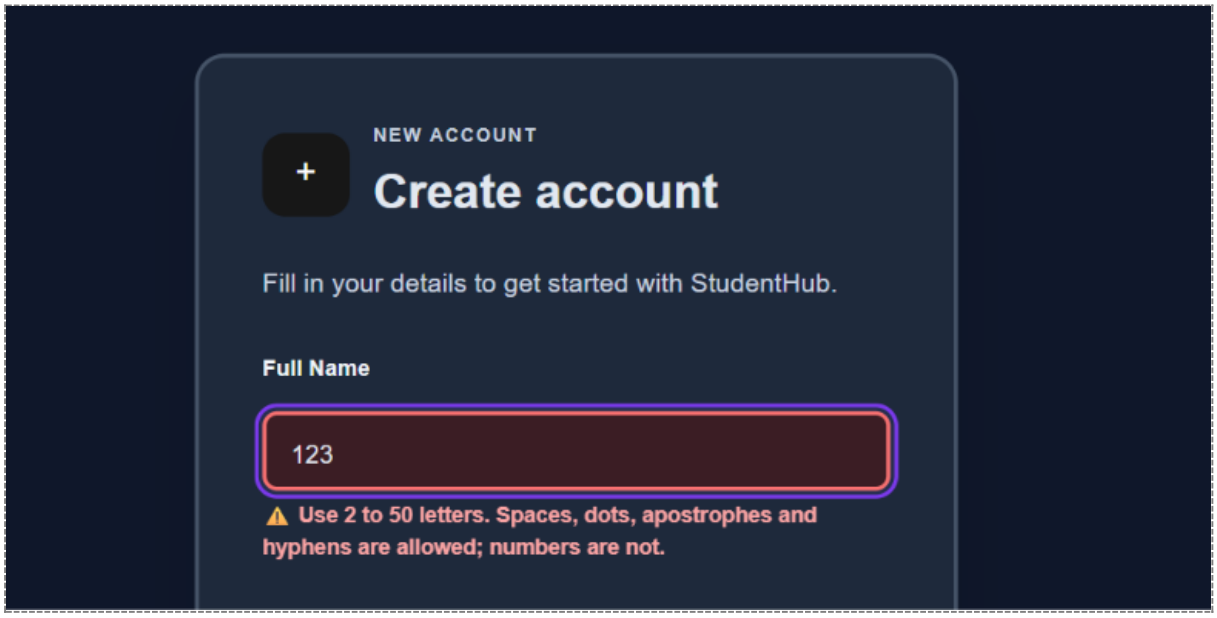


Figure 2: Name field rejecting digits

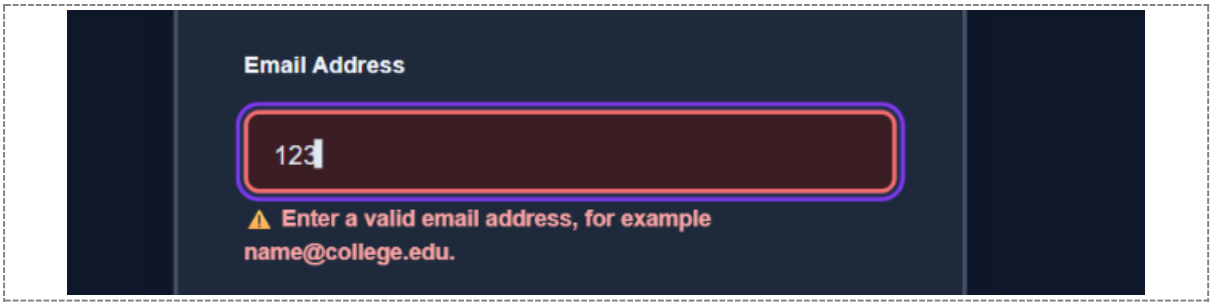


Figure 3: Email field rejecting a bad address

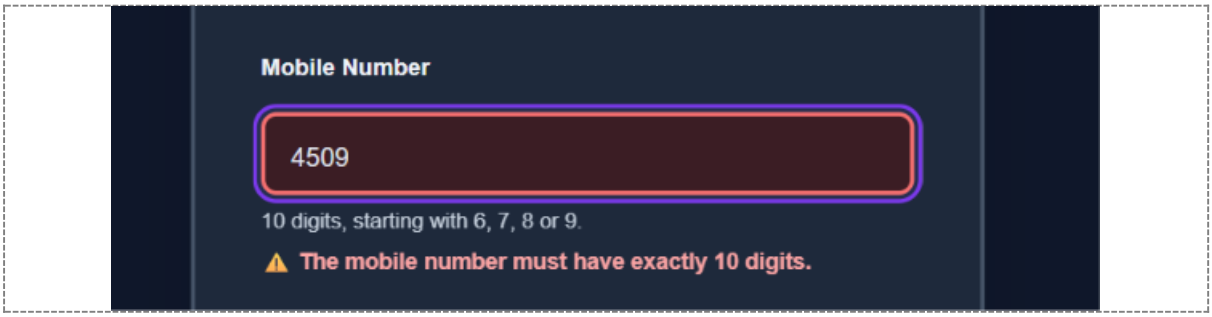


Figure 4: Mobile number validation

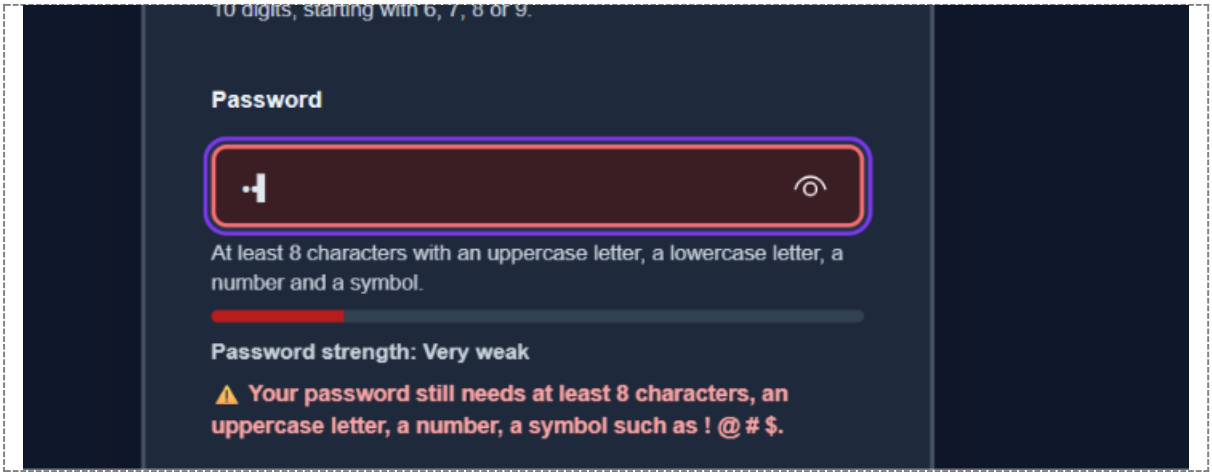


Figure 5(a): Password strength meter on a weak password

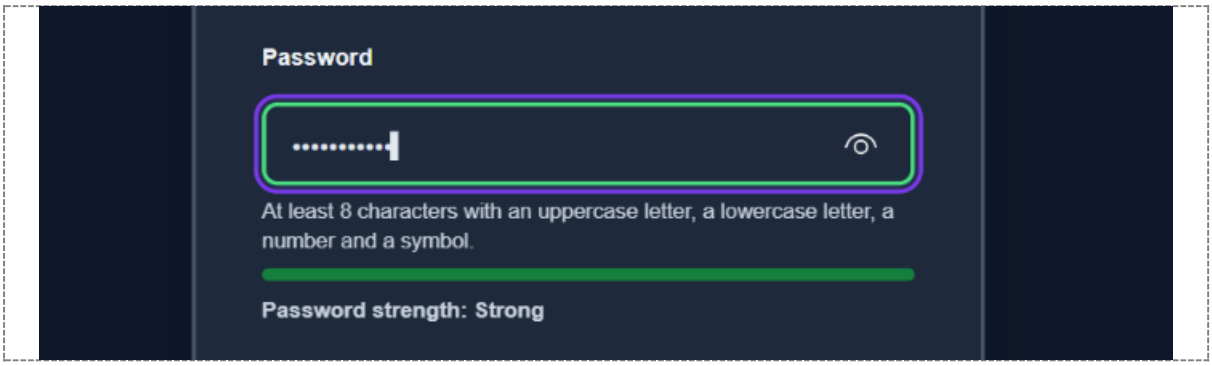


Figure 5(b): Password strength meter on a strong password

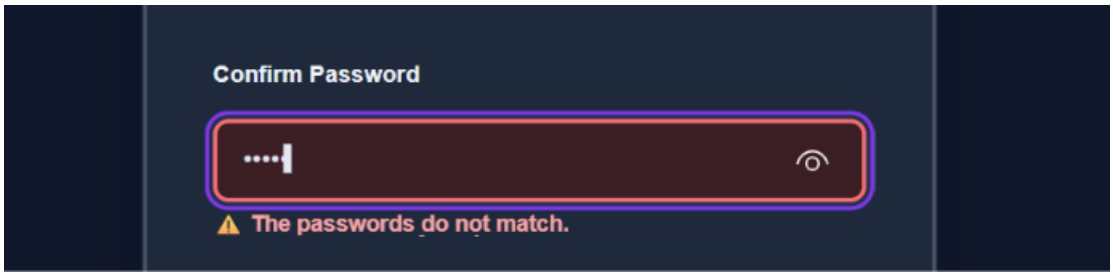


Figure 6: Confirm-password mismatch

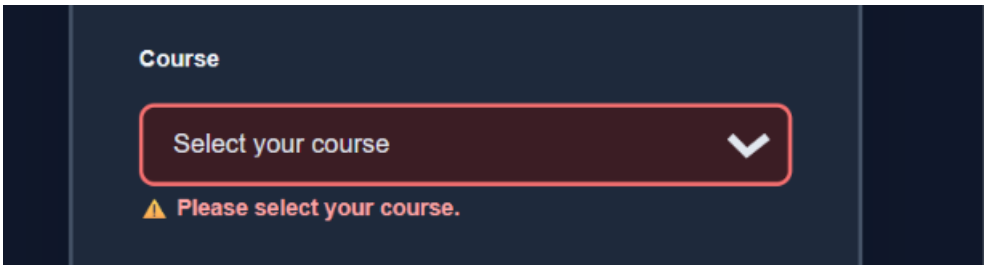


Figure 7: Course and year required errors

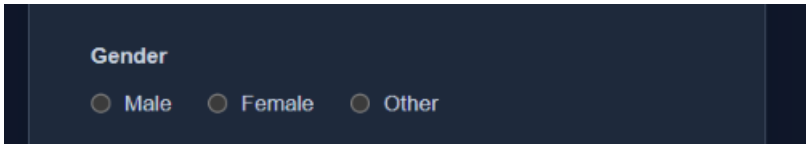


Figure 8: Gender required error

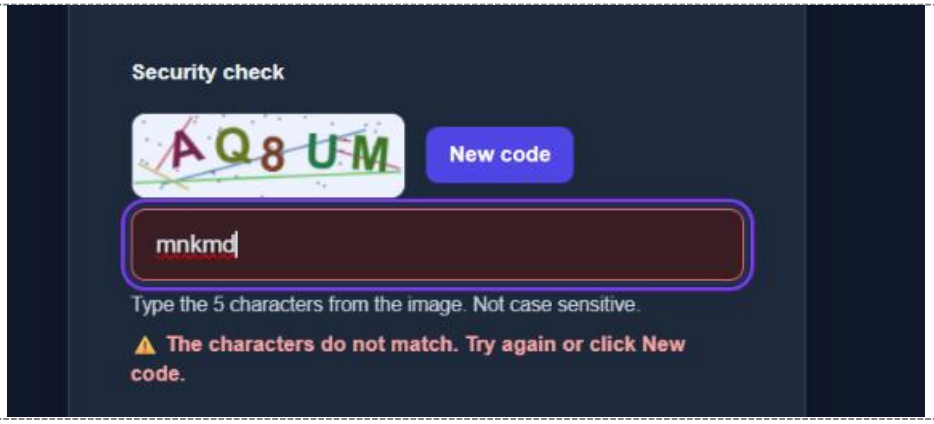


Figure 9: CAPTCHA mismatch

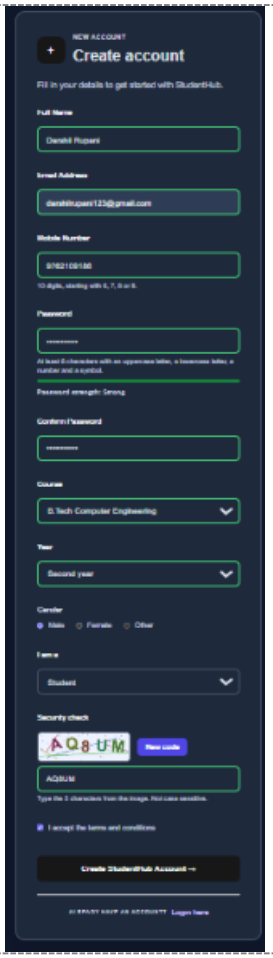


Figure 10: Terms and conditions required error

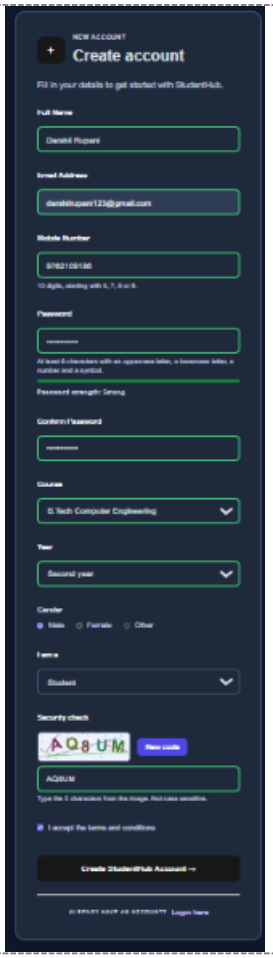


Figure 11: Successful registration after passing all checks

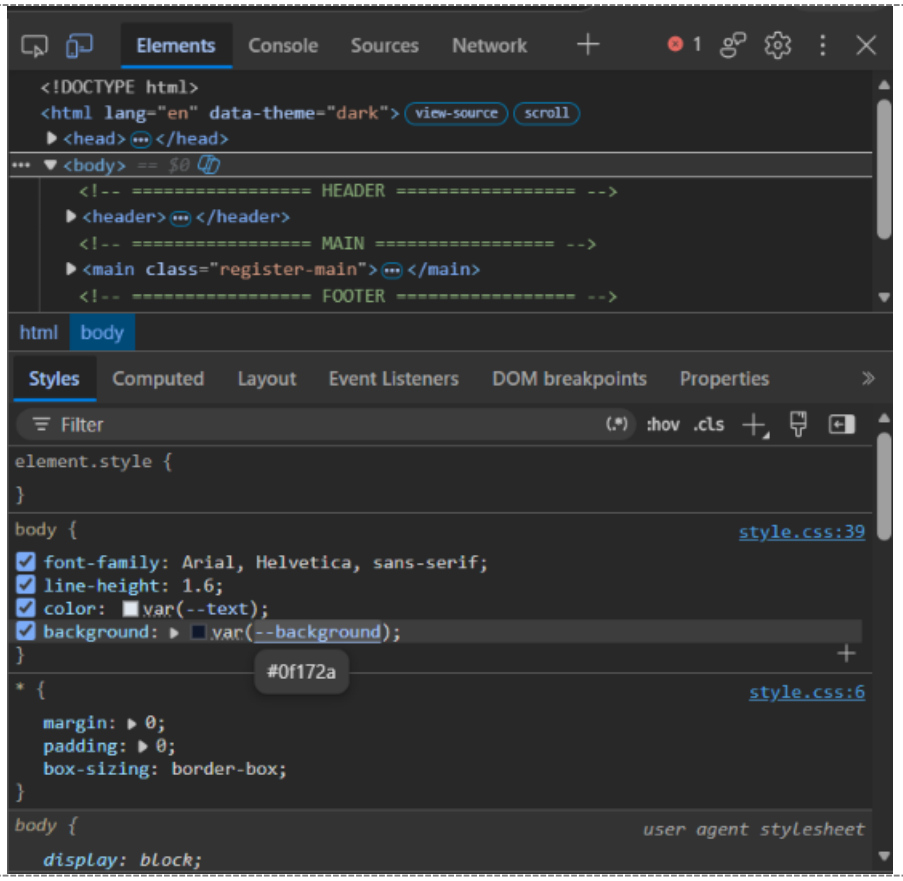


Figure 12: Console during testing

Key Questions and Answers

Q1. Are correct input types and attributes used?

Yes. The form uses type="email" for email, type="tel" for mobile, type="password" for the password fields, and radio buttons and a checkbox for gender and terms. required, minlength, maxlength, inputmode and autocomplete are set on the relevant fields, and novalidate on the form lets the custom JavaScript messages appear instead of the browser's default pop-ups.

Q2. Are validation errors displayed near the relevant fields?

Yes. Every field has its own <p class="error-message"> placed directly below it, linked to the field with aria-describedby. This keeps each message next to the field it belongs to rather than in one combined list, so it is clear which field needs fixing.

Q3. Is password strength checked?

Yes. Five checks are run on the password (length of at least 8, an uppercase letter, a lowercase letter, a number and a symbol), and a meter fills in proportionally with a label from Very weak to Strong. The same checks decide the password error message, listing exactly what is still missing.

Q4. Is the form accessible using keyboard and screen-reader-friendly labels?

Yes. Every input has a <label for> pointing to its id, so clicking a label focuses its field and screen readers announce the label with the field. All controls are native elements reachable with Tab. Invalid fields get aria-invalid="true", and the status message after submitting uses role="status" so it is announced automatically.

Post Laboratory Work:

Validation test cases with screenshots for valid and invalid inputs were captured and are included above.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal
(FOR SOURCE CODE)

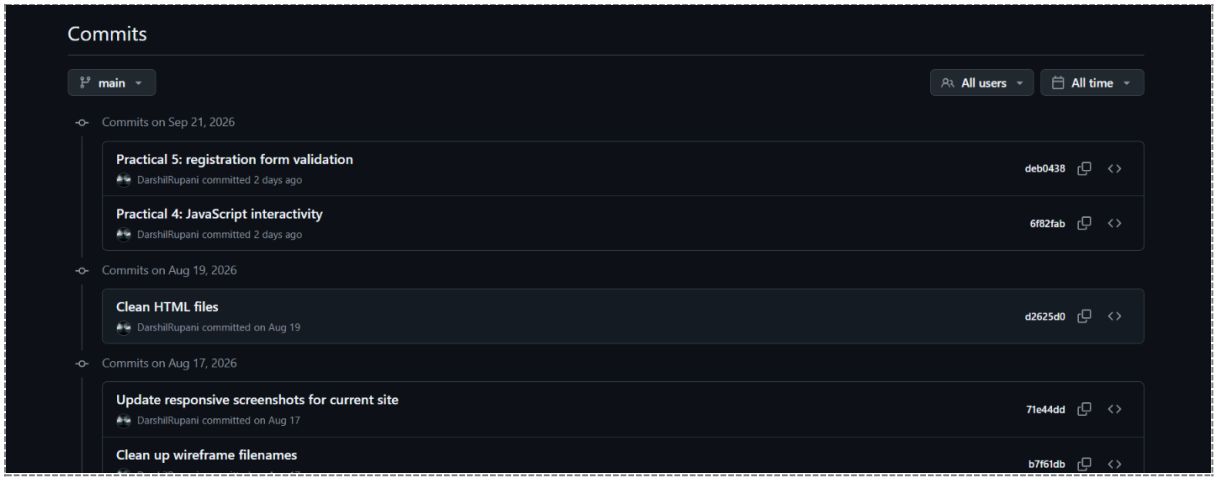


Figure 13: Practical 5 commit on GitHub

Conclusion:

The StudentHub registration form was extended with name, email, mobile, password, confirm password, course, year, gender, a CAPTCHA and terms acceptance, validated using regular expressions and JavaScript. A password strength meter, real-time validation and a canvas-based CAPTCHA were

added as extensions, and error messages were placed next to each field with ARIA attributes to keep the form accessible.

PRACTICAL – 6

Rendering External JSON Data using Fetch API, Search and Filter

AIM:

To dynamically render event and student data on the StudentHub Events and Student Directory pages from external JSON files using the Fetch API, and to implement client-side search, category/department filtering, sorting and pagination using array methods, together with loading and error handling and a localStorage caching extension for offline-style fallback.

Introduction:

The StudentHub Events page, which previously displayed a fixed set of hard-coded event cards, was rebuilt to load its content from an external JSON file at runtime using the Fetch API. Three reusable JavaScript modules were added — js/modules/api.js, js/modules/list-utils.js and js/modules/render.js — along with two page-specific scripts, js/events.js and js/students.js, which power a new Student Directory page built from a second JSON dataset. Both pages share the same search, category filter, sort and pagination logic, and the same loading, error and offline-caching behaviour.

Tools and Technology:

VS Code, JavaScript (ES6 modules), the Fetch API, JSON, HTML5, CSS3, localStorage, browser developer tools (Console, Application/Storage), Git and GitHub, and a local development server (VS Code Live Server).

Datasets Used:

Dataset File	Records	Key Fields
data/events.json	16	id, title, category, date, location, description
data/students.json	16	id, name, department, semester, cgpa, email
data/faqs.json	16	id, category, question, answer (prepared as a third dataset)

Every dataset is loaded and cached through the same fetchJson() helper, and every list page reuses the same filter, sort, paginate and render functions, so the Events and Student Directory pages behave identically when loading, searching, sorting, paging through results, or recovering from a failed request.

Key Skills and Array Methods Used:

- fetch() and async/await to request the JSON files
- try/catch and response.ok to detect and handle network or server errors
- Array.filter() and String.includes() for the search box and category filter
- Array.sort() with localeCompare() and numeric comparison for the sort options
- Array.slice() to split the filtered, sorted list into pages
- Array.map() and template literals to render each record as an HTML card
- Set with map() to build a unique list of categories/departments for the dropdown
- localStorage.setItem()/getItem() to cache the last successful response

Implementation Overview:

1. Fetching and Rendering

A shared fetchJson() helper (js/modules/api.js) calls fetch() on the JSON URL, checks response.ok, parses the JSON, and returns the data. js/events.js and js/students.js call this helper and pass the result to render.js, which maps each record to an HTML card using the page's existing card styling.

2. Search and Category Filter

filterItems() (js/modules/list-utils.js) checks the search text against several fields (title, location, description for events; name, department, email for students) and checks the selected category/department, combining both conditions with Array.filter().

3. Sorting

sortItems() reads the selected option (for example date-asc, title-desc, cgpa-desc) and sorts a copy of the filtered array using Array.sort(), switching between localeCompare() for text fields and numeric subtraction for number fields.

4. Pagination

paginate() slices the sorted array into pages of 6 records using Array.slice(). The page number is kept in a variable and re-applied whenever the search, filter or sort changes.

5. Loading and Error Handling

While the fetch is in progress, a "Loading..." message is shown. If the request fails (wrong URL, server down), a try/catch block catches the error; if a cached copy exists in localStorage it is used instead (with a visible notice), and if no cache exists a clear red error message is shown instead of a blank or broken page.

6. Modularity

The JavaScript is split into reusable modules: api.js (fetching and caching), list-utils.js (filter, sort, paginate — pure functions with no DOM code), and render.js (all DOM/HTML rendering). events.js and students.js are small page-specific files that import these modules and wire them to each page's controls.

7. Advanced Extension — Offline-style Caching

Every successful fetch is saved to localStorage under a page-specific key (studenthub-events-cache, studenthub-students-cache) along with a timestamp. If a later fetch fails, the cached copy is used automatically so the page still shows data instead of an empty screen.

Screenshots:

The JSON dataset, the search/filter/sort/pagination controls, the Student Directory page, the offline caching fallback, and the error state were all tested in the browser.



Figure 1: data/events.json opened directly in the browser, showing 16 valid JSON records.

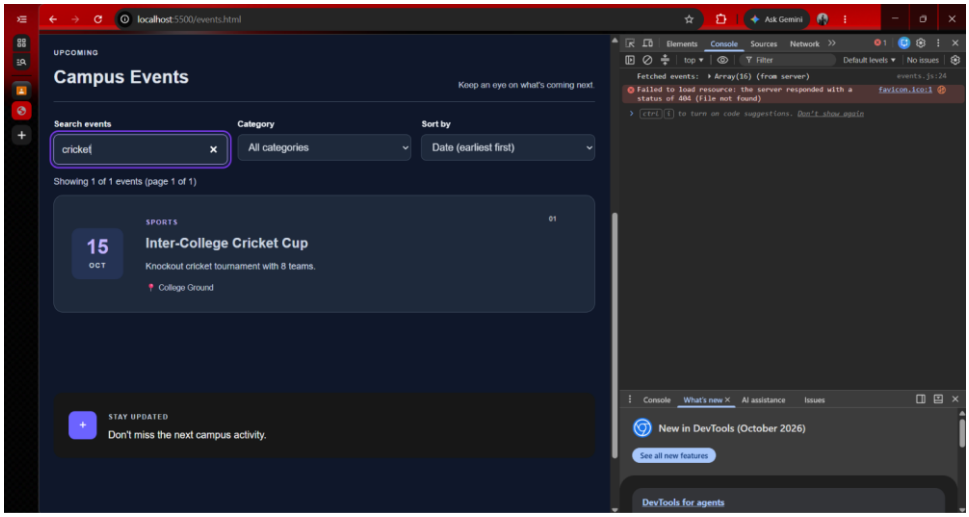


Figure 2: Search test — searching "cricket" correctly returns 1 of 16 events.

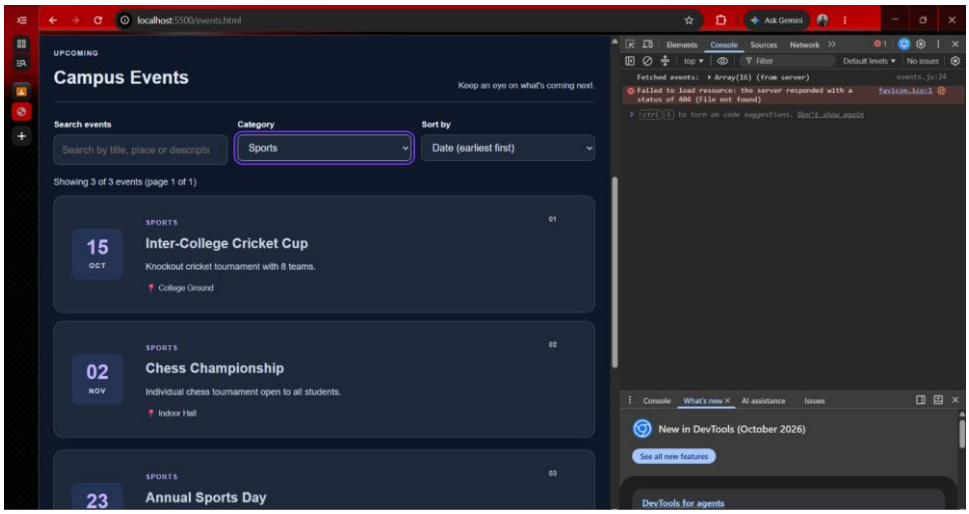


Figure 3: Category filter test — selecting "Sports" returns 3 of 16 events.

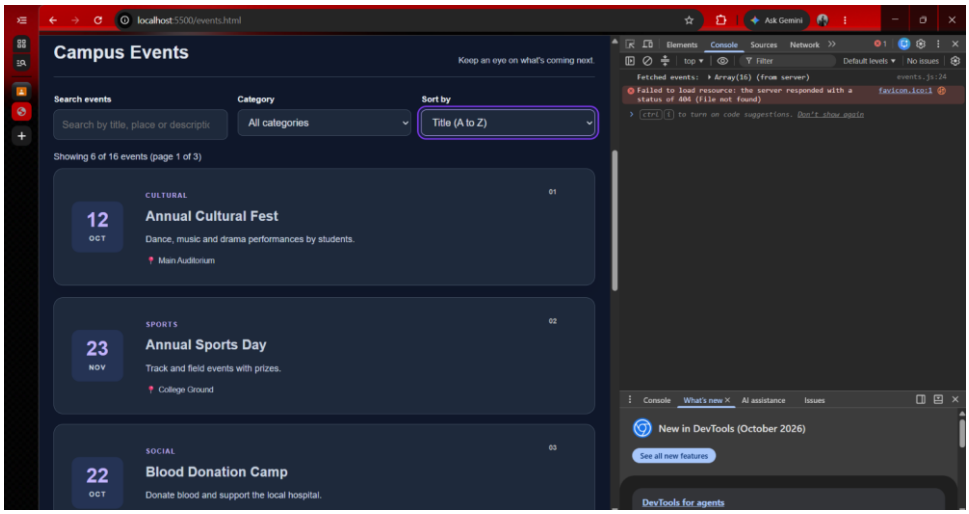


Figure 4: Sort test — "Title (A to Z)" places "Annual Cultural Fest" first.

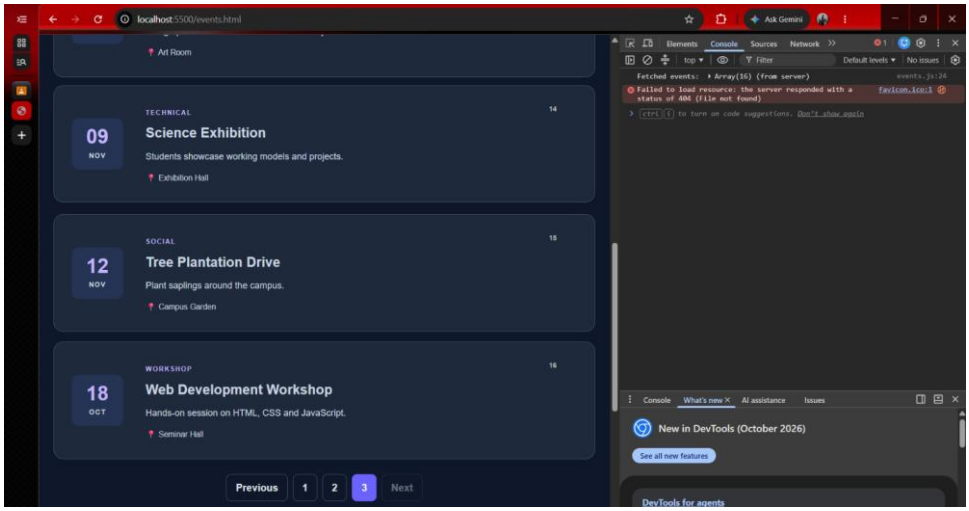


Figure 5: Pagination test — page 3 of 3 shows the last 4 events, with "Next" disabled.

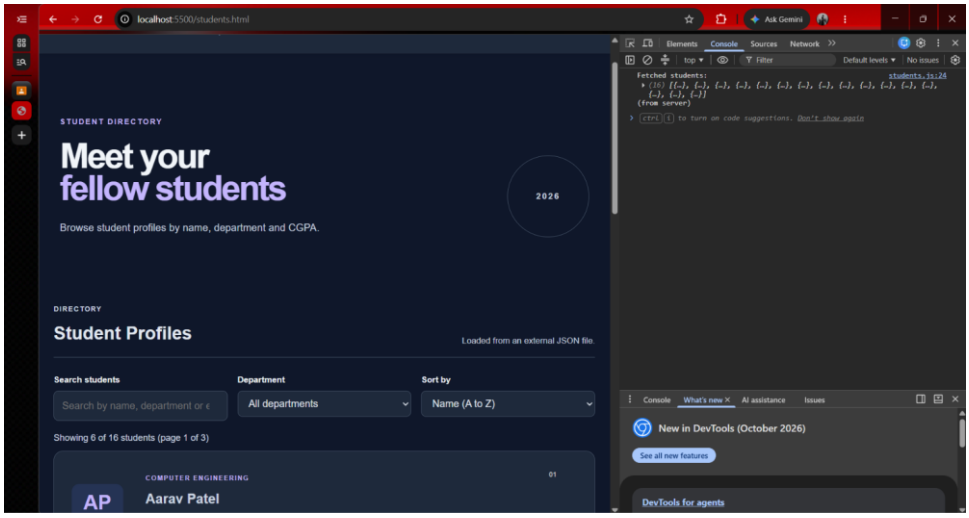


Figure 6: Student Directory page — students.html rendering all 16 records from data/students.json.

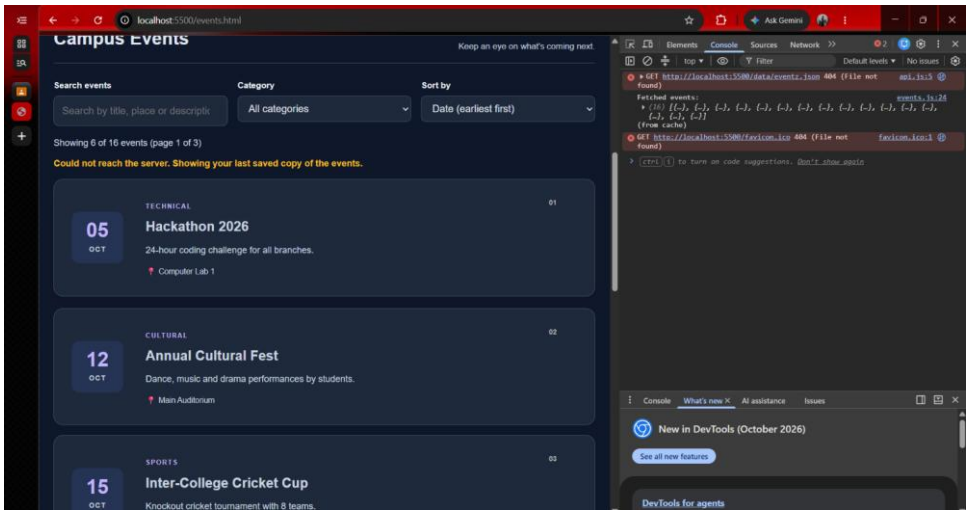


Figure 7: Cache fallback test — with the data URL intentionally broken, the page still shows all 16 events from the cached copy, with a visible notice and a 404 logged in the console.

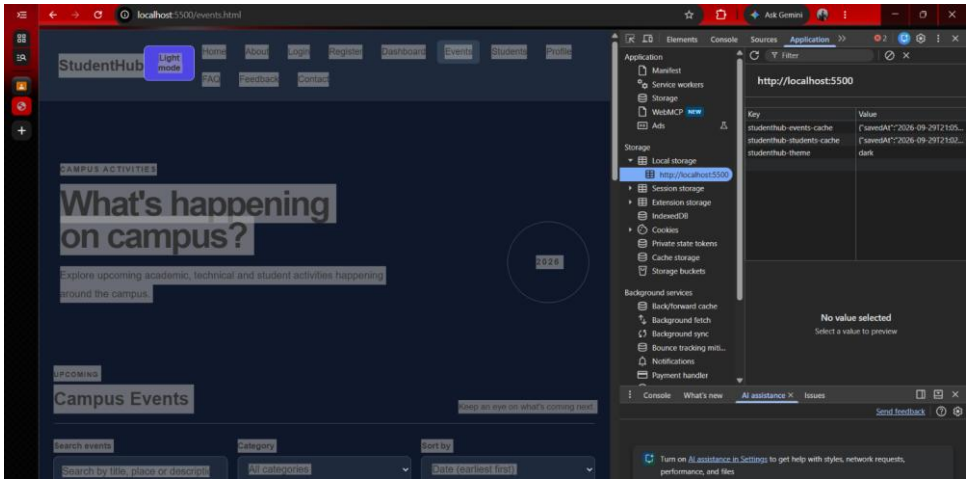


Figure 8: DevTools Application tab showing studenthub-events-cache and studenthub-students-cache saved in Local Storage.

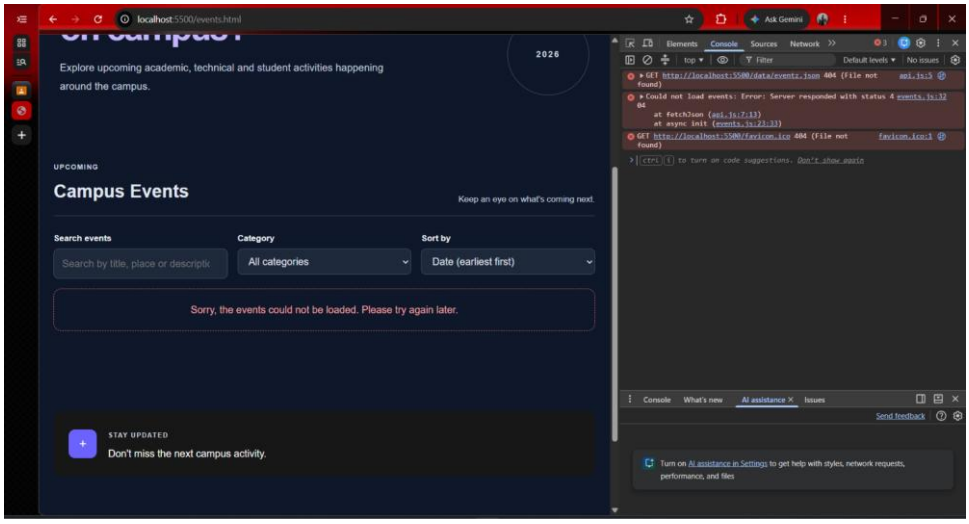


Figure 9: Error handling test — with the data URL broken and no cache available, a clear error message is shown instead of a broken page.

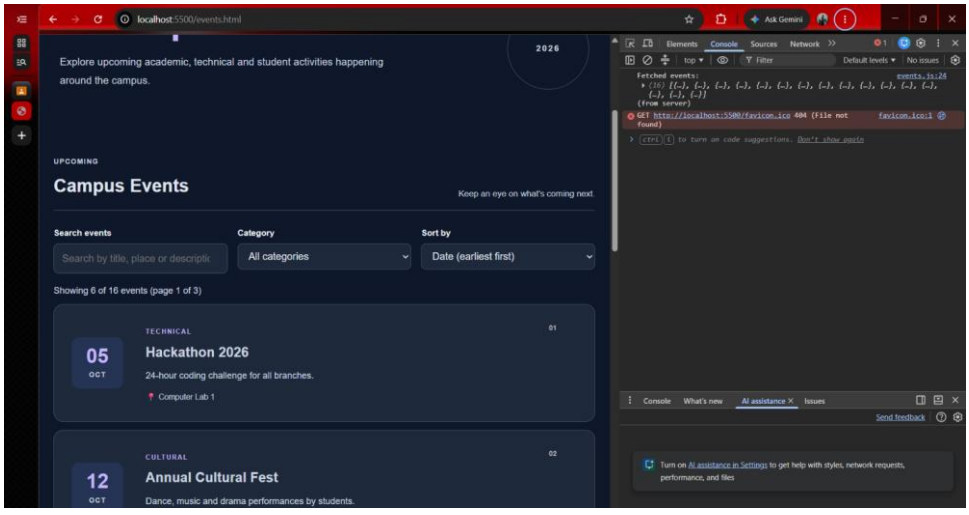


Figure 10: Final verification — after restoring the correct URL, the Events page loads normally again from the server.

Key Questions and Answers

Q1. How is JSON fetched, parsed and rendered?

fetch() requests the JSON file over HTTP. response.json() parses the response body into a JavaScript array. Array.map() then converts each record into an HTML string, which is inserted into the page with innerHTML.

Q2. Which array methods are used for search, filter, sort and pagination?

filter() and includes() for search and category matching, sort() with localeCompare()/numeric comparison for ordering, slice() to select one page of results, map() to render cards, and Set with map() to build the category/department dropdown.

Q3. How are loading and error states handled?

A "Loading..." message is shown while the fetch is in progress. A try/catch block, together with a check on response.ok, catches network and server errors. On failure the app first tries a cached copy from localStorage; if none exists, a clear error message is displayed instead of a blank or broken page.

Q4. How is modularity maintained in the JavaScript files?

The code is split by responsibility into ES modules: api.js handles fetching and caching, list-utils.js holds pure filter/sort/paginate functions, and render.js holds all DOM rendering. Each page (events.js, students.js) imports only what it needs from these shared modules.

Post Laboratory Work:

JSON files, rendered views, and test screenshots for search, filter, sort, pagination, loading, error handling and the caching extension were captured and are included above.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

(FOR SOURCE CODE)

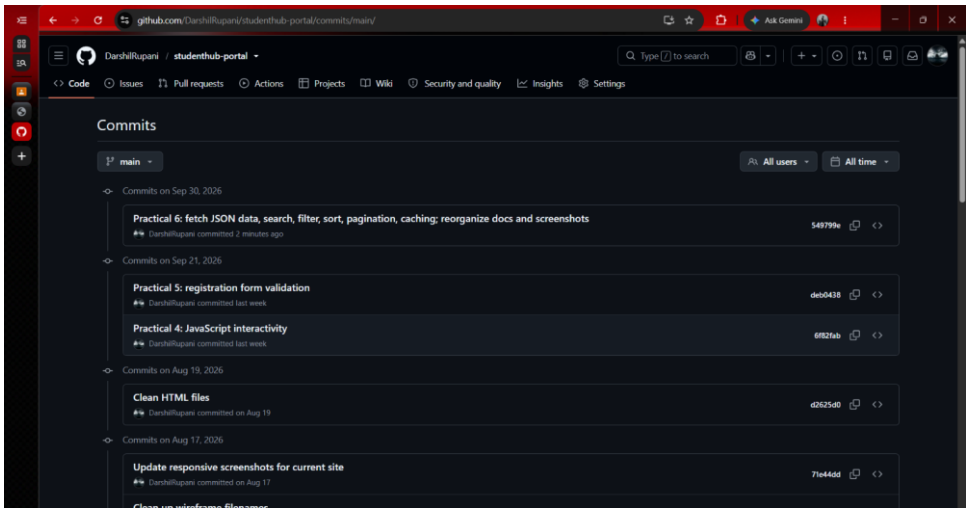


Figure 11: Practical 6 commit on GitHub.

Conclusion:

Practical 6 replaced hard-coded content on the Events page with data fetched from an external JSON file, and used the same pattern to build a new Student Directory page from a second JSON file. Search, category filtering, sorting and pagination were implemented using standard array methods, and the app was tested for both normal operation and failure conditions, including a working

localStorage caching extension that keeps the page usable when the network request fails. All required screenshots for search, filter, sort, pagination, error handling and caching are included above.

PRACTICAL – 7

PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

AIM:

To process the StudentHub registration form using PHP, validate and sanitize inputs on the server, store accepted registration requests in CSV and JSON files, and display clear success and error messages. The practical also implements duplicate-email checking, a records-view page and session-based CSRF protection.

Introduction:

The existing StudentHub registration form was connected to a PHP backend running through XAMPP Apache. JavaScript performs client-side checks and submits FormData using fetch(). PHP independently checks the submitted values before saving them. Valid requests are stored as structured JSON records and CSV rows; invalid requests return field-specific errors without adding a registration.

Tools and Technology:

VS Code, PHP, XAMPP Apache, HTML5, CSS3, JavaScript, Fetch API, FormData, JSON, CSV, PHP sessions, browser developer tools, Git and GitHub.

Files Used:

File	Purpose
register.html / js/validation.js	Registration form, client validation and POST submission.
php/get-csrf-token.php	Generates a token and stores it in the PHP session.
php/process-registration.php	Validates inputs, checks duplicates and writes records.
registrations.html / js/registrations.js	Displays saved registration records.
php/storage/registrations.json	Structured registration records.
php/storage/registrations.csv	Header row followed by registration rows.

Key Skills Used:

POST handling with $_POST; validation with regular expressions, filter_var() and allowlists; sanitization with trim(), strip_tags() and htmlspecialchars(); JSON encoding and decoding; CSV writing with fputcsv(); exclusive file locks with flock(); and CSRF checking with PHP sessions and hash_equals().

Implementation Overview:

1. Form Submission using POST

The submit handler prevents the default form action, validates the inputs locally and sends new FormData(form) to the PHP processing endpoint with method: POST. PHP rejects other HTTP methods with a 405 response.

2. Server-Side Validation and Sanitization

PHP checks the name length, email format, 10-digit mobile number, password rules, matching confirmation, course, year, gender and acceptance of terms. String inputs are checked before use. Names are trimmed, stripped of HTML tags and escaped before storage. Invalid inputs produce an errors object with the relevant field keys.

3. Duplicate-Email Checking

The existing JSON records are read while holding an exclusive lock. strcasecmp() compares the submitted email with saved emails without case sensitivity. A duplicate is rejected before a new record is written.

4. CSV and JSON Storage

Accepted requests contain an ID, name, email, mobile number, course, year, gender, role and timestamp. JSON stores an array of records. CSV stores a header and one row per request. Password values are not saved; this practical records account requests rather than implementing login authentication.

5. Checked File Handling

Folder creation, file opening, locking, reading, encoding and writing are checked. Malformed existing JSON is rejected instead of discarded. Both files are locked before writing; detected failures trigger restoration attempts and clear errors. Separate file writes are not a crash-proof transaction.

6. Success and Error Responses

PHP returns JSON containing success, message and errors. JavaScript displays success or error messages and marks fields with server errors. A successful submission resets the form and requests a fresh token.

7. Advanced Extension — CSRF Protection and Records View

The browser requests a session token from get-csrf-token.php and includes it as csrf_token. PHP uses hash_equals() to verify it before validation and storage. An invalid token returns 403. The Registrations page displays saved records and was checked after refresh.

Testing Summary:

Test	**Result**
Valid registration and JSON/CSV storage	Passed; rechecked after the PHP update (user confirmed).
Duplicate email	Rejected with a field error; no additional record.
Direct POST with mobile = 123	Rejected by PHP despite bypassing client validation.
Invalid CSRF token	403 Forbidden and success: false.
Records view after refresh	Passed (user confirmed); screenshot to be inserted.
Simulated file-write failure	Not executed; failure handling added in code.

Screenshots:

The following screenshots were captured during the practical. The storage images show the initial successful record; the later successful save after the file-handling update was confirmed separately.

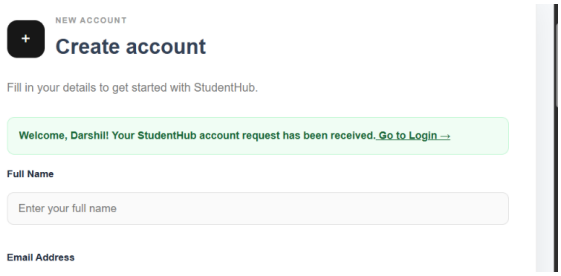


Figure 1: Successful registration response; the form resets after PHP accepts the request.

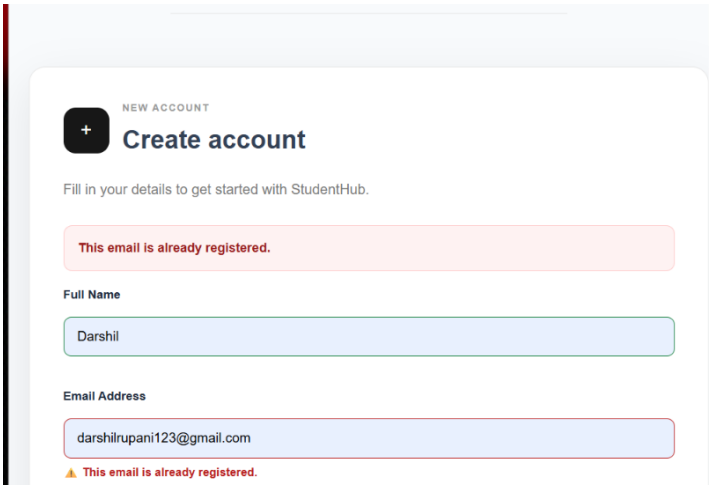


Figure 2: Duplicate-email rejection displayed at the top of the form and below the email field.



Figure 3: registrations.json showing one saved registration record.

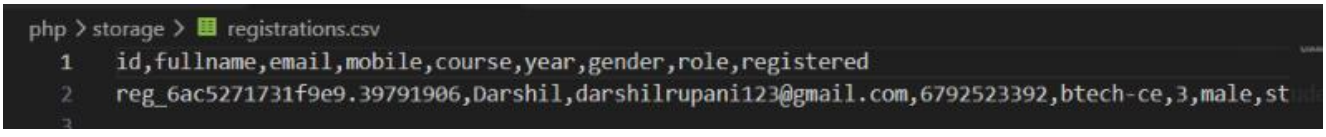


Figure 4: registrations.csv showing the header and one saved registration row.

Screenshots — Server Validation and CSRF:

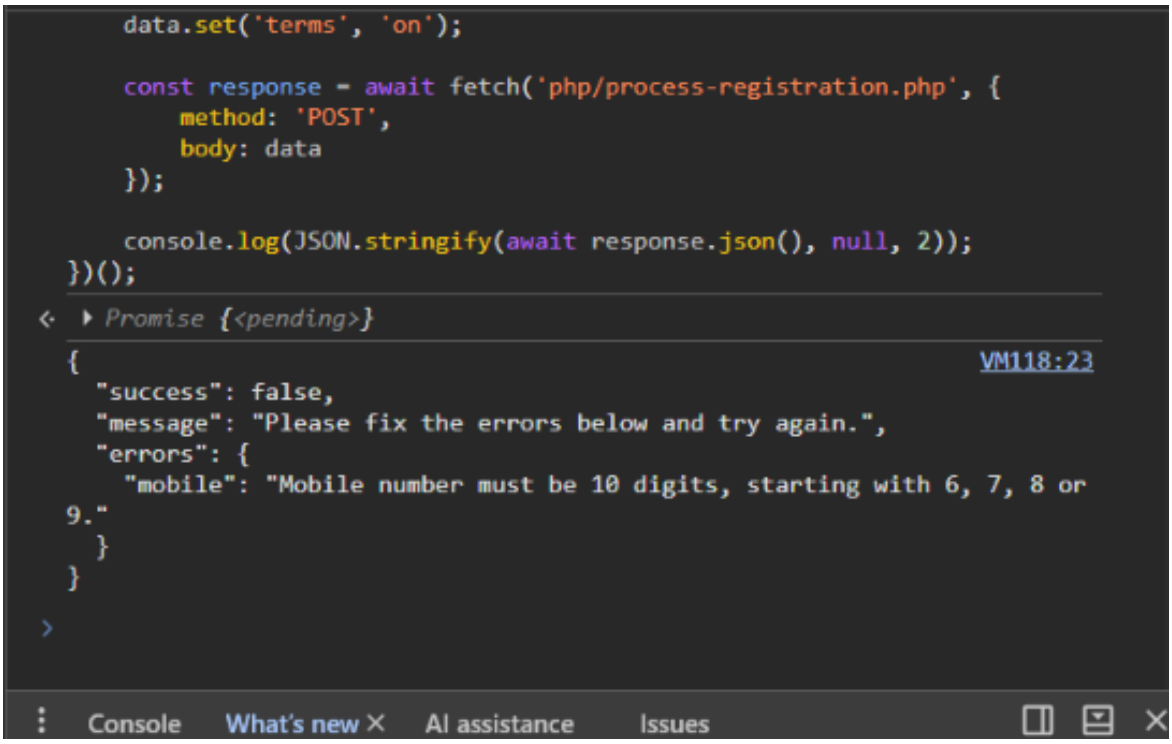


Figure 5: Direct POST test with mobile number 123; PHP returns success: false and a mobile validation error.

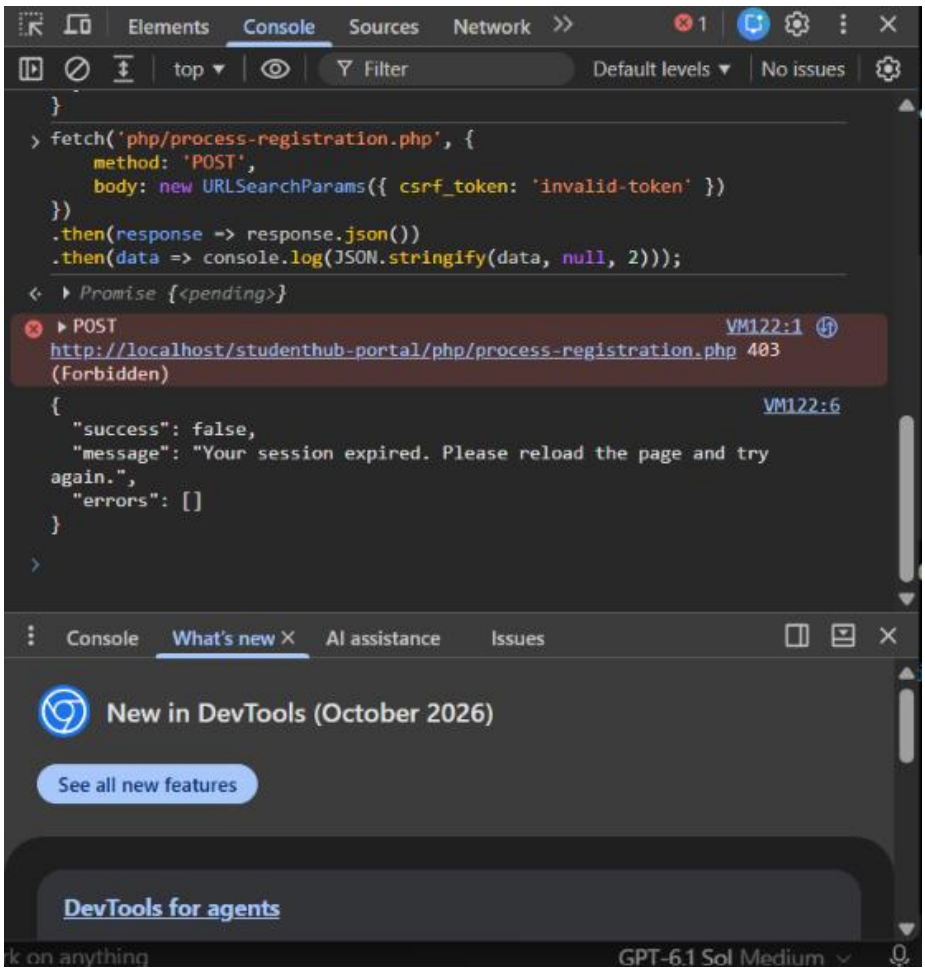


Figure 6: Invalid CSRF token rejected with HTTP 403 and the session-expired message.

Screenshots — Records View and Source Submission:

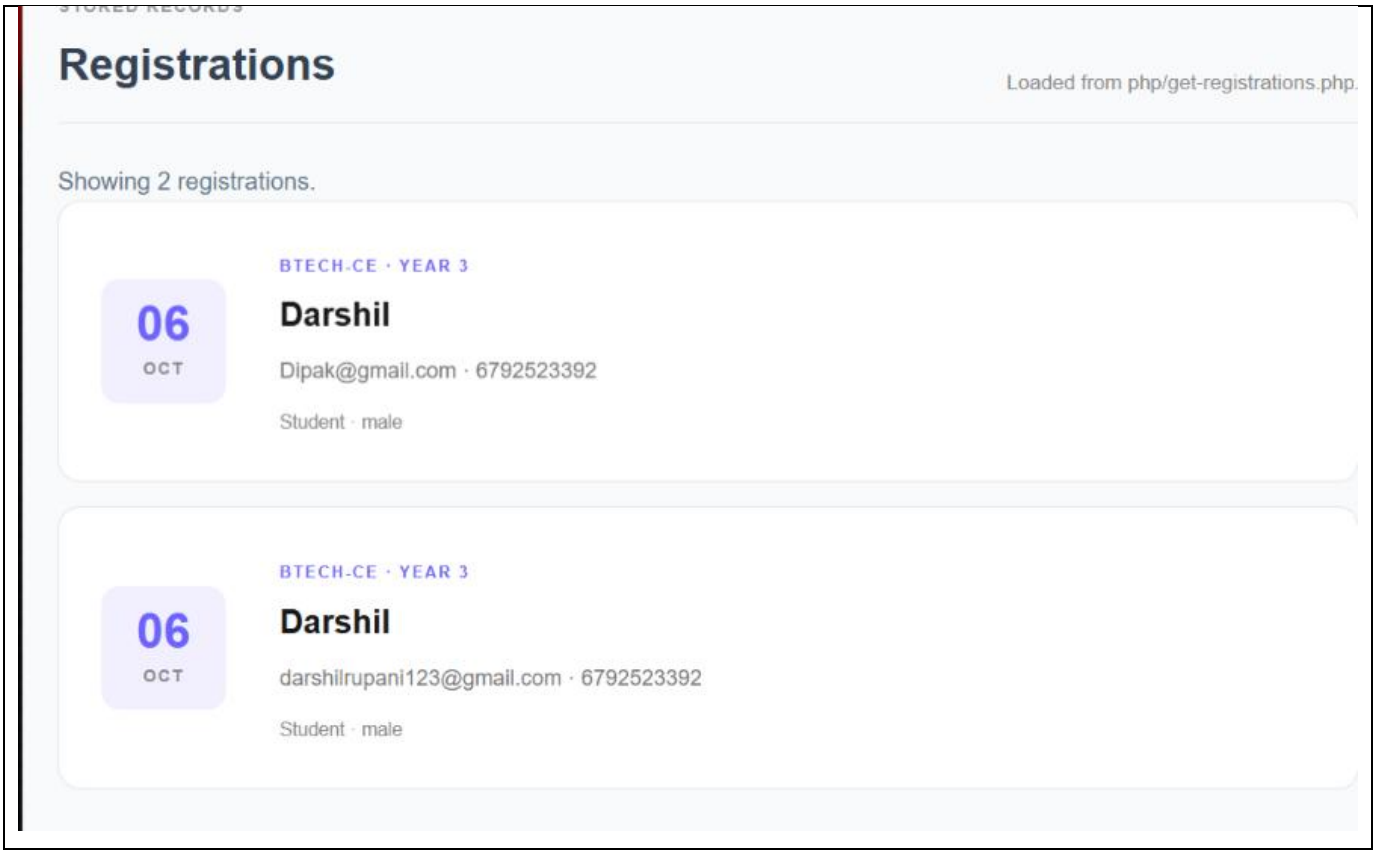


Figure 7: Registrations page displaying records retrieved from storage.

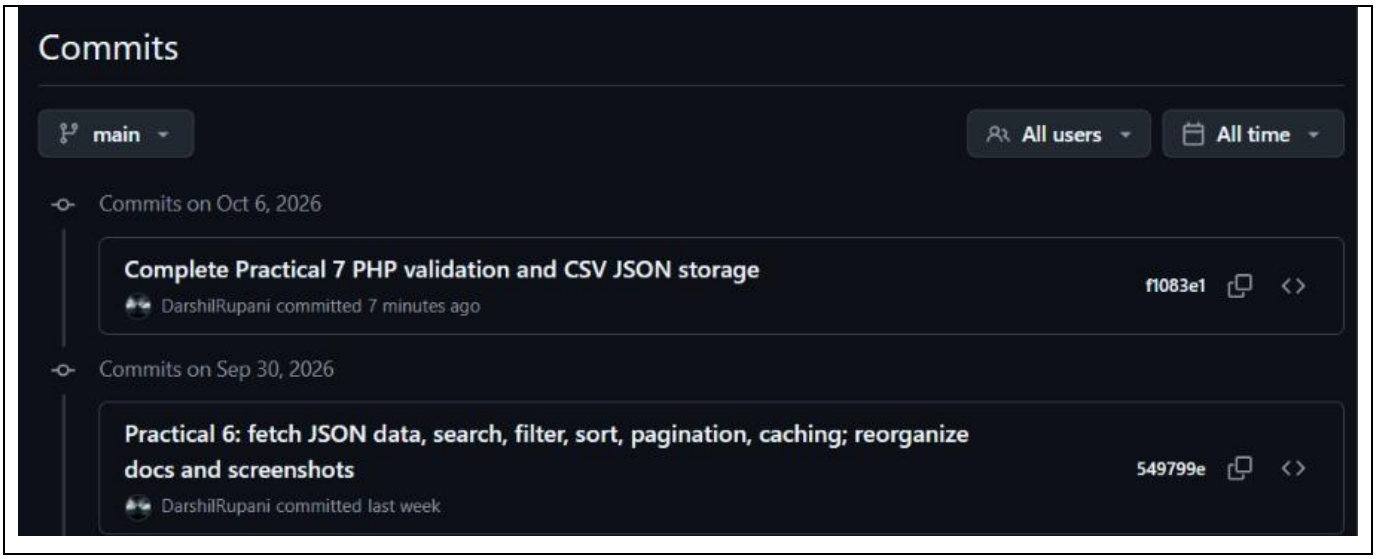


Figure 8: Practical 7 source-code submission on GitHub.

GITHUB REPOSITORY LINK:

https://github.com/DarshilRupani/studenthub-portal

(FOR SOURCE CODE)

Practical 7 source pushed to main: commit f1083e1 — Complete Practical 7 PHP validation and CSV JSON storage. Push verified from terminal output.

Key Questions and Answers

Q1. Is the form submitted using POST?

Yes. JavaScript sends the form using fetch() with method: POST and a FormData body. PHP reads $_POST and checks REQUEST_METHOD; requests using other methods are rejected.

Q2. Are inputs validated and sanitized server side?

Yes. PHP validates required fields, email, mobile, password and confirmation, course, year, gender and terms. trim() removes surrounding spaces, strip_tags() removes HTML tags from the name, and htmlspecialchars() escapes special characters in the stored name. filter_var() validates the email, and allowlists restrict dropdown and radio values. These checks run independently of JavaScript.

Q3. Is file writing handled safely?

The processing code checks folder creation, file opening, locks, reads, encoding, writes and flushes. Exclusive locks protect updates made through this endpoint. Existing invalid JSON is not silently overwritten. Detected failures trigger restoration attempts; incomplete restoration is reported. JSON and CSV are separate files, so a process crash between writes is still a limitation.

Q4. Are success and error responses displayed clearly?

Yes. PHP returns success, message and errors as JSON. JavaScript displays a success message after saving, a general error message for rejected requests and field-level messages for validation errors. Invalid CSRF tokens return HTTP 403, while storage errors return HTTP 500.

Post Laboratory Work:

Submit the PHP source files, test cases and generated CSV/JSON files along with this report. The report includes evidence of successful submission, duplicate rejection, storage, direct server-side validation and CSRF rejection. Insert the records-view and GitHub screenshots in the labeled spaces before final submission.

Conclusion:

Practical 7 connected the StudentHub registration form to PHP processing through XAMPP. Valid account requests are saved in CSV and JSON, while invalid inputs, duplicate emails and invalid CSRF tokens are rejected with clear messages. File operations now include error checks and restoration attempts. Successful submission and storage were rechecked after the update; simulated write failures remain untested.

PRACTICAL 8

MySQL Schema Design, ER Model, PDO Connectivity and Prepared Statements

AIM

To design a normalized StudentHub database with students, events and registrations, define primary and foreign keys, connect PHP securely using PDO, and retrieve records using a prepared statement.

Introduction

StudentHub was extended with a relational database named studenthub_db. Students and events are stored separately, while registrations links each student to an event. The database was populated with sample records and accessed through PHP using a dedicated database account. Browser tests confirmed the connection and demonstrated valid, empty and invalid-input results.

Tools and Technology

VS Code, XAMPP, MariaDB 10.4.32, phpMyAdmin 5.2.1, PHP 8.2.12, PDO MySQL driver, SQL, browser, Git and GitHub. MariaDB is the MySQL-compatible database server supplied with this XAMPP installation.

Files Used

File	Purpose
sql/studenthub_db.sql	Database structure, seed data, indexes and constraints.
php/db.php	Reusable PDO connection returned to calling scripts.
php/db-config.php	Local database settings and password; excluded from Git.
php/db-config.example.php	Configuration template containing a password placeholder.
php/test-connection.php	Connection test, database user and table counts.
php/prepared-demo.php	Validated student ID and parameterized registration query.
.gitignore	Excludes the private configuration and Practical 7 storage.

Key Skills Used

Relational schema design, normalization, primary and foreign keys, UNIQUE and CHECK constraints, SQL JOINs, seed data, database privileges, PDO exception handling and prepared statements.

Implementation Overview

1 Database and Tables

The database uses InnoDB tables and utf8mb4_unicode_ci collation. Each table has an unsigned integer primary key with AUTO_INCREMENT. Students contain contact and study details; events contain the title, description, date and venue.

2 Relationships and Constraints

registrations.student_id references students.student_id, and registrations.event_id references events.event_id. Updates cascade to the referencing keys; deleting a referenced parent is restricted. A unique student/event pair prevents duplicate event signups, unique student email prevents duplicates, and chk_student_year limits the year to 1 through 4.

3 Normalization and ER Model

Each column stores a single value, and each table represents one entity or relationship. Student and event details are kept in their own tables rather than repeated for every registration. The registrations table resolves the many-to-many relationship into two one-to-many relationships. In this practical, registrations means event signups, separate from the account requests stored in Practical 7.

4 Seed Data and JOIN Query

Five students, three events and six registrations were inserted. An INNER JOIN across the three tables displayed each registration with its student name, event title, date and venue.

5 PDO Connectivity

A dedicated studenthub_app account was granted SELECT, INSERT, UPDATE and DELETE on studenthub_db. db.php reads local configuration and creates a PDO connection using charset=utf8mb4, exception error mode, associative fetch mode and disabled emulated prepares. The test page confirmed the database account and all three record counts.

6 Prepared Statement and Error Handling

prepared-demo.php validates student_id as a positive integer, prepares a JOIN query containing :student_id, binds the value using PDO::PARAM_INT, and executes it. Invalid input is rejected before the query. Exceptions are caught; a general browser message is returned and technical details are logged on the server.

7 SQL Export and Source Submission

The SQL dump includes structure, seed records, indexes and foreign keys. The students definition was corrected in the export to retain its CHECK constraint, engine and character settings. The corrected text was reviewed; a fresh import was not tested. Source files were pushed to GitHub in commit 0ee6a87.

Database Structure and ER Model

Table	Primary key	Other constraints	Rows
students	student_id	Unique email; year CHECK 1–4	5
events	event_id	Required title, date and venue	3
registrations	registration_id	Two foreign keys; unique student/event pair	6

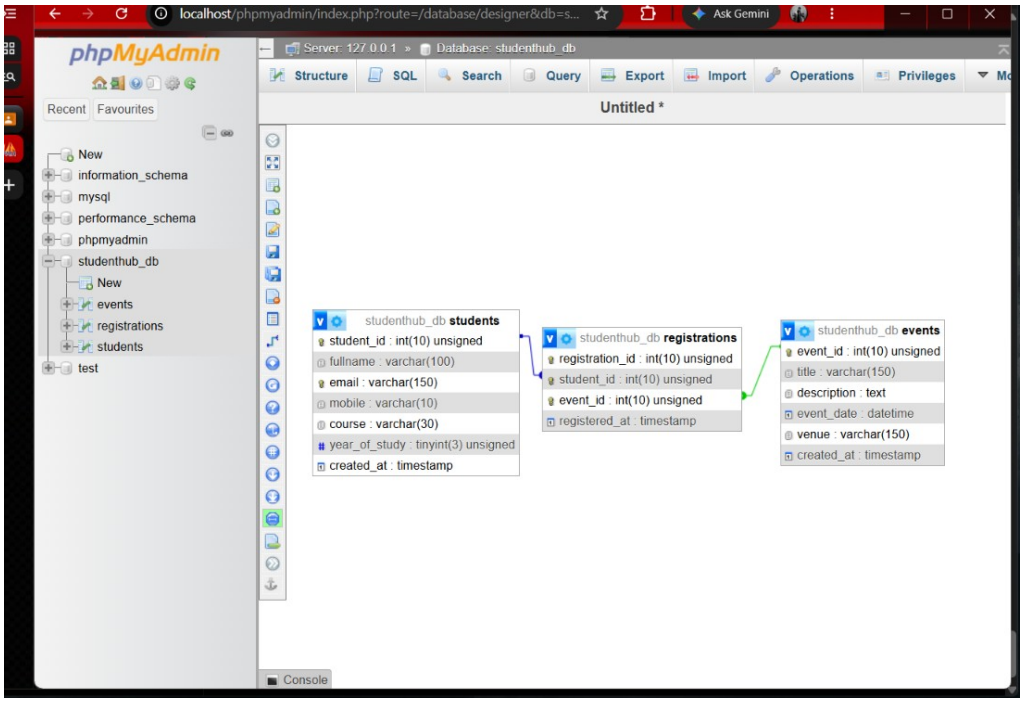


Figure 1: phpMyAdmin Designer showing students and events connected through registrations.

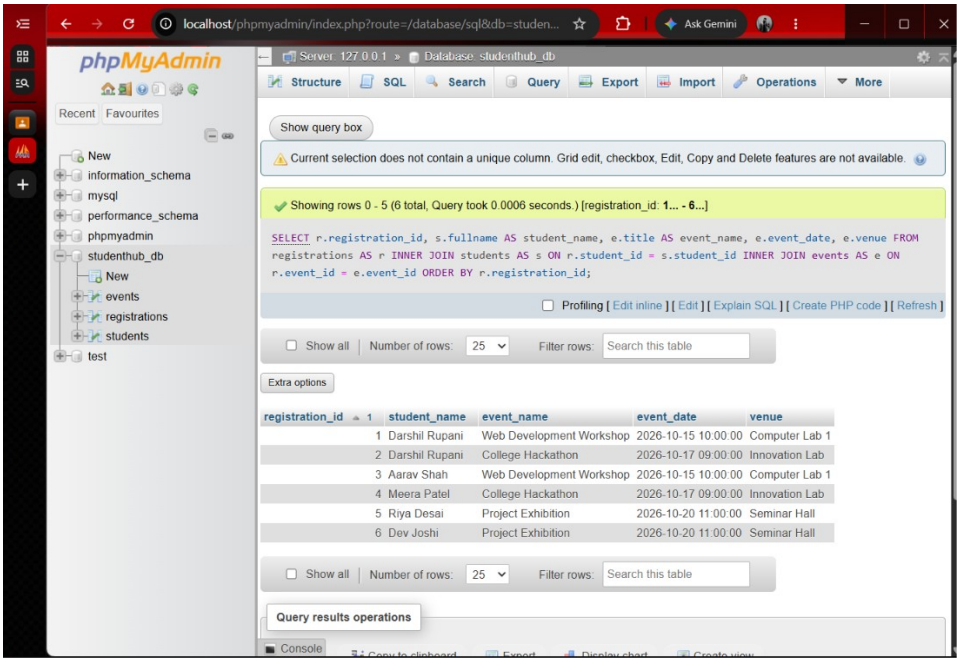


Figure 2: JOIN query displaying all six registrations with student and event details.

Screenshots of PDO and Prepared Statement Results

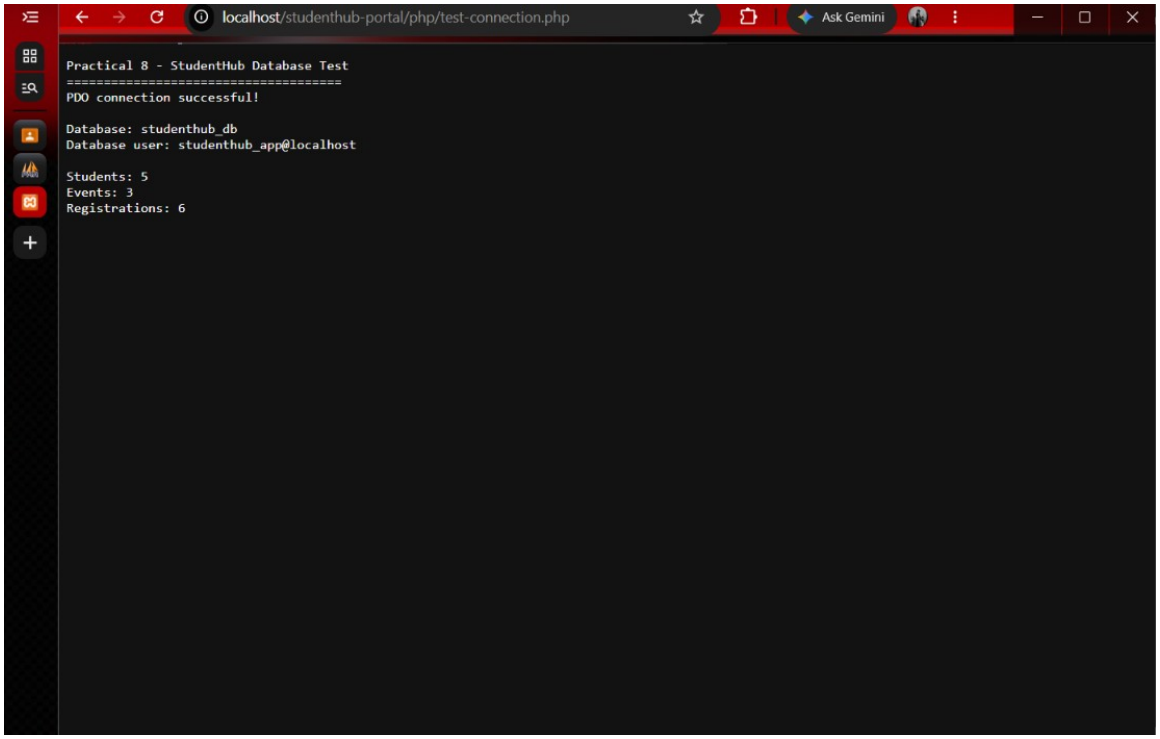


Figure 3: Successful PDO connection using studenthub_app, with 5 students, 3 events and 6 registrations.

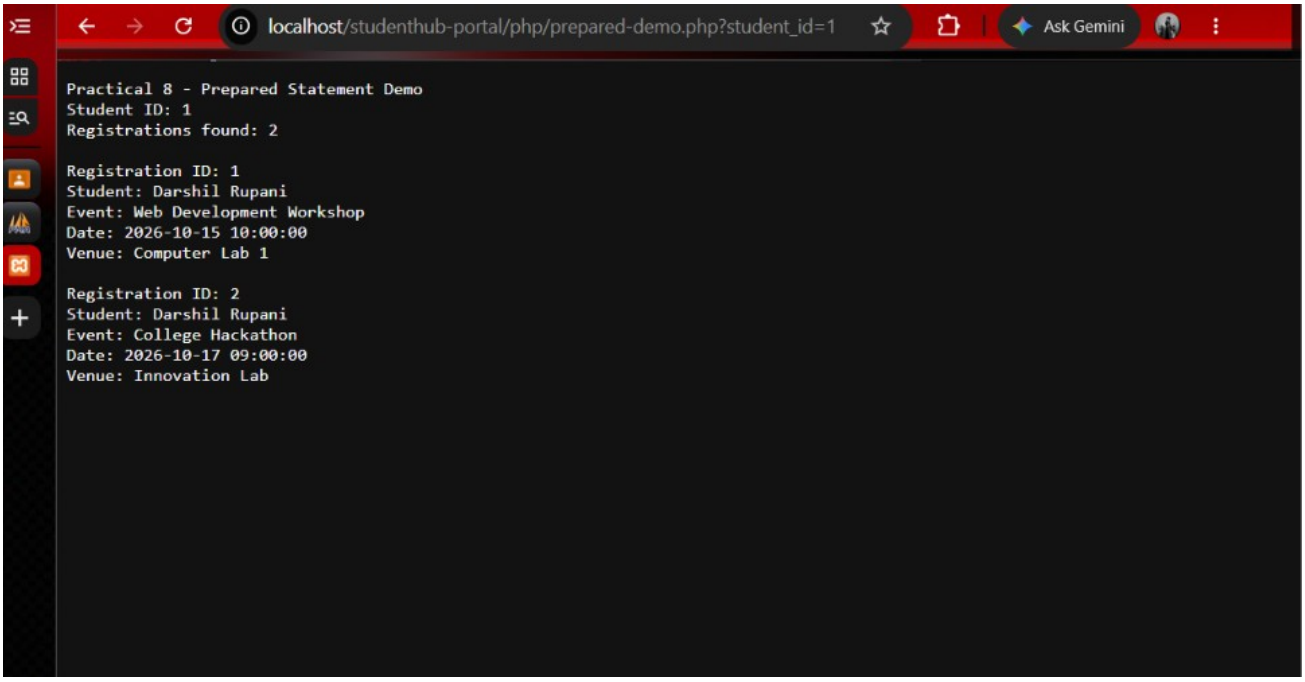


Figure 4: student_id=1 returns two event registrations for Darshil Rupani.

Connection test URL: http://localhost/studenthub-portal/php/test-connection.php

Prepared-statement URL: http://localhost/studenthub-portal/php/prepared-demo.php?student_id=1

Screenshots of Additional Test Cases

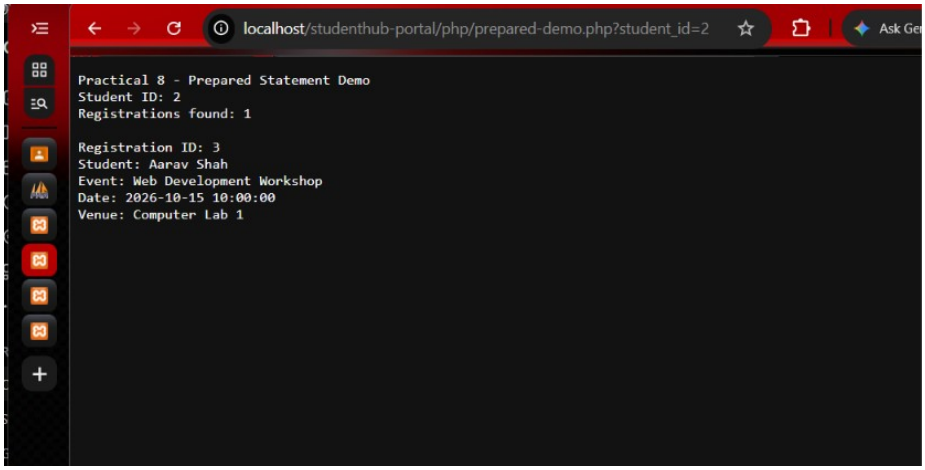


Figure 5: student_id=2 returns one registration for Aarav Shah.

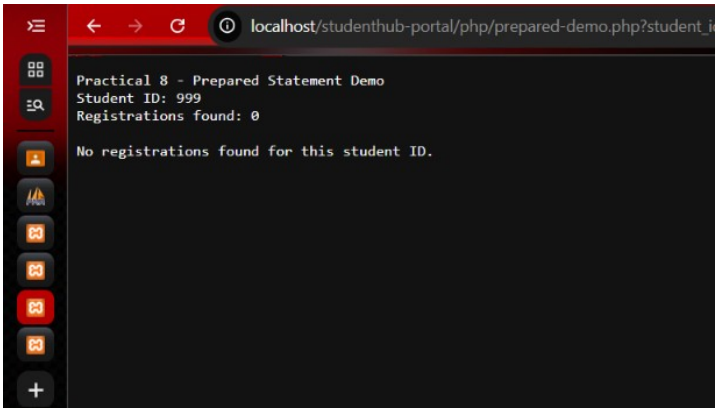


Figure 6: student_id=999 returns zero registrations and a clear message.

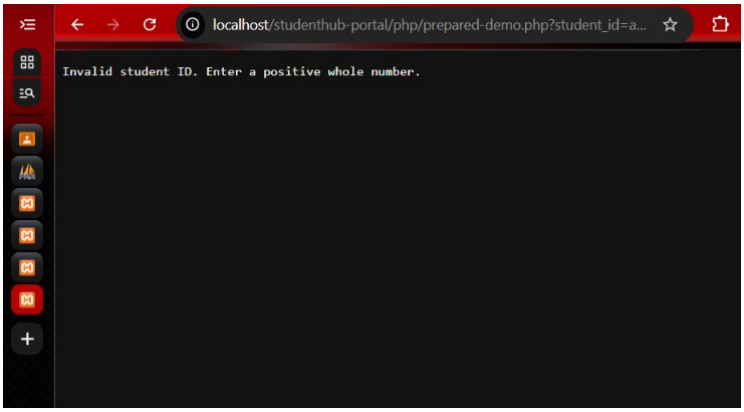


Figure 7: Non-numeric input is rejected with an invalid student ID message.

Testing Summary

Test input or action	Observed result
PDO connection test	Successful; counts: students 5, events 3, registrations 6.
student_id=1 / student_id=2	Two registrations / one registration returned.
student_id=999	Zero registrations with a no-records message.
Non-numeric student_id	Rejected: enter a positive whole number.

Key Questions and Answers

Q1 Is the schema normalized properly

The design separates student details, event details and the association between them. Values are atomic, and descriptive attributes depend on the key of their own entity. The registration table stores references rather than repeating names and event descriptions. This structure supports third normal form for the modeled data.

Q2 Are primary keys and foreign keys correctly defined

Each table has an AUTO_INCREMENT primary key. Both registration foreign keys reference the matching unsigned integer parent keys. The ER diagram shows both relationships. A UNIQUE constraint on student_id and event_id prevents duplicate signups for the same event.

Q3 Is the database connection handled securely

PHP connects using the dedicated studenthub_app account rather than root. The password is kept in db-config.php, which Git ignores; the shared example contains a placeholder. PDO exceptions are caught by the calling pages, and detailed errors are logged rather than shown in the browser.

Q4 Why use a prepared statement

The query structure is prepared separately from the supplied value. The student ID is bound to :student_id as an integer, so it is treated as a parameter rather than concatenated SQL. Positive-integer validation also provides a clear response for unsuitable input.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal

Practical 8 source commit: 0ee6a87
Message: Complete Practical 8 database schema and PDO prepared statements
Push to main was confirmed from terminal output. A later report update was pushed as 82dac4a; the working tree was clean at that point.

Post Laboratory Work

Submit sql/studenthub_db.sql, php/db.php, the supporting PHP scripts and the connection-test screenshot with this report. To reproduce the setup, import the dump into an empty studenthub_db database, create the application user with the required privileges, and copy db-config.example.php to db-config.php with local credentials.

Conclusion

The StudentHub database was designed and populated successfully. Its ER model represents the student-to-event relationship through registrations. PDO connectivity and parameterized retrieval were demonstrated with successful, empty and invalid-input cases. The intermediate PDO and prepared-statement extension was completed. The optional stored-procedure extension was not implemented.

Verification scope: screenshots confirm the browser results. A fresh dump import, deliberate constraint violations and a forced connection failure were not tested.

PRACTICAL 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

AIM

To implement secure user registration in StudentHub using PHP and MySQLi, validate input on the frontend and backend, check duplicate emails, hash passwords using password_hash(), and display appropriate messages.

Introduction

StudentHub was extended to store user accounts in studenthub_db. Registration details are validated before inserting them using a MySQLi prepared statement. Duplicate emails are rejected, and passwords are stored as hashes. Tests confirmed successful registration, invalid-input handling and duplicate-email rejection.

Tools and Technology

VS Code, XAMPP, MariaDB, phpMyAdmin, PHP, MySQLi, HTML, CSS, JavaScript, SQL, browser, Git and GitHub. MariaDB is the MySQL-compatible database server supplied with XAMPP.

Files Used

File	Purpose
register.html	User registration form and field error messages.
js/validation.js	Frontend validation and registration submission.
php/process-registration.php	Backend validation, duplicate check, hashing and insert.
php/db-mysqli.php	Reusable MySQLi database connection.
php/db-config.php	Private database settings; excluded from Git.
php/get-csrf-token.php	Session CSRF token and security code.
sql/practical9.sql	Users and registration audit table definitions.

Key Skills Used

Secure registration, frontend and backend validation, MySQLi connectivity, prepared statements, duplicate checks, password hashing, database inserts, CSRF protection and audit logging.

Implementation Overview

1 Database and Tables

The users table stores full name, email, mobile, password hash, course, year, gender, role and creation time. user_id is an AUTO_INCREMENT primary key, and email has a UNIQUE constraint. The registration_audit table records successful account creation.

2 Frontend and Backend Validation

JavaScript validates the form before submission. PHP repeats the validation on the server. Both apply matching rules for name, email, mobile, password and selection fields. Password confirmation and acceptance of terms are required.

3 MySQLi Connectivity

db-mysqli.php reads the local database configuration and connects using the studenthub_app account. The connection uses utf8mb4 and MySQLi exception reporting. The private configuration file remains excluded from Git.

4 Duplicate Email Check

A prepared SELECT checks whether the email already exists. The email is trimmed and converted to lowercase. A UNIQUE constraint prevents duplicate inserts, and duplicate requests receive an appropriate email error.

5 Password Hashing

The password is hashed using password_hash() with PASSWORD_DEFAULT before insertion. The password_hash column uses VARCHAR(255). Plaintext passwords are not stored. password_verify() confirmed the correct test password and rejected an incorrect password.

6 Prepared Insert and Error Handling

User details are bound as parameters to a prepared INSERT statement. Valid input creates an account and displays a success message. Validation errors are returned against their fields. Database errors return a general message, with details logged on the server.

7 CSRF Token and Audit Log

A session CSRF token is generated and checked using hash_equals(). Incorrect tokens are rejected. Account creation and its audit entry are saved in one transaction, so both operations commit together.

Screenshots of Registration and Validation Results

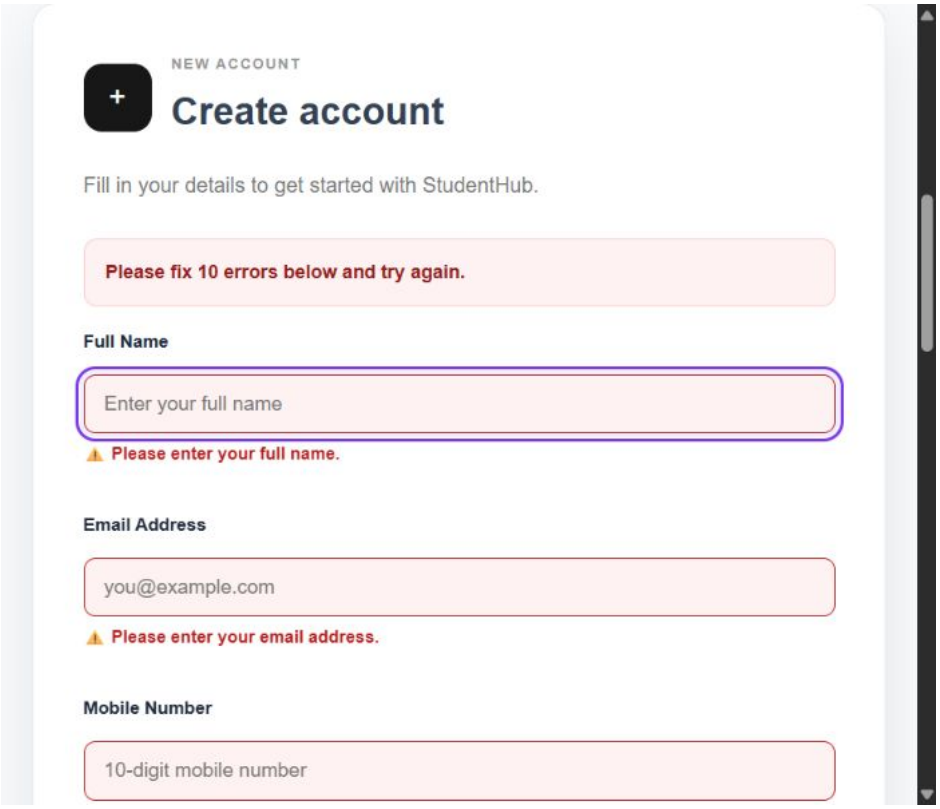


Figure 1: Registration form displaying required-field validation errors.



Figure 2: Recorded HTTP responses for valid, duplicate and invalid email registration.

Screenshots of Database Insert and Audit Log

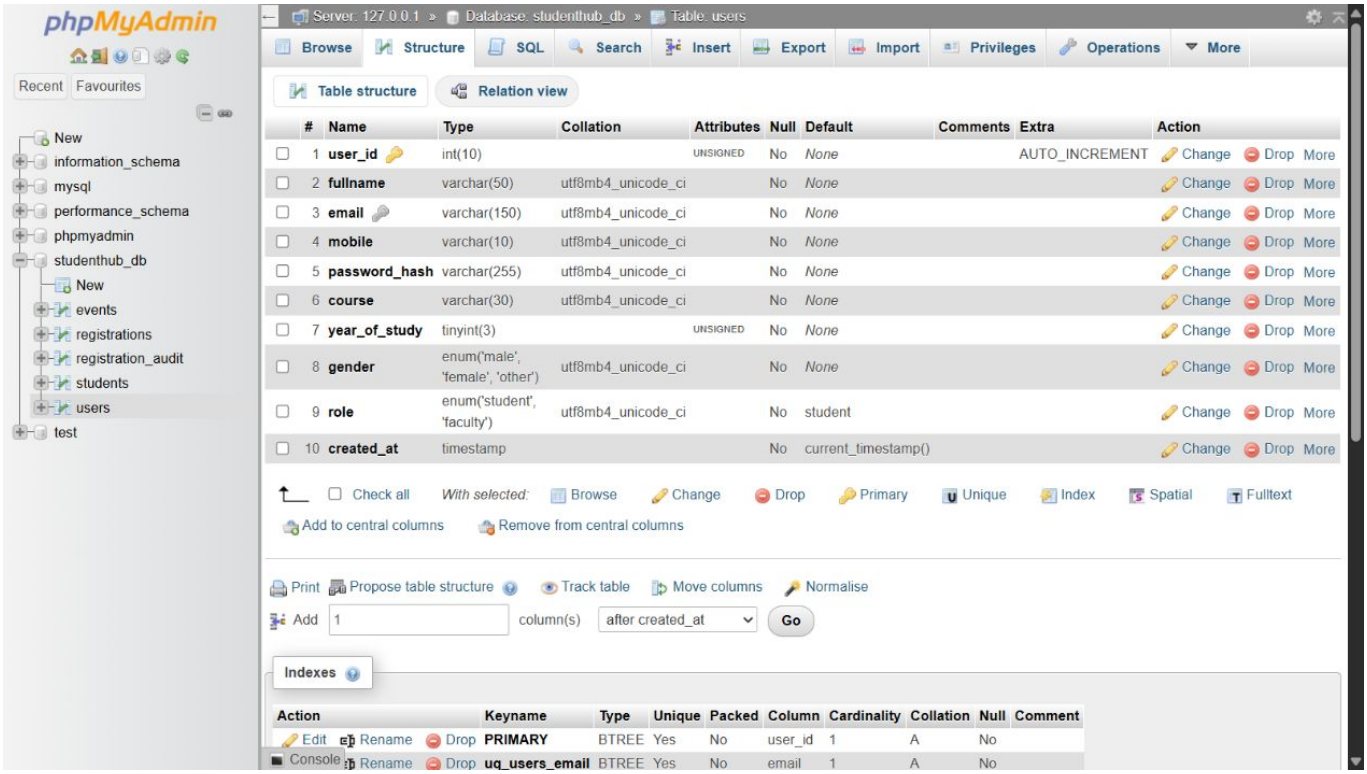


Figure 3: phpMyAdmin users table structure showing the password_hash column.

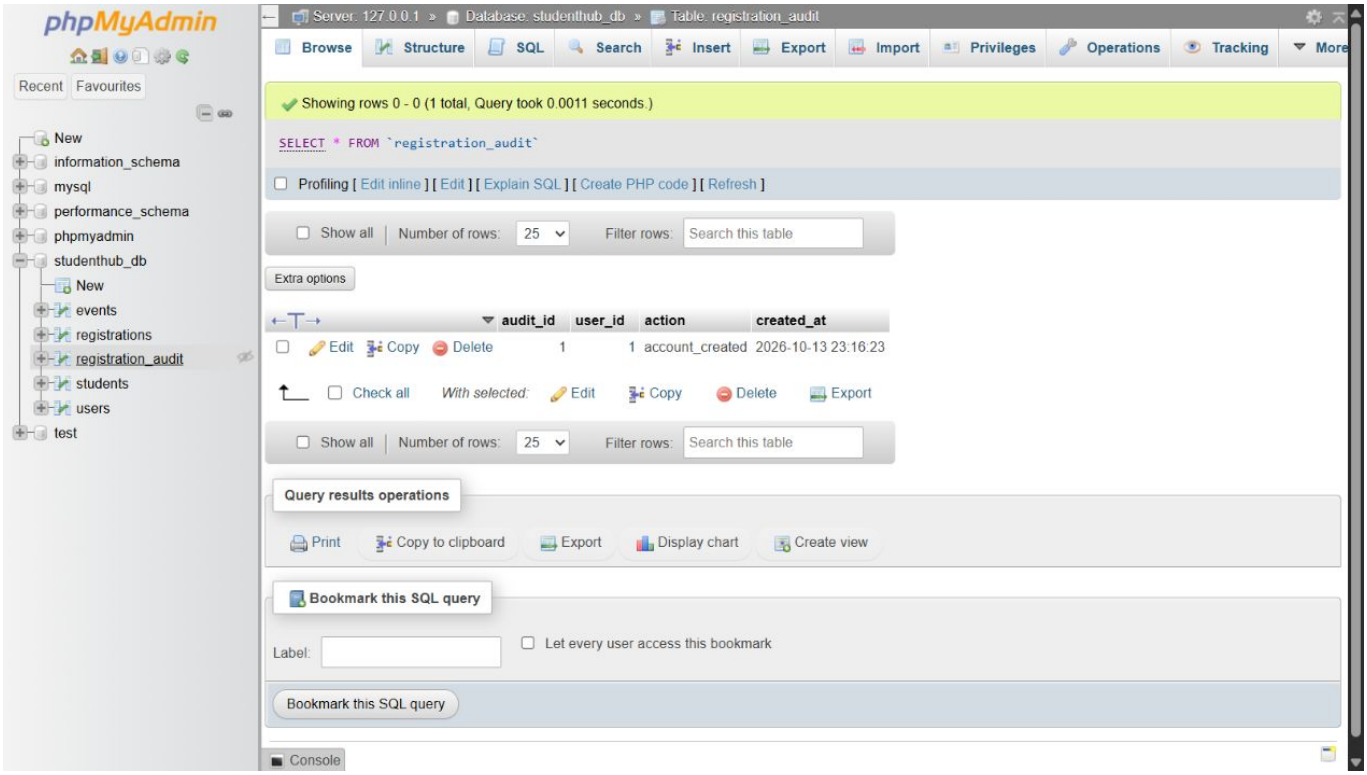


Figure 4: Successful account_created entry in registration_audit.

Database Structure and Registration Results

Table	Primary key	Other constraints
users	user_id	Unique email; year CHECK 1-4
registration_audit	audit_id	Foreign key referencing users.user_id

Valid Registration

Full name: Practical Test
Email: practical9.1791913582@example.com
Course: BCA; year: 1
Result: Welcome, Practical Test! Your StudentHub account has been created.

Database Verification

Check	Observed result
Inserted account	User ID 1; Practical Test
Password storage	Hash stored instead of plaintext
Correct password	Verified successfully
Incorrect password	Rejected
Audit log	One account_created entry

Testing Summary

Test input or action	Observed result
Valid registration	Account created successfully.
Duplicate email / changed email case	Rejected: this email is already registered.
Invalid email	Rejected with an email validation error.
Weak password / mismatched confirmation	Rejected with password errors.
Invalid course, year or role	Rejected with field validation errors.
Terms not accepted	Rejected: terms must be accepted.
Incorrect security code	Rejected with a security-code error.
Incorrect CSRF token	Rejected with a session error.

Registration URL: http://localhost/studenthub-portal/register.html

Key Questions and Answers

Q1 Is password_hash() used before storing passwords

Yes. PHP calls password_hash() with PASSWORD_DEFAULT before inserting the account. Only the resulting hash is stored in password_hash. The stored hash was verified using password_verify().

Q2 Is duplicate email/username checked

Email is used as the unique identifier in the StudentHub registration form. A prepared query checks for an existing email, and a UNIQUE constraint prevents duplicate inserts. Tests rejected identical emails and duplicates with different capitalization.

Q3 Are frontend and backend validations consistent

JavaScript and PHP apply matching rules for names, emails, mobile numbers, passwords and allowed field values. PHP also verifies the CSRF token and security code. Invalid-input tests confirmed that backend validation rejects unsuitable data.

Q4 Why use a prepared statement

Prepared statements keep SQL structure separate from supplied values. User input is bound as parameters rather than concatenated into the query, protecting the insert and duplicate-email query from SQL injection.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 9 source commit: 6a89671
Message: Complete Practical 9 secure MySQLi user registration

Post Laboratory Work

Submit the registration source files, sql/practical9.sql and this registration test report covering valid, invalid and duplicate cases. Start Apache and MySQL, import the new table definitions into studenthub_db, and use the local database configuration to reproduce the setup.

Conclusion

Secure user registration was implemented using MySQLi. Valid details were inserted, duplicate emails were rejected, and passwords were stored as hashes. Validation and password verification tests passed. The intermediate prepared-statement extension and the advanced CSRF token and audit log extension were completed.

Verification scope: HTTP and database tests confirm the recorded results. Required-field validation was also checked in the browser. Concurrent duplicate requests and a forced audit-write failure were not tested.

PRACTICAL 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

AIM

To implement secure login and logout in StudentHub using PHP sessions, verify passwords, protect dashboards according to user roles, handle session timeout, and add a secure remember-me option.

Introduction

The StudentHub login form was connected to the users table created in Practical 9. Passwords are checked using password_verify(). After login, the session ID is regenerated and the user is redirected to the appropriate dashboard. Logout, unauthorized access and inactivity timeout were tested using sample accounts.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, phpMyAdmin, HTML, CSS, browser, Git and GitHub.

Files Used

File	Purpose
login.php	Login form, password verification and role redirect.
php/auth.php	Session settings, timeout, role checks and remember tokens.
dashboard.php	Protected student dashboard.
admin-dashboard.php / faculty-dashboard.php	Protected admin and faculty dashboards.
dashboard-router.php	Opens the dashboard matching the account role.
logout.php	POST logout, token revocation and session destruction.
php/access-denied.php	Unauthorized role message.
sql/practical10.sql	Admin role, last_login and remember_tokens table.
scripts/seed-practical10.php	Creates hashed student, admin and faculty lab accounts.

Key Skills Used

PHP sessions, cookies, password verification, session ID regeneration, role-based access, inactivity timeout, prepared statements, token hashing and logout.

Implementation Overview

1 Session Settings

PHP sessions use strict mode and cookie-only session IDs. Cookies are HttpOnly and SameSite=Lax. The Secure flag is enabled when the site uses HTTPS. Protected responses use Cache-Control: no-store.

2 Login and Password Verification

A prepared query finds the user by email. password_verify() checks the supplied password against the stored hash. Incorrect credentials display Invalid email or password. Successful login calls session_regenerate_id(true) before storing the user ID, name, role and activity time.

3 Role-Based Dashboards

Student, admin and faculty accounts open separate dashboards. Each dashboard checks the logged-in user and current database role before displaying content. A student opening the admin dashboard receives HTTP 403. Visitors without a session are sent to the login page.

4 Logout

The logout button submits a POST request with a CSRF token. The remember token is revoked, session data is cleared, the session cookie is expired and session_destroy() is called. The login page displays a logout confirmation.

5 Session Timeout

The server checks the time since the last authenticated request. After 15 minutes of inactivity, it clears the session and remember cookie and returns to login. A temporary two-second setting was used to test and capture timeout, then removed.

6 Last-Login Timestamp

The users table contains a last_login column. Successful password login and remember-me restoration update this timestamp. Database checks confirmed timestamps for all three lab accounts.

7 Remember-Me Token

Remember me creates a random selector and validator using random_bytes(). The database stores the SHA-256 hash of the validator with a 30-day expiry. The token is verified and rotated when restoring a session. Old and modified tokens are rejected. Logout and timeout revoke the token.

Screenshots of Successful Login and Role-Based Redirects

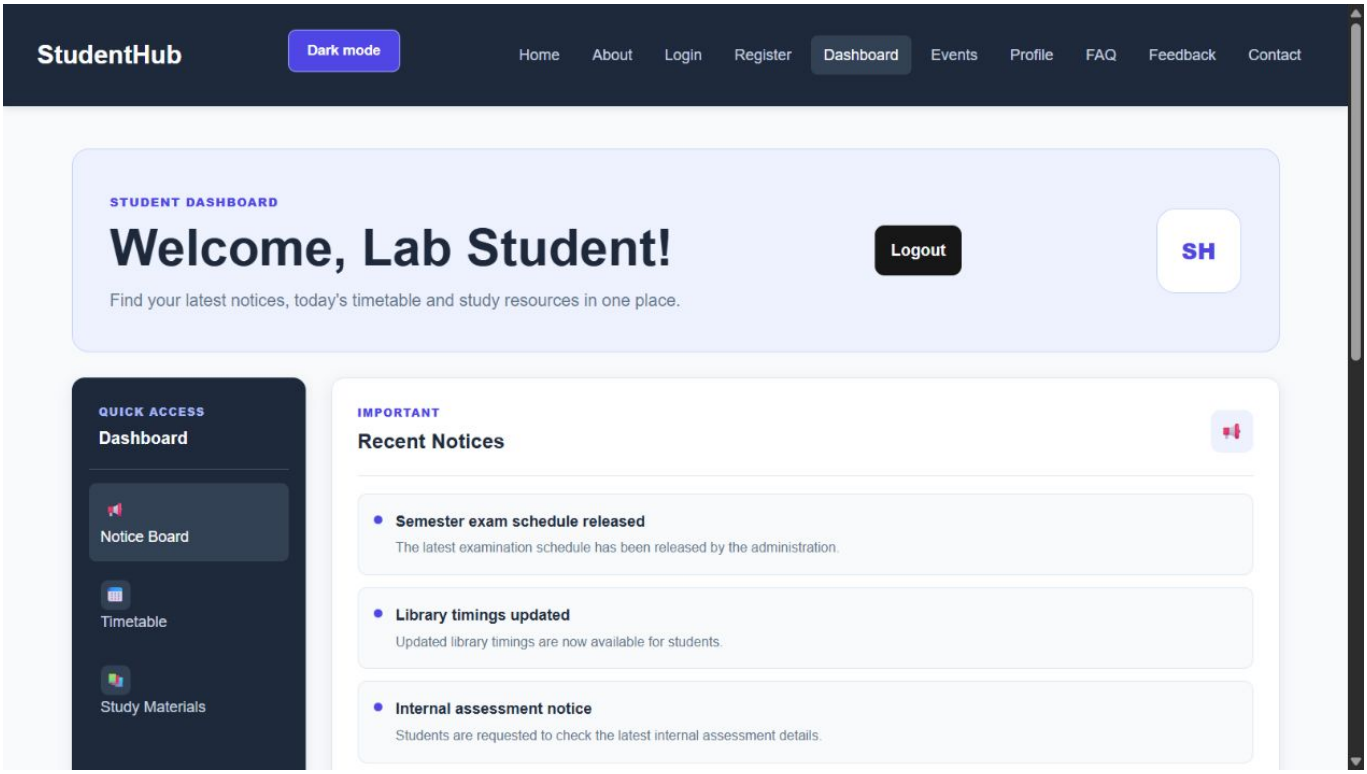


Figure 1: Successful student login displaying the student dashboard.

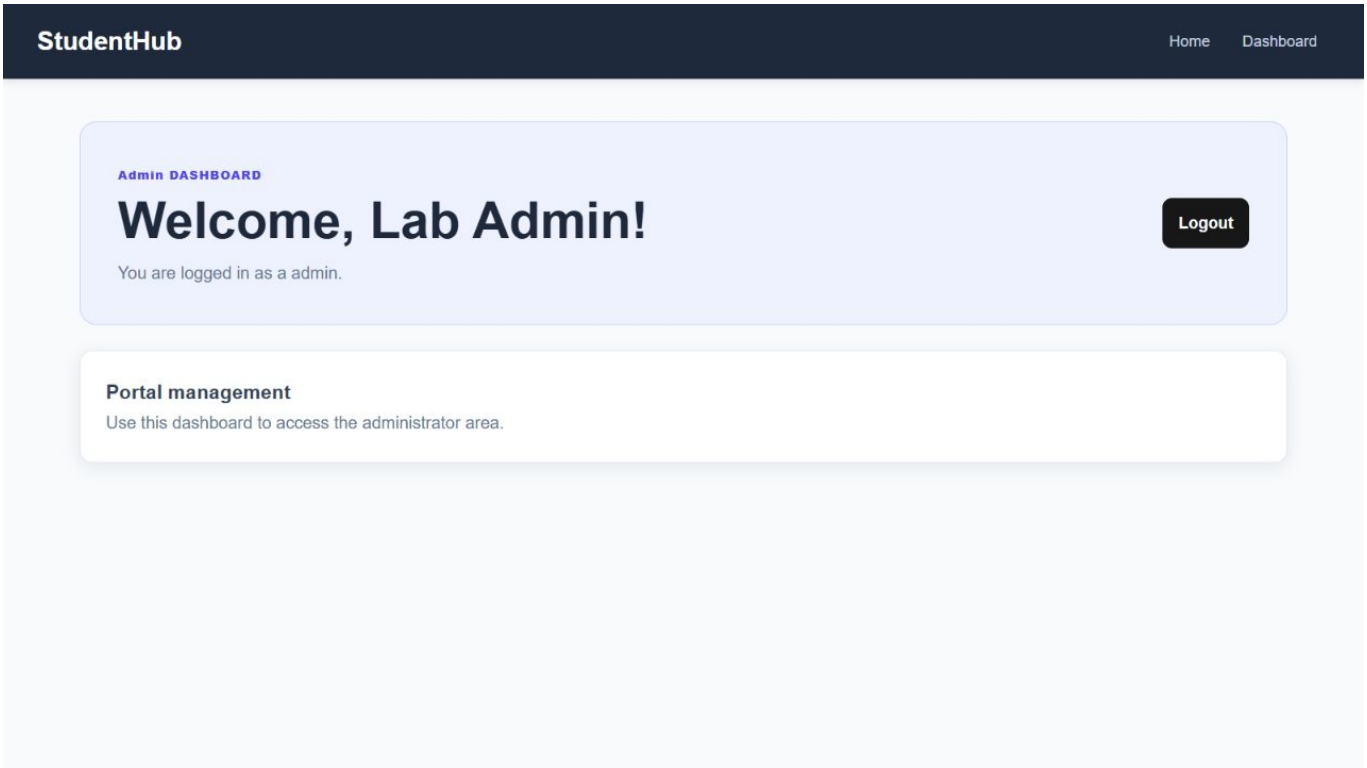


Figure 2: Successful admin login displaying the admin dashboard.

Screenshots of Failed Login and Logout

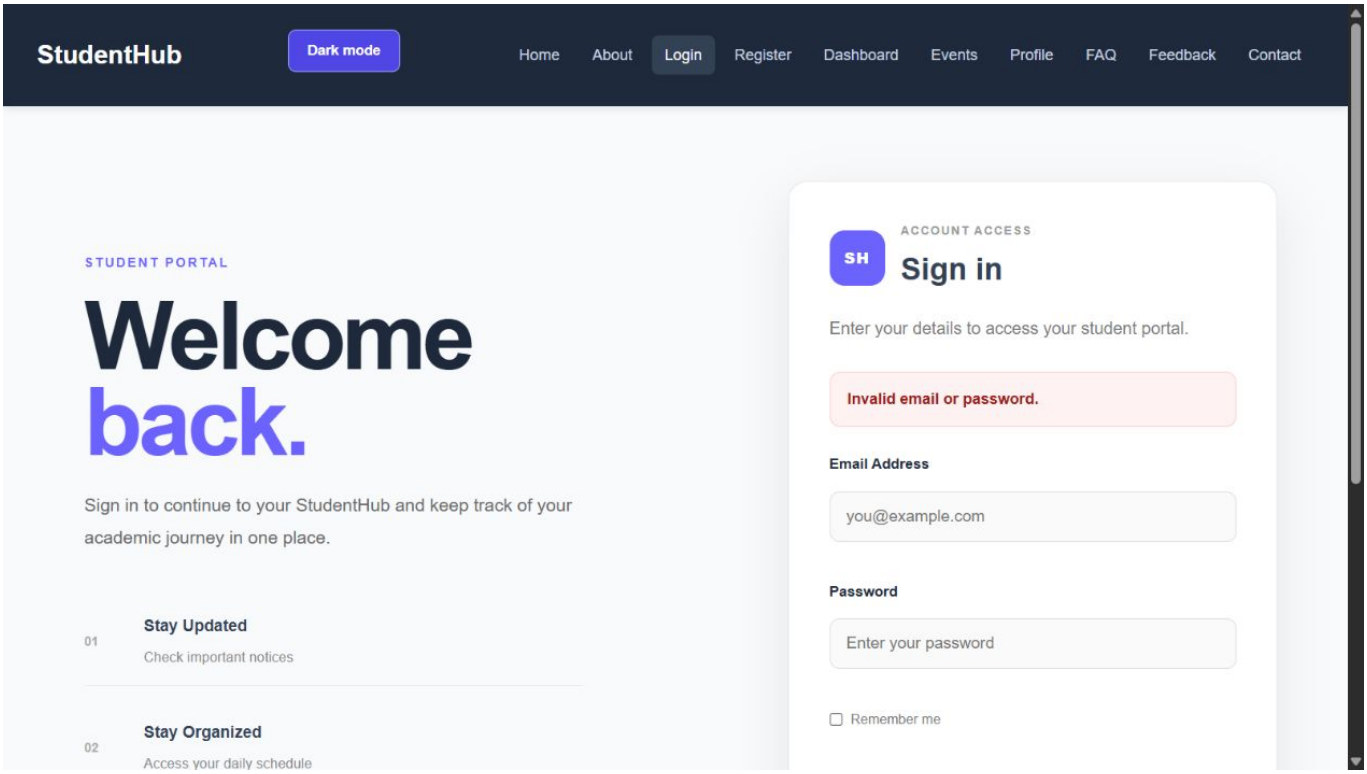


Figure 3: Incorrect password displaying the failed-login message.

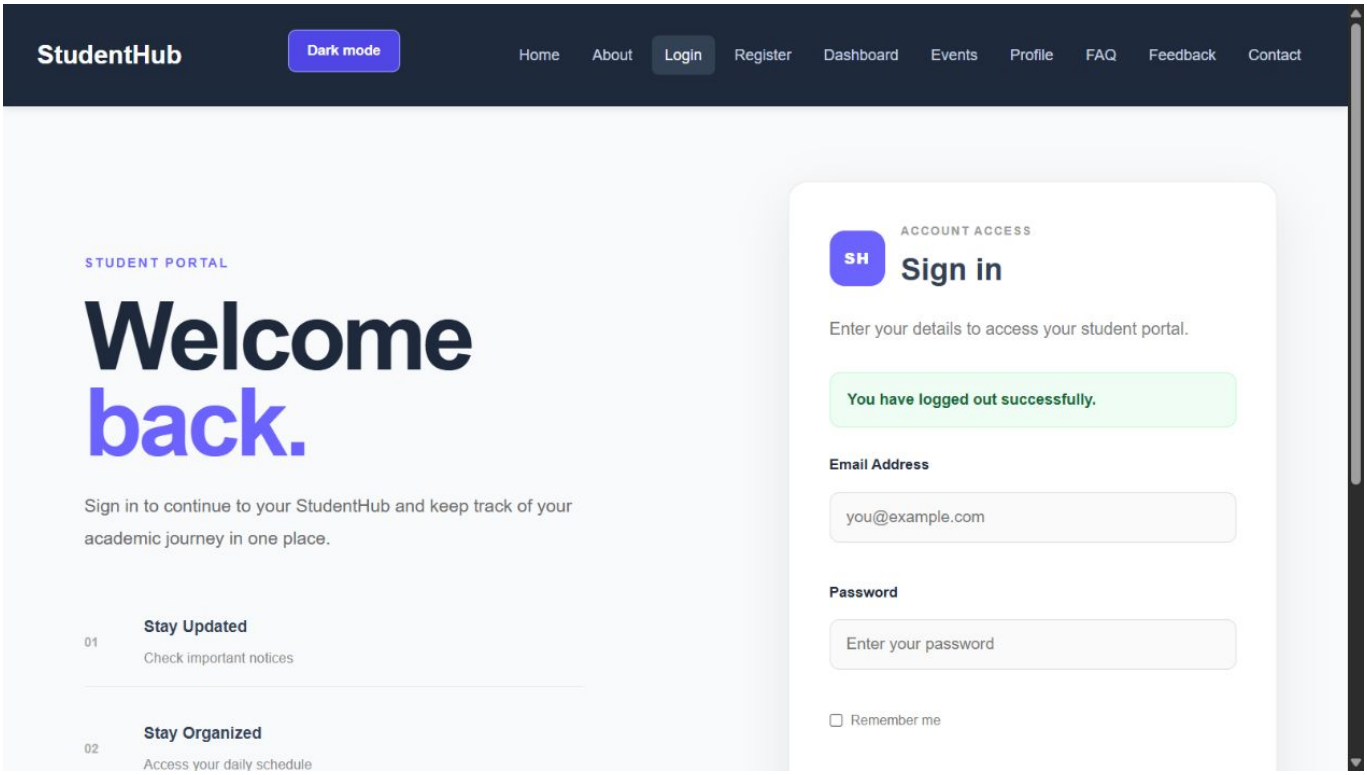


Figure 4: Successful logout returning the user to the login page.

Screenshots of Timeout and Unauthorized Role Access

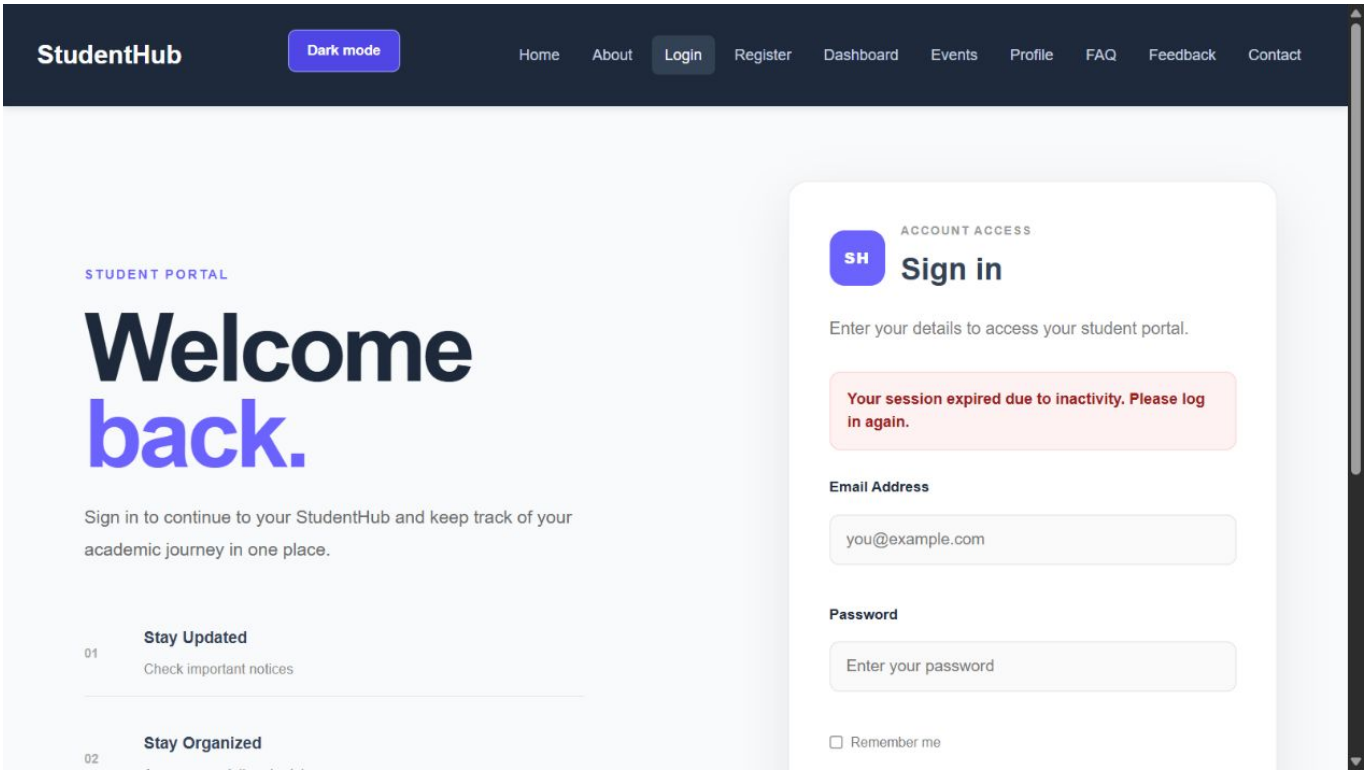


Figure 5: Expired session displaying the inactivity-timeout message.

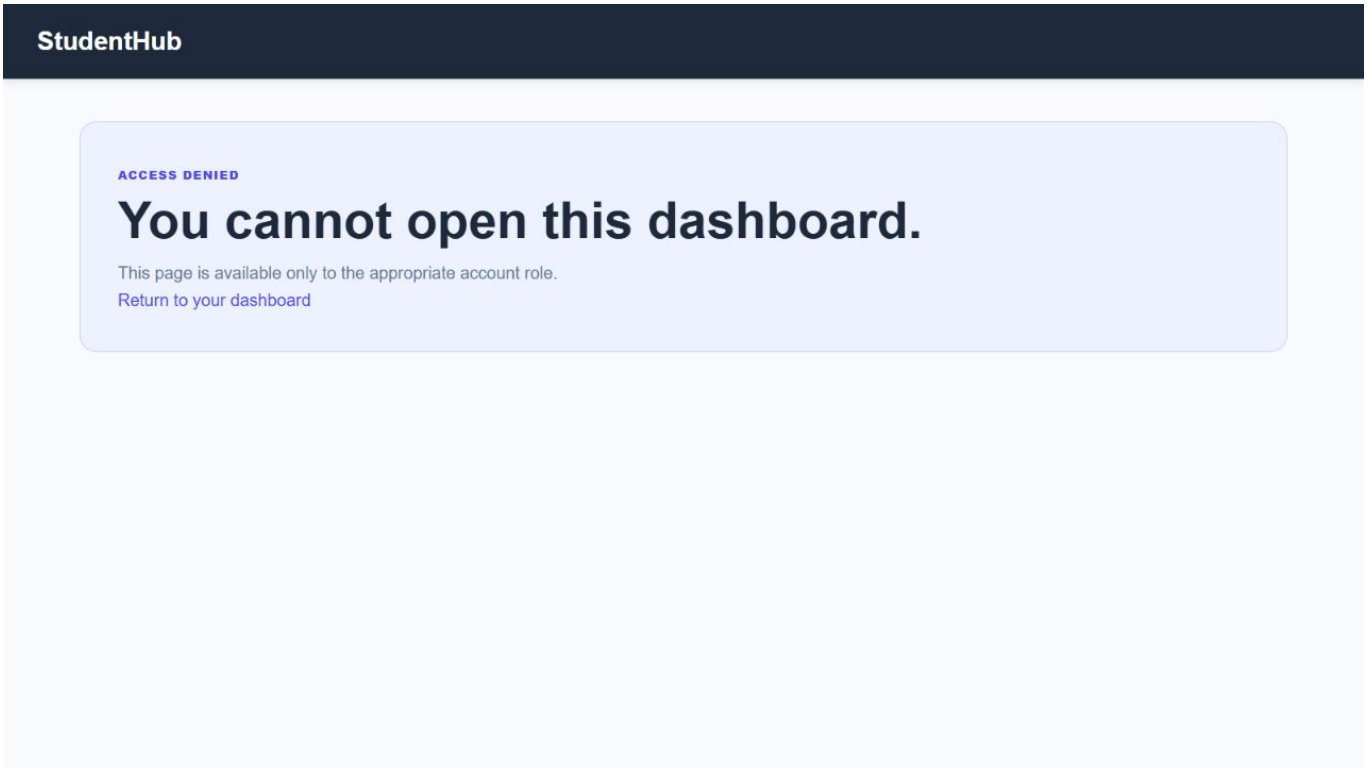


Figure 6: A student account is blocked from opening the admin dashboard.

Unauthorized Access and Testing Summary

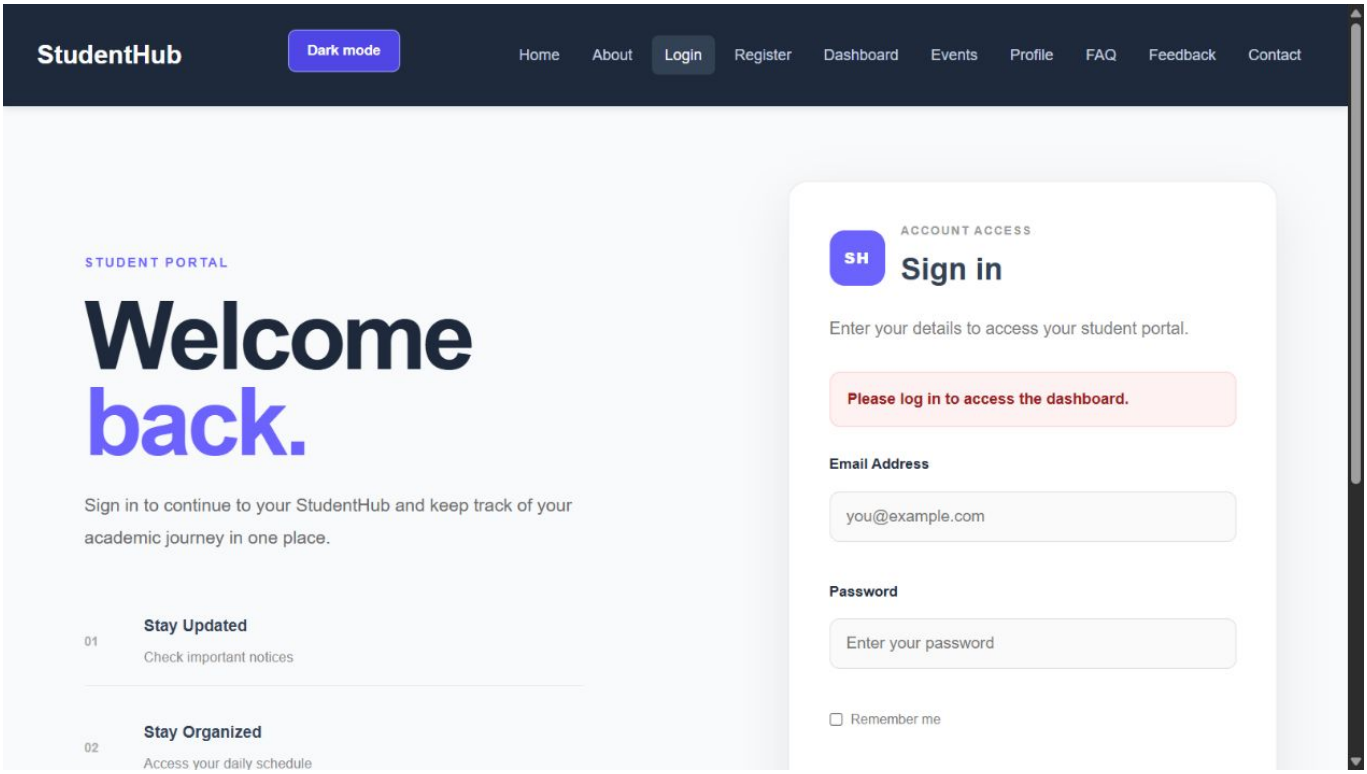


Figure 7: Opening the dashboard without login redirects to the login page.

Test input or action	Observed result
Correct student / admin / faculty login	Redirected to the matching dashboard.
Incorrect password	Invalid email or password displayed.
Session ID after login	Changed successfully.
Session cookie settings	HttpOnly and SameSite=Lax confirmed.
Anonymous / incorrect-role access	Login redirect / HTTP 403.
Logout / dashboard after logout	Session closed / login required.
Session timeout	Session expired; remember cookie cleared.
Remember me / token rotation	Login restored; a new token was issued.
Old / modified remember token	Rejected.
Last-login timestamp	Updated for all three lab accounts.

All 25 HTTP authentication checks passed. The normal session timeout is 15 minutes. Local tests used HTTP; the HTTPS Secure-cookie flag was not tested in a browser.

Key Questions and Answers

Q1 Are sessions started and destroyed securely

Sessions start with strict mode and cookie-only IDs. HttpOnly and SameSite settings are applied, and the ID is regenerated after login. Logout clears the session, expires the cookie and calls session_destroy().

Q2 Is password_verify() used correctly

Yes. The supplied login password is checked against password_hash from the users table. Correct credentials allow login, while incorrect credentials display a general error message.

Q3 Is unauthorized dashboard access blocked

Yes. PHP checks authentication and the database role before rendering a dashboard. Anonymous users must log in, and users with the wrong role receive HTTP 403.

Q4 Is session timeout implemented

Yes. The last_activity value is checked on authenticated requests. After 15 minutes of inactivity, the session is destroyed and the user is asked to log in again.

Q5 Are admin and student users redirected to appropriate dashboards

Yes. Student accounts open dashboard.php and admin accounts open admin-dashboard.php. Faculty accounts have their own dashboard. All three redirects were tested.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 10 source commit: aebd4ed
Message: Complete Practical 10 session login logout and role-based access

Post Laboratory Work

Submit the login and session source files, sql/practical10.sql and this report containing screenshots for login success, failed login, logout, timeout and unauthorized access. Use the seeded sample accounts to repeat the tests.

Conclusion

Secure login and logout were implemented using PHP sessions. Role-based dashboard access, password verification and inactivity timeout worked correctly in the tests. The intermediate last-login timestamp and advanced remember-me token extensions were also completed.

PRACTICAL 11

Student Management CRUD with Search and Filter

AIM

To create a student management module in StudentHub for adding, viewing, editing, deleting, searching and filtering student records using MySQLi prepared statements.

Introduction

A student management page was added to the admin dashboard. Student details are stored in the students table and displayed with pagination. The page provides validation and clear success or failure messages. Twenty sample students were added, along with CSV export and soft deletion using a status column.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, phpMyAdmin, HTML, CSS, browser, Git and GitHub.

Files Used

File	Purpose
students-manage.php	Student form, list, edit, delete confirmation and CSV export.
php/student-functions.php	Validation, search filters and prepared queries.
css/student-management.css	Form, table and pagination styling.
admin-dashboard.php	Link to the student management page.
sql/practical11.sql	Status column and twenty fictional seed records.
sql/practical11-students-dump.sql	Exported students table with structure and data.
scripts/test-practical11.py	Repeatable CRUD, search and access tests.
scripts/export-practical11.php	Generates the students-only SQL dump.

Key Skills Used

PHP-MySQL CRUD, prepared statements, validation, search, filters, pagination, admin access, CSV export, soft deletion and user feedback messages.

Implementation Overview

1 Database and Sample Records

The existing students table stores student ID, full name, email, mobile, course, year and creation time. A status column was added with active and deleted values. The seed file adds twenty fictional students while retaining the existing records.

2 Add and View Students

The Add student form validates the entered details and uses a prepared INSERT statement. A SELECT query loads the student list. Duplicate emails and invalid fields are rejected with appropriate messages. Only admin accounts can access this module.

3 Edit Student Details

Edit opens the selected record in the form. Saving the changes uses a prepared UPDATE statement. The student ID and email are checked, and a success message confirms the update.

4 Delete Confirmation and Soft Delete

Delete opens a confirmation page displaying the student name and email. The user must confirm before submitting. The advanced soft-delete option uses UPDATE to set status to deleted instead of physically removing the row. The record is hidden from the active list and can be restored.

5 Search and Filters

Search works on name, email and course, including displayed course names such as B.Tech IT. Course, year and status filters can be combined. Prepared parameters are used for search values, and no-results searches display a clear message.

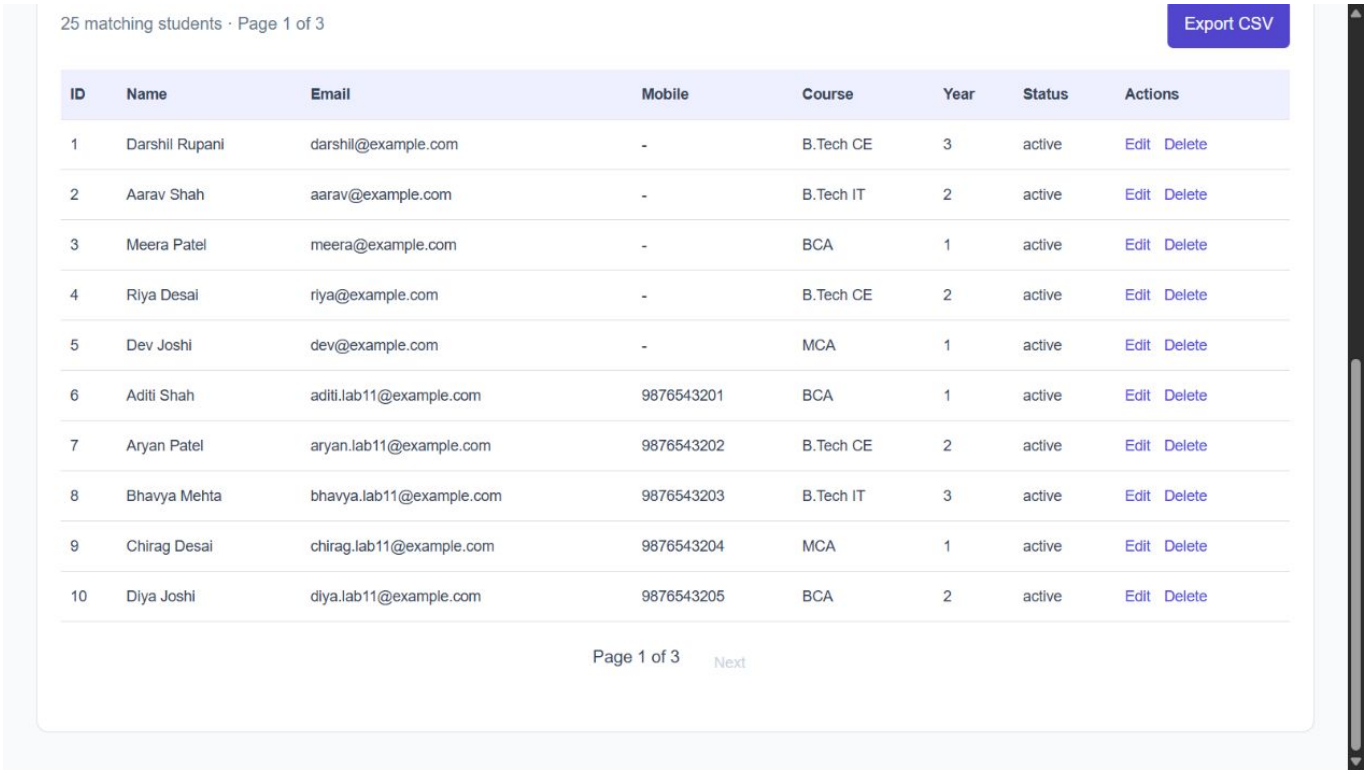
6 Pagination and CSV Export

The list displays ten records per page. Previous and Next links preserve the current search and filters. Export CSV downloads all matching records across pages. Values that could be interpreted as spreadsheet formulas are escaped.

7 SQL Dump and Testing

The student table structure and records were exported to a separate SQL file. The dump was imported successfully into a temporary table for verification. CRUD operations, invalid inputs, duplicate emails, access checks, search, filters, pagination and CSV export were tested.

Screenshots of Student List and Add Operation



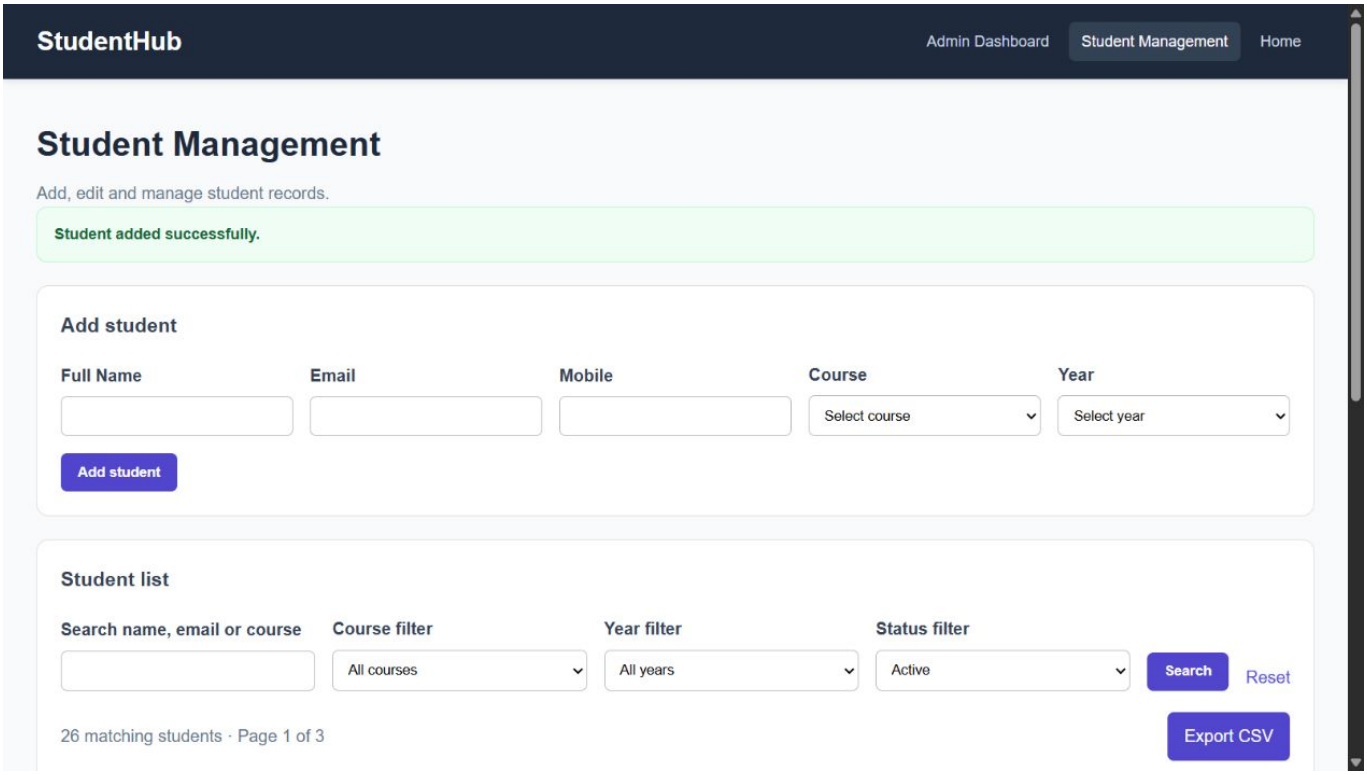
25 matching students · Page 1 of 3

Export CSV

ID	Name	Email	Mobile	Course	Year	Status	Actions
1	Darshil Rupani	darshil@example.com	-	B.Tech CE	3	active	Edit Delete
2	Aarav Shah	aarav@example.com	-	B.Tech IT	2	active	Edit Delete
3	Meera Patel	meera@example.com	-	BCA	1	active	Edit Delete
4	Riya Desai	riya@example.com	-	B.Tech CE	2	active	Edit Delete
5	Dev Joshi	dev@example.com	-	MCA	1	active	Edit Delete
6	Aditi Shah	aditi.lab11@example.com	9876543201	BCA	1	active	Edit Delete
7	Aryan Patel	aryan.lab11@example.com	9876543202	B.Tech CE	2	active	Edit Delete
8	Bhavya Mehta	bhavya.lab11@example.com	9876543203	B.Tech IT	3	active	Edit Delete
9	Chirag Desai	chirag.lab11@example.com	9876543204	MCA	1	active	Edit Delete
10	Diya Joshi	diya.lab11@example.com	9876543205	BCA	2	active	Edit Delete

Page 1 of 3 Next

Figure 1: Database student list displaying the first page of records.



StudentHub

Admin Dashboard | Student Management | Home

Student Management

Add, edit and manage student records.

Student added successfully.

Add student

Full Name | Email | Mobile | Course | Year

Select course | Select year

Add student

Student list

Search name, email or course | Course filter | Year filter | Status filter

All courses | All years | Active

Search Reset

26 matching students · Page 1 of 3

Export CSV

Figure 2: Valid student details added successfully.

Screenshots of Edit, Search and Filter

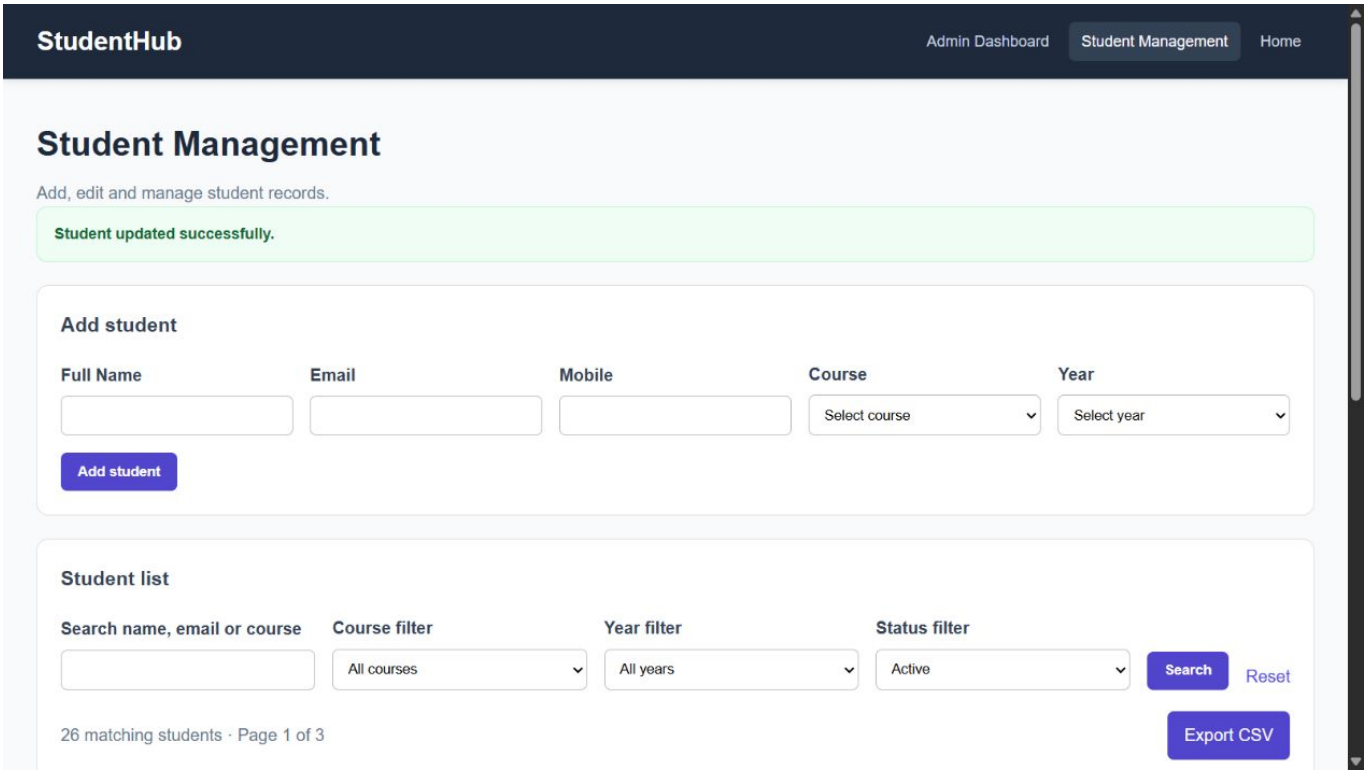


Figure 3: Student details updated successfully.

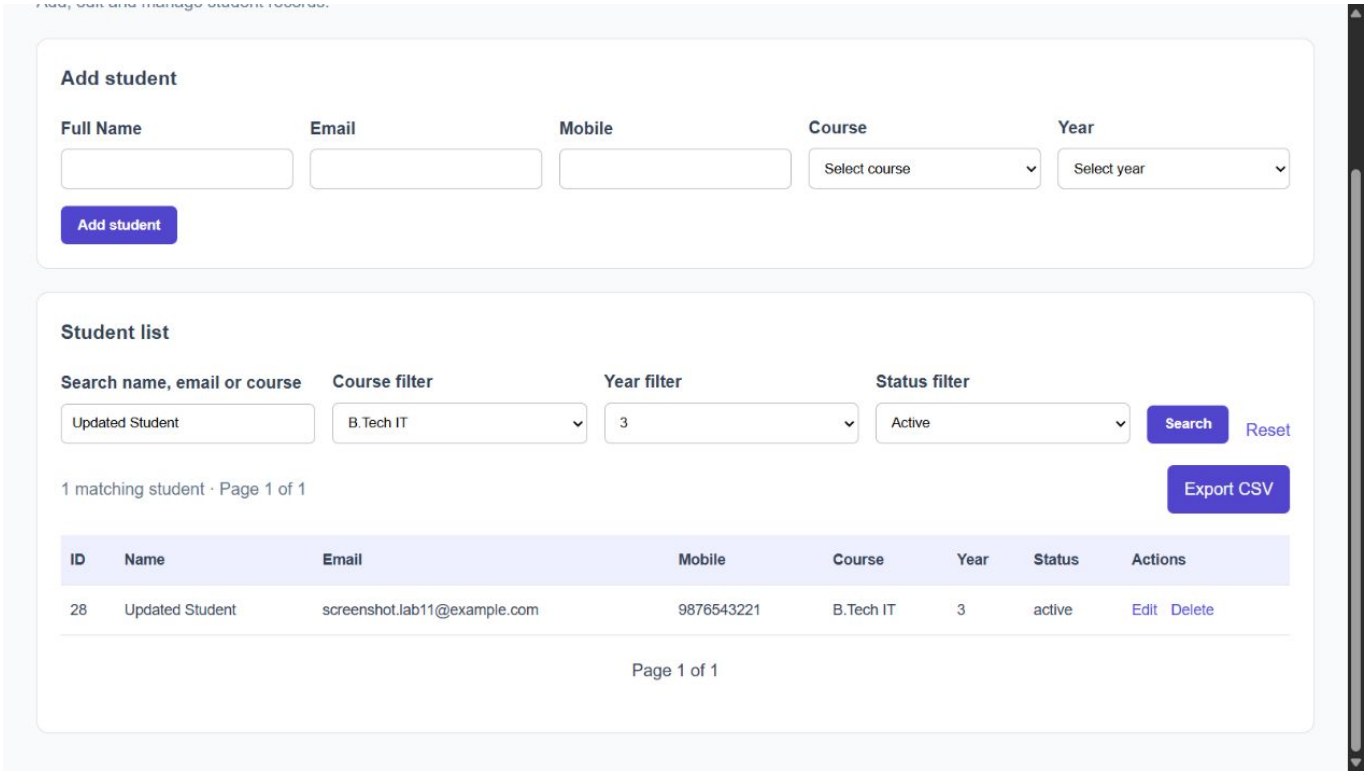


Figure 4: Search by name combined with course and year filters.

Screenshots of Delete Confirmation and Result

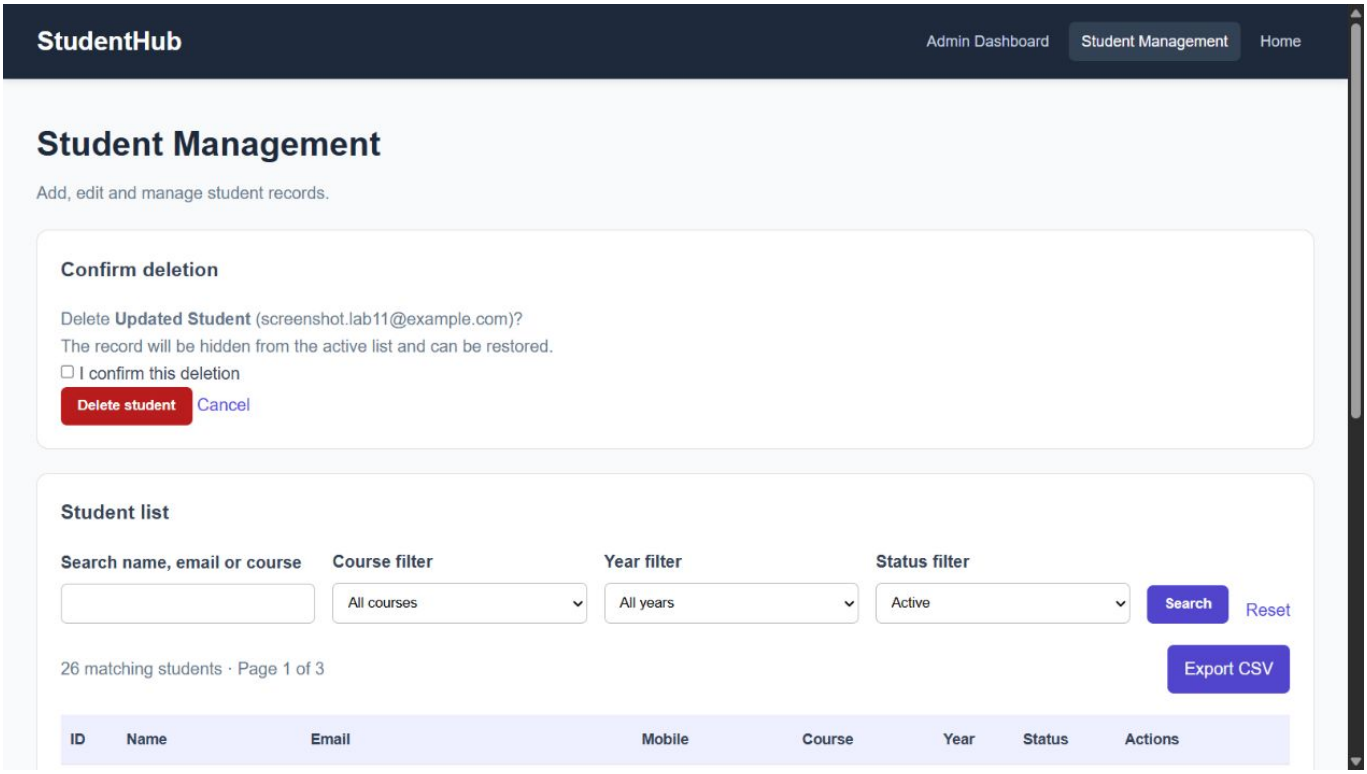


Figure 5: Confirmation is required before deleting the selected student.

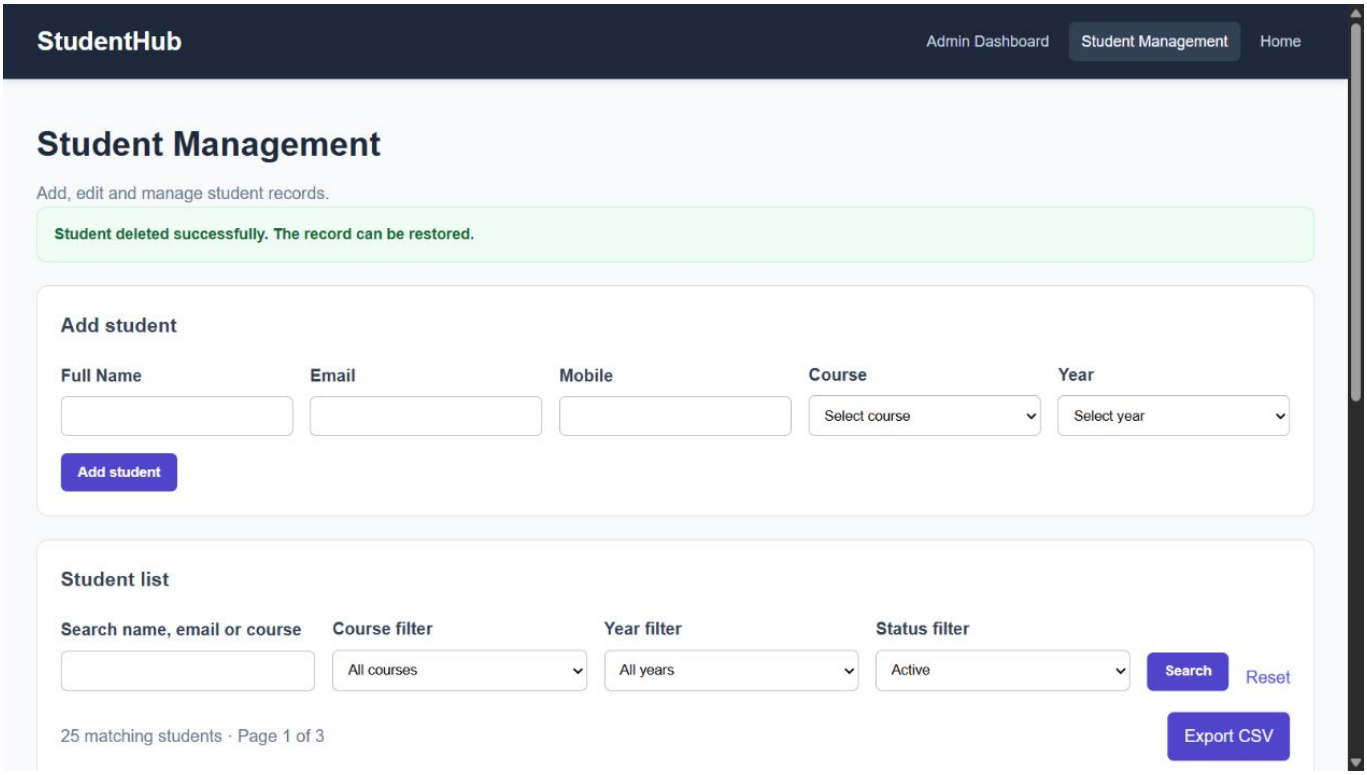


Figure 6: Successful soft deletion with a clear confirmation message.

Screenshots of Soft Delete and Pagination

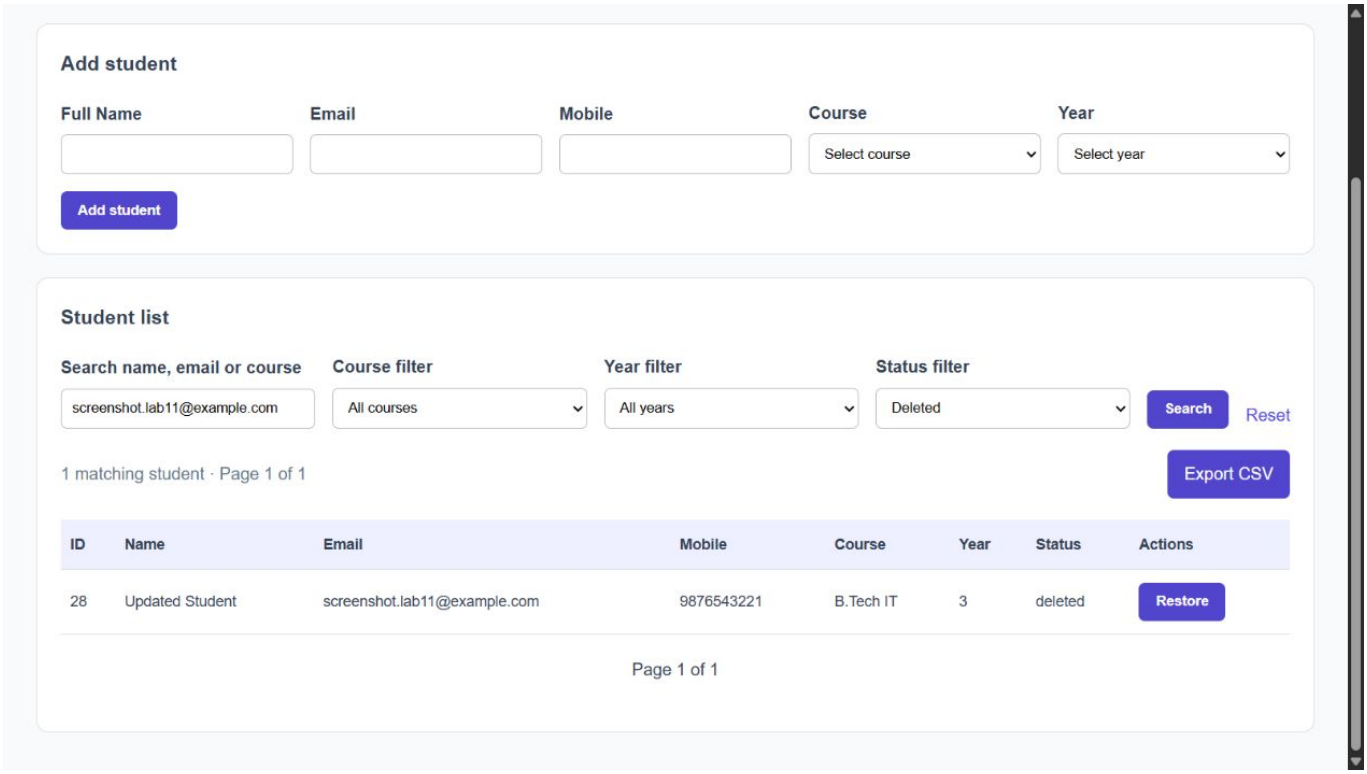


Figure 7: The deleted record remains stored and has a Restore option.

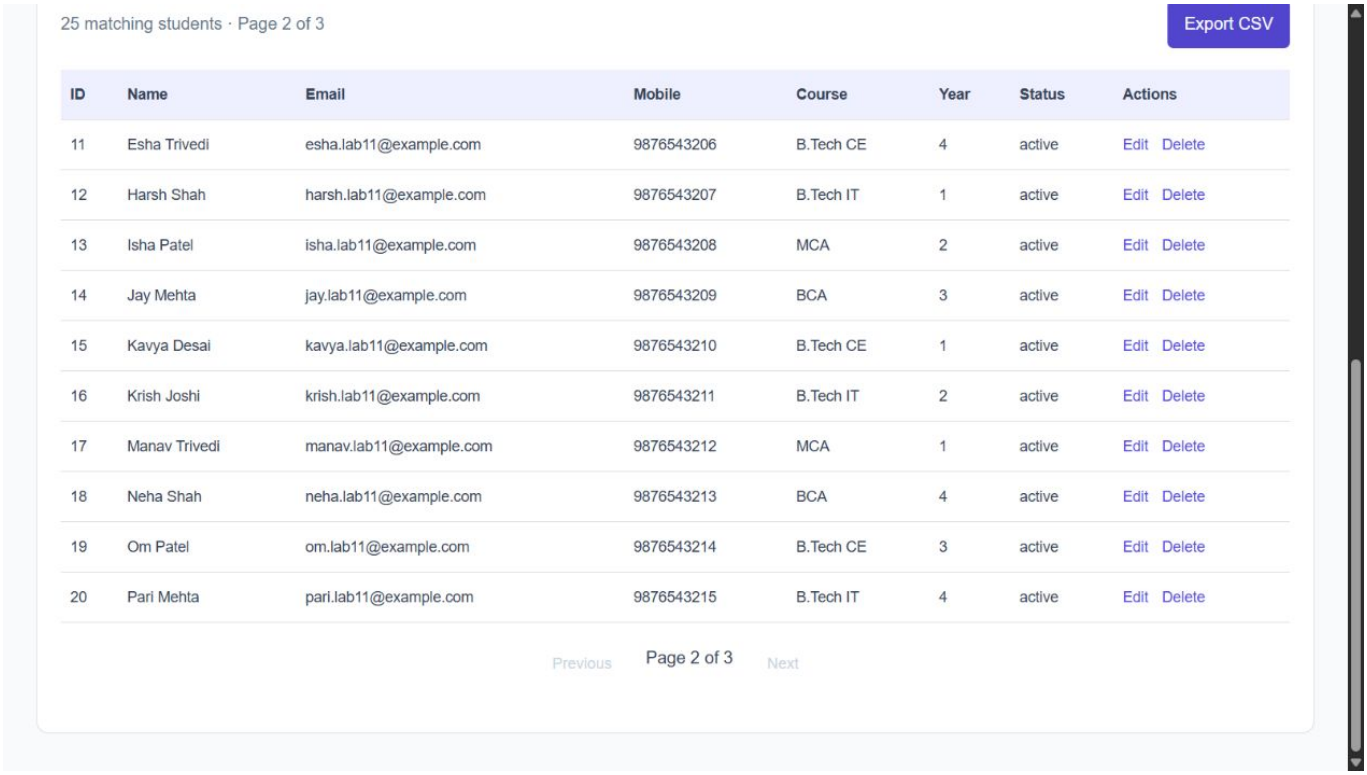


Figure 8: Second page of student records with Previous and Next links.

Database and Export Results

Check	Observed result
New fictional seed records	20
Active students after testing	25
Records in SQL dump	29
SQL dump verification	Imported successfully into a temporary table.
Sample CSV	25 active students; seven columns.

Testing Summary

Test input or action	Observed result
Admin / student / anonymous access	Allowed / HTTP 403 / login redirect.
Add valid student	Student added successfully.
Duplicate email / invalid fields	Rejected with clear validation messages.
Edit student	Updated name, course and year saved.
Search by name / email / course	Matching records returned.
Displayed course name search	B.Tech IT matches its stored course code.
Course and year filters	Only matching course/year records returned.
Pagination / empty search	Correct page / No students found.
SQL text / literal percent search	Treated as search data; no unexpected rows.
Delete without confirmation	Rejected; record retained.
Confirmed deletion	Status changed to deleted.
Deleted status / Restore	Record remains available and can be restored.
CSV export	All matching records exported across pages.
Invalid CSRF / missing student	HTTP 403 / clear not-found message.

All 26 HTTP checks passed. The twenty seed records and the existing sample students remain active. The four demonstration records remain soft-deleted.

Key Questions and Answers

Q1 Are SELECT, INSERT, UPDATE and DELETE operations working

SELECT loads the list, INSERT adds students and UPDATE saves edits. The delete action works through the advanced soft-delete approach: status is updated to deleted instead of issuing a permanent SQL DELETE. Add, edit, delete and restore tests passed.

Q2 Is search by name/email/course implemented

Yes. A prepared query searches name, email and course. The displayed course name is also supported. Course, year and status filters can be applied together, and pagination retains the selected filters.

Q3 Are delete operations confirmed before execution

Yes. Clicking Delete opens a confirmation page with the selected student details. A confirmation checkbox and CSRF token are required before the POST request changes the status. Submitting without confirmation was rejected.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 11 source commit: d16c903
Message: Complete Practical 11 student CRUD search filters and soft delete

Conclusion

The student management module was completed using MySQLi prepared statements. Add, view, edit, confirmed deletion, search, filters and pagination worked correctly in the tests. The intermediate CSV export and advanced soft-delete extensions were completed. The SQL dump was exported and verified successfully.

PRACTICAL 12

Event Management CRUD with Poster Upload and File Validation

AIM

To develop an admin event management module in StudentHub with add, view, edit and delete operations, along with secure poster upload and file validation.

Introduction

The admin dashboard provides a Manage events page. Event details and poster filenames are stored in MySQL. The form validates the event fields and uploaded image before saving. Category and open/closed filters, random image names, upload directory protection and an audit log are also included.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, Bootstrap 5.3.3, HTML, CSS, JavaScript, browser, Git and GitHub.

Files Used

File	Purpose
events-manage.php	Event form, list, edit and confirmed deletion.
php/event-functions.php	Prepared queries, validation, posters and audit log.
event-poster.php	Displays a poster using its database event ID.
php/get-events.php	Loads database events for the campus event page.
css/event-management.css	Event form and list styling with Bootstrap.
sql/practical12.sql	Event columns, sample records and audit table.
sql/practical12-events-dump.sql	Events table structure and sample data.
scripts/setup-practical12.php	Creates the protected poster directory.
scripts/test-practical12.py	Repeatable CRUD and upload validation tests.

Key Skills Used

PHP-MySQL CRUD, prepared statements, image upload, server-side validation, event status, admin access, filters and feedback messages.

Implementation Overview

1 Database and Admin Access

The events table stores title, description, event date, venue, category, status and optional poster filename. The module is linked from the admin dashboard. Anonymous visitors are redirected to login and student accounts are denied access.

2 Add, View and Edit Events

The form checks title, venue, date, category and status. MySQLi prepared INSERT, SELECT and UPDATE statements save and display the records. Editing without a new image keeps the current poster. Changes are also displayed on the public campus event page.

3 Poster Type and Size Validation

JPEG, PNG and WebP images are accepted up to 2 MB. The server checks actual file size, MIME type, matching extension and image contents. Image dimensions must not exceed 4096 by 4096 pixels. Text, PHP, SVG, mismatched and oversized files are rejected with clear messages.

4 Delete Confirmation

Delete opens a confirmation page before making changes. Confirmation and a valid CSRF token are required. An unregistered event is removed with a prepared DELETE statement. Events with registrations are retained and can be marked closed instead.

5 Category and Status Filters

Search matches title or venue. Category and open/closed status filters can be combined with search. The list displays the matching records or a no-results message.

6 Image Renaming and Directory Protection

Each uploaded image receives a random hexadecimal filename. Direct access to the upload directory is blocked by its Apache configuration. Posters are served through the event ID. Replaced and deleted poster files are removed after a successful database commit.

7 Audit Log and SQL Dump

Add, update and delete actions are recorded in event_audit with the event title, admin ID, time and before/after values. Database changes and audit entries use a transaction. The events-only SQL dump was exported and imported into a temporary table for verification.

Screenshots of Add Operation and Poster

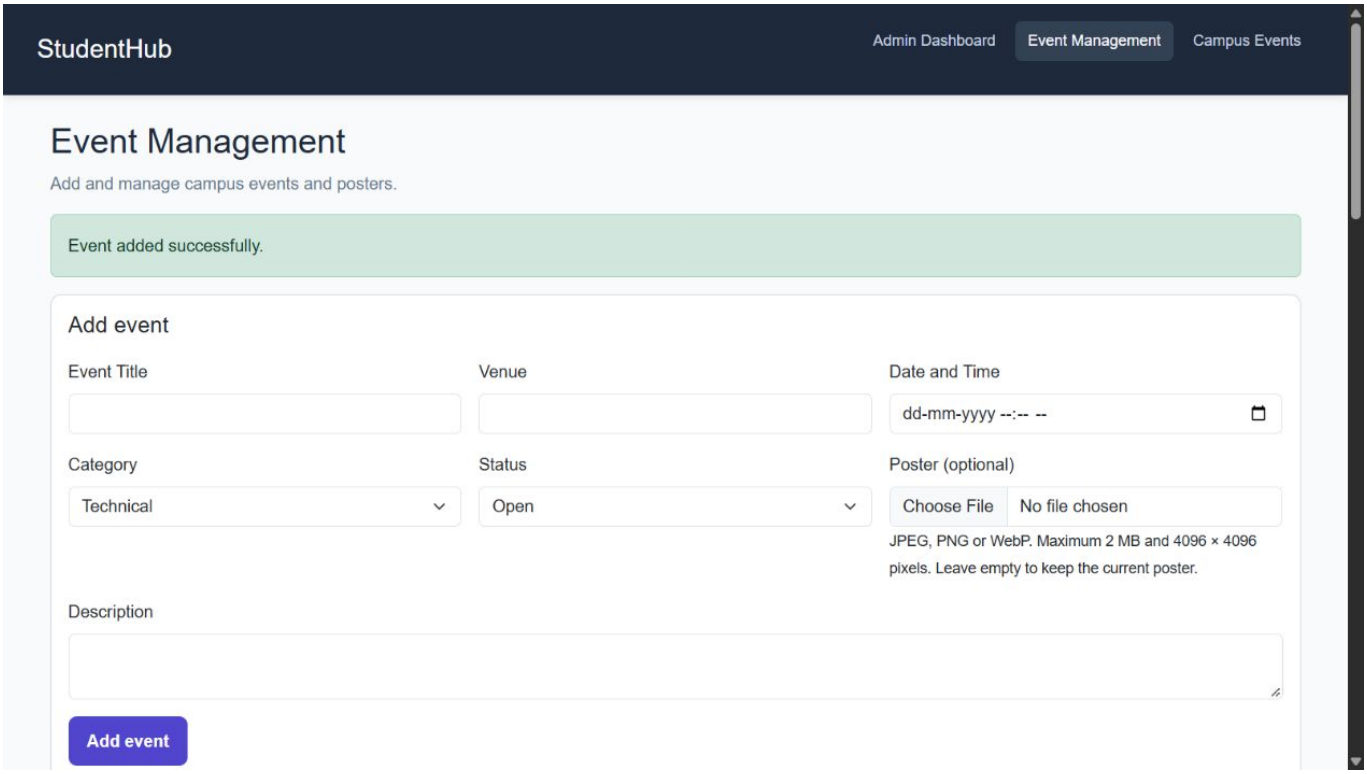


Figure 1: Event added successfully with a valid PNG poster.

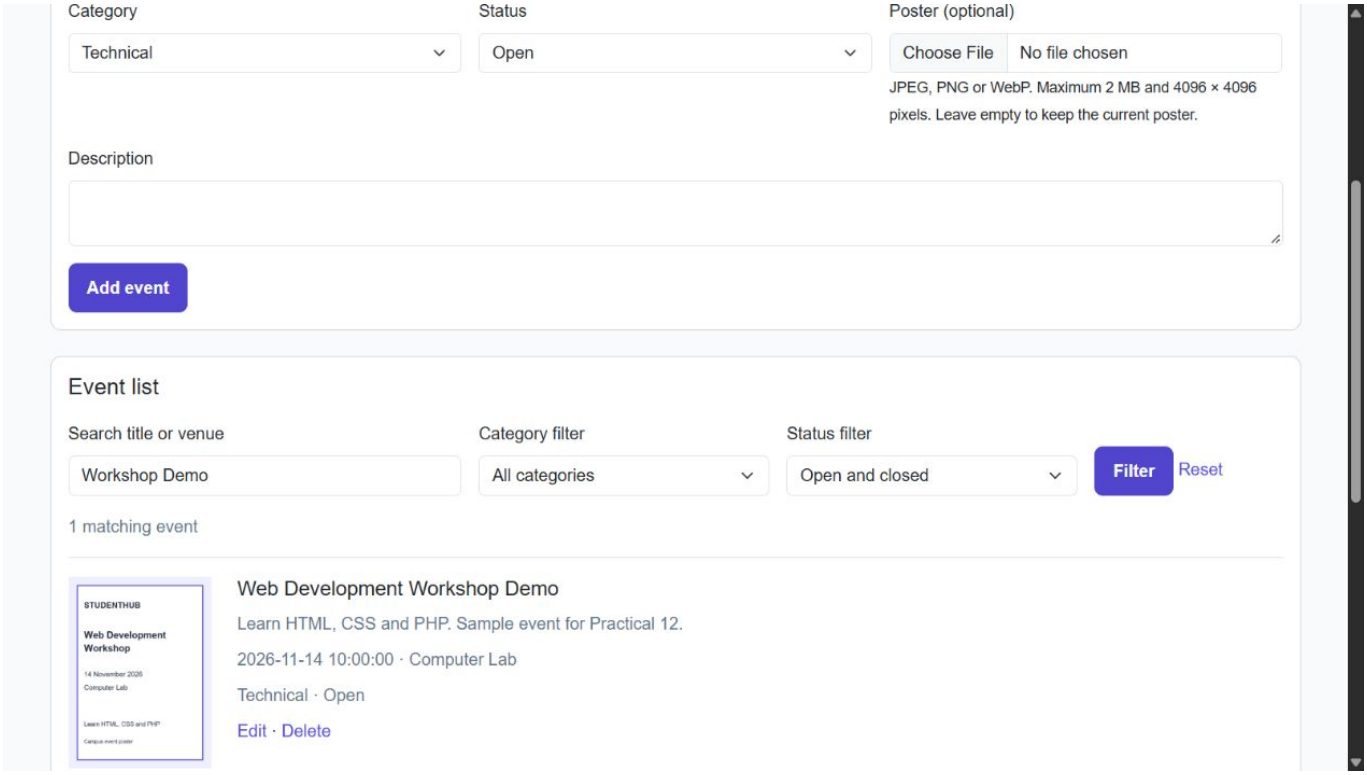


Figure 2: Saved event and poster displayed in the database event list.

Screenshots of Edit and Filters

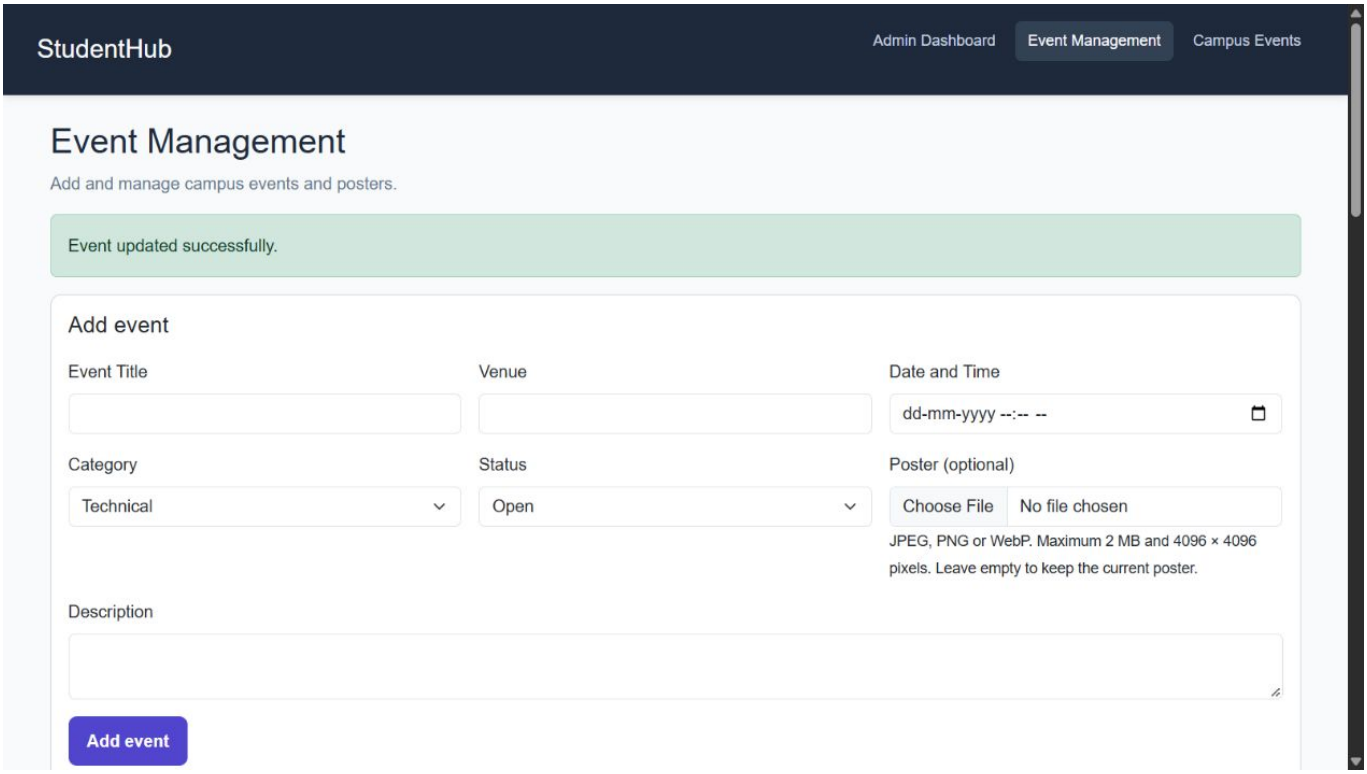


Figure 3: Event details updated successfully.

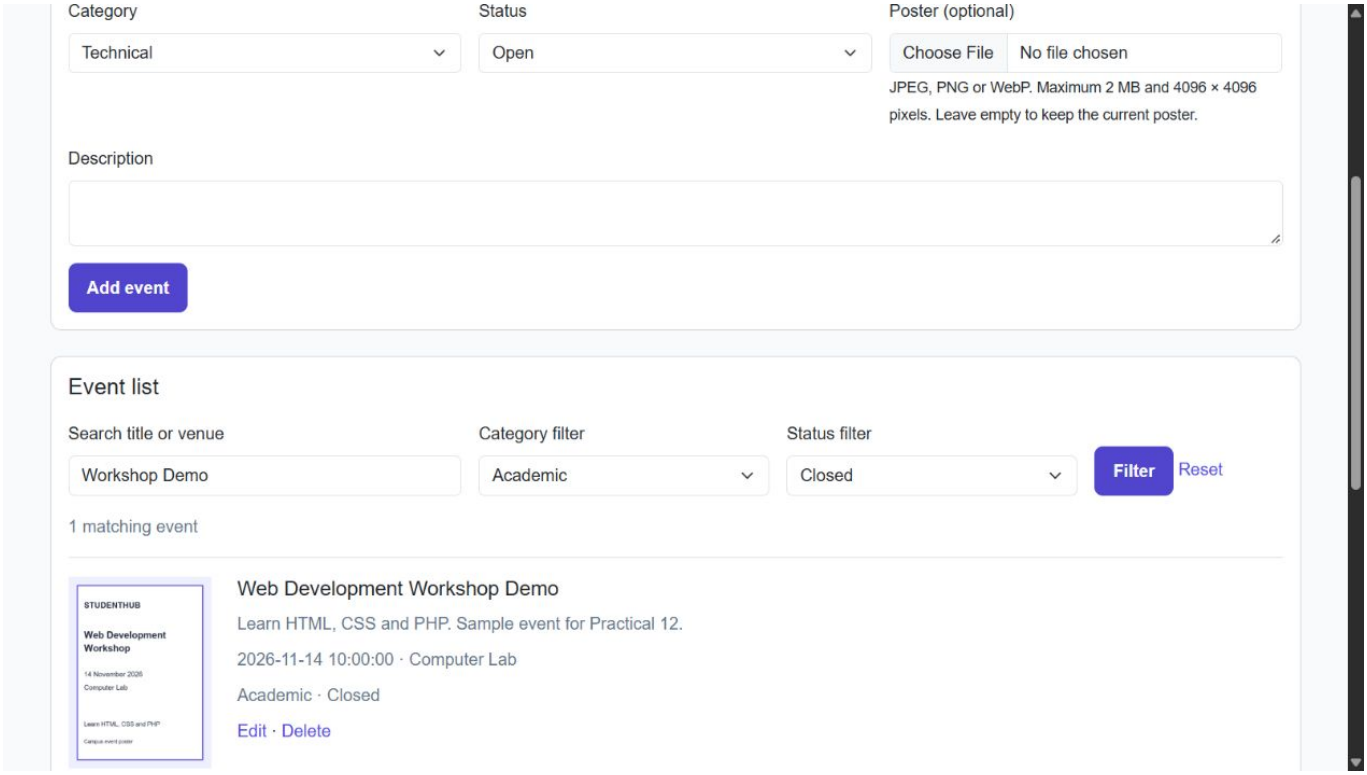


Figure 4: Search combined with academic category and closed status.

Screenshots of Upload Validation

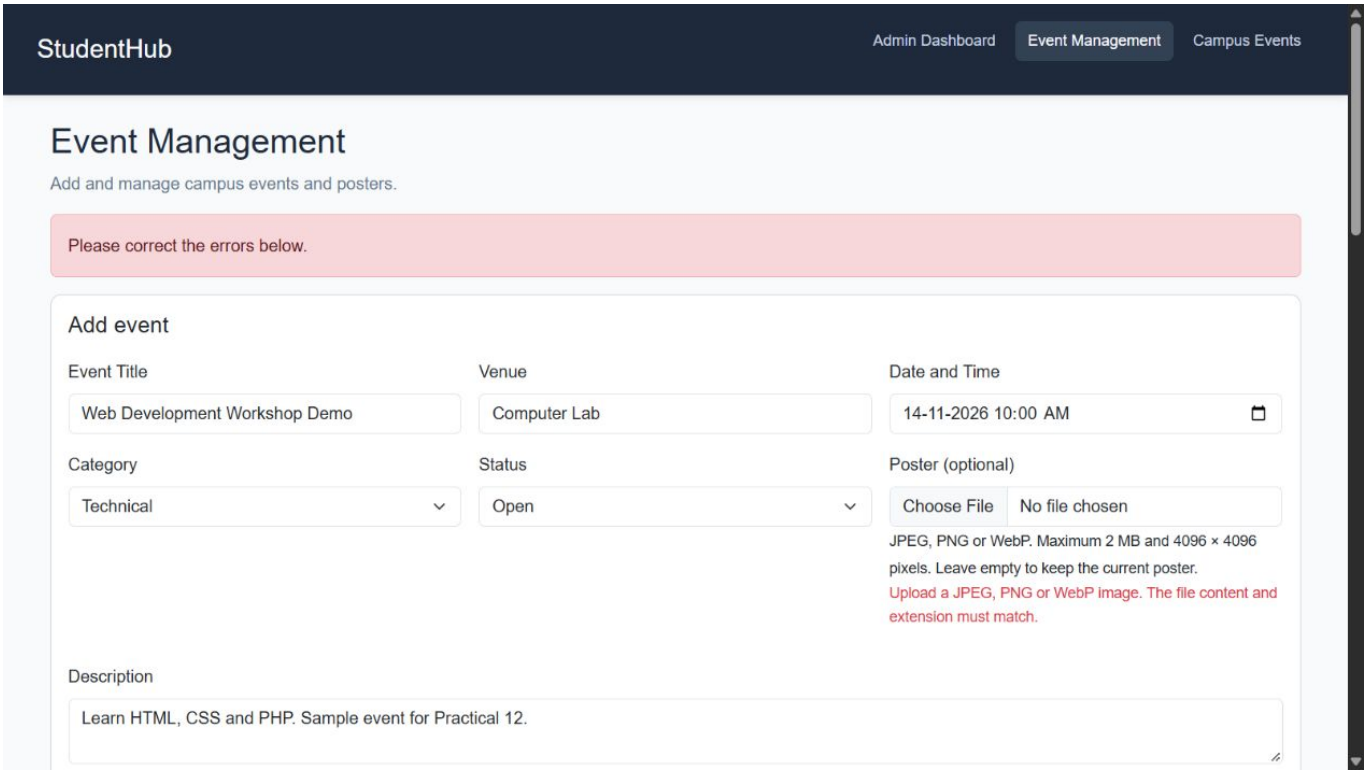


Figure 5: Unsupported poster file rejected with a validation message.

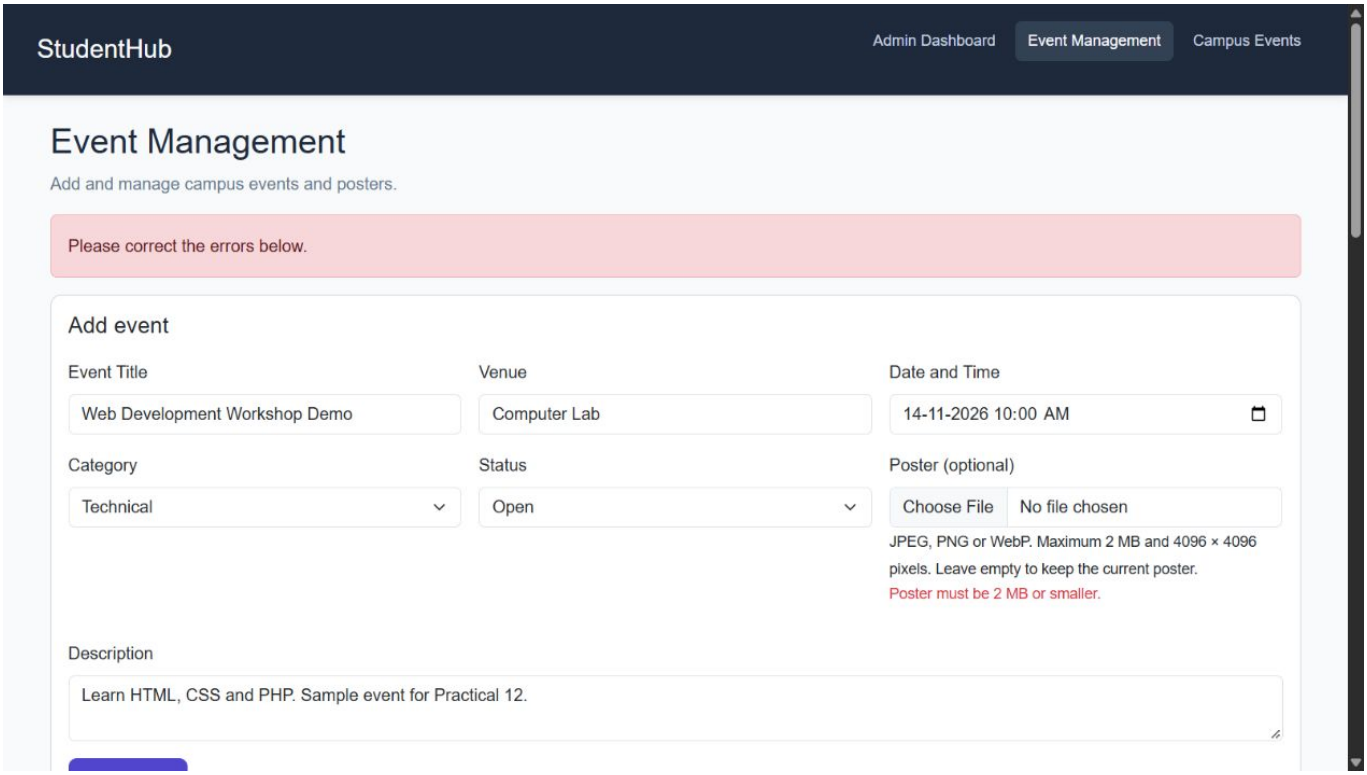


Figure 6: Poster larger than 2 MB rejected.

Screenshots of Delete Confirmation and Audit Log

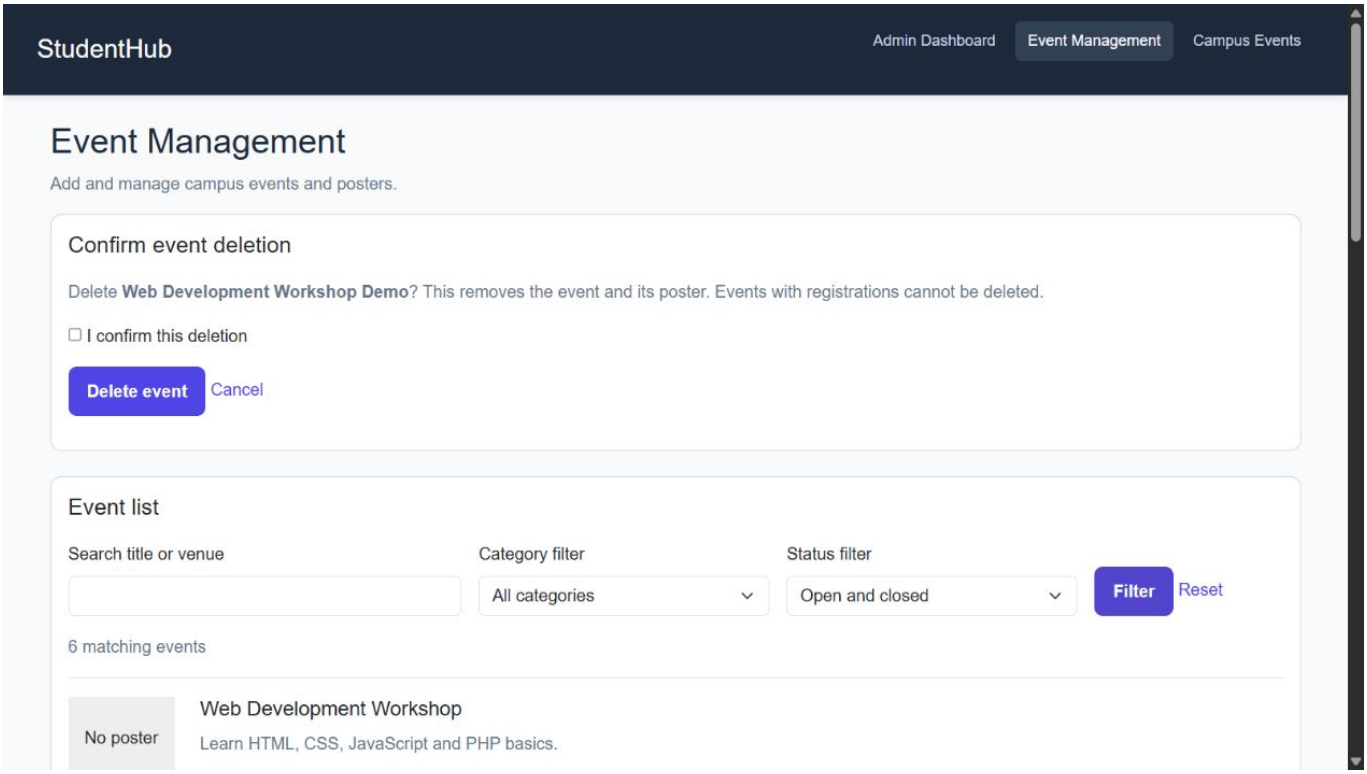


Figure 7: Confirmation is required before deleting the selected event.

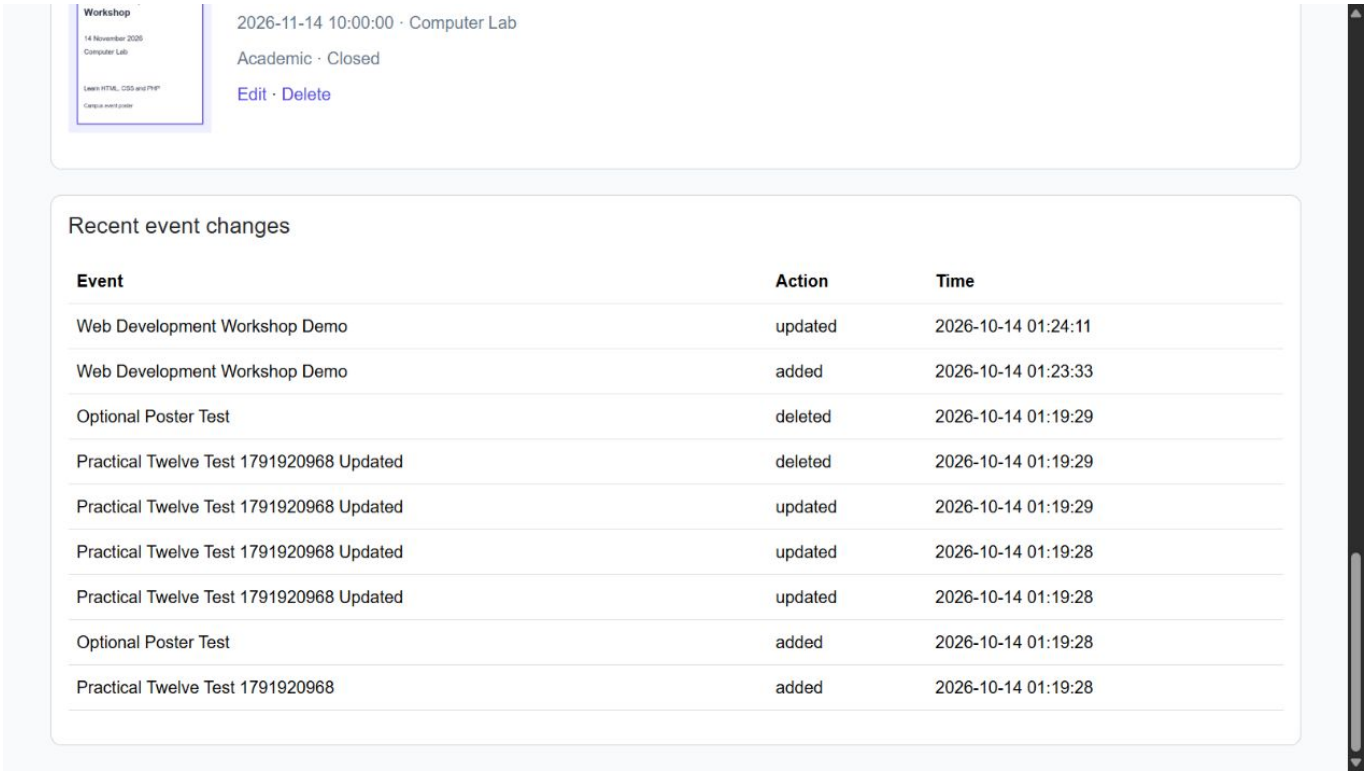


Figure 8: Audit entries show add, update and delete actions from testing.

Database Results

Check	Observed result
Records in events table and SQL dump	6
Events with posters	1
SQL dump verification	Imported successfully into a temporary table.
Audit actions	Added, updated and deleted entries recorded.

Testing Summary

Test input or action	Observed result
Admin / student / anonymous access	Allowed / HTTP 403 / login redirect.
Add with poster / no poster	Both valid events saved.
PNG, JPEG and WebP images	Accepted with random filenames.
Text, PHP, SVG and spoofed image	Rejected with validation messages.
Wrong extension / file above 2 MB	Rejected.
Image above 4096 pixels / invalid fields	Rejected.
Direct upload directory access	HTTP 403; poster endpoint works.
Edit without poster / replace poster	Old image retained / replaced image cleaned up.
Search, category and open/closed filters	Matching records displayed.
Campus event page	Updated database event returned.
Unconfirmed delete / invalid CSRF	Rejected; event retained.
Event with registrations	Deletion blocked; closing suggested.
Confirmed delete	Event and its poster removed.
Deleted event audit	Entry remains after deletion.

All 36 HTTP checks passed. Only temporary test events were deleted. Six sample events remain in the database, including the workshop demonstration event.

Key Questions and Answers

Q1 Are add, update and delete functionalities implemented correctly

Yes. Prepared INSERT, UPDATE and DELETE statements perform these operations. Valid changes were saved, deletion required confirmation and registered events were protected.

Q2 Is the UI linked with the database correctly

Yes. The admin list loads records from MySQL. Form changes update the stored records, and the campus event page also loads events from the database.

Q3 Are poster uploads validated for type and size

Yes. Actual MIME type, matching extension, file size and image contents are checked on the server. JPEG, PNG and WebP files up to 2 MB are accepted. Invalid type and size tests were rejected.

Q4 Are success and failure messages displayed

Yes. Add, update and delete actions display success messages. Invalid fields, rejected posters, missing confirmation and registered-event deletion display appropriate error messages.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 12 source commit: 9bc9bff
Message: Complete Practical 12 event CRUD and validated poster uploads

Conclusion

The event management module was completed with CRUD operations, validated poster upload and admin access. Category and open/closed filters were added. Random image renaming, upload directory protection and audit logging were also implemented. The test cases passed and the SQL dump was verified.

PRACTICAL 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

AIM

To build PHP JSON API endpoints for campus events and consume them using JavaScript Fetch API for asynchronous list, add, update and delete operations.

Introduction

The new API reads event records from MySQL and returns JSON with HTTP status codes. Public requests can list or read events. Create, update and delete operations require an administrator session and CSRF token. An admin-only demo page uses Fetch and updates the list without a full page reload.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, HTML, CSS, JavaScript Fetch API, browser, Git and GitHub.

Files Used

File	Purpose
php/api-events.php	JSON GET, POST, PUT and DELETE API with validation and role checks.
api-events-demo.php	Admin-only Fetch client and event form.
js/api-demo.js	Asynchronous list, create, edit and delete requests.
php/event-functions.php	Shared event validation, prepared statements and audit helper.
docs/Practical13-setup.md	Local setup and browser Network panel steps.
scripts/test-practical13.py	Repeatable authenticated API integration checks.
scripts/test-practical13-input.php	Database-free validation checks.

Key Skills Used

PHP JSON API, HTTP methods and status codes, Fetch API, asynchronous CRUD, JSON parsing, server-side validation, CSRF protection and frontend/backend integration.

Implementation Overview

1 JSON Read Endpoints

GET returns an event collection with a count or one event by ID. Event fields are selected from the MySQL database using prepared statements. Missing and malformed identifiers return explicit JSON errors.

2 Create and Update

POST creates an event and returns HTTP 201. PUT updates an event by ID. Both methods validate title, date, venue, category and status before database writes, then record the admin action.

3 Delete and Referential Checks

DELETE removes only an event that has no registrations. A referenced event returns HTTP 409 and remains available for closure through the event management page.

4 Access and Error Handling

Read requests are public. Mutations require a current admin role and the session CSRF token in X-CSRF-Token. Validation failures use HTTP 422, unauthorized requests use 401 or 403, and unexpected failures return a generic 500 response.

5 Fetch Client without Reload

The admin demo loads JSON on page open, submits JSON with Fetch and refreshes the table after each successful change. Errors appear in a live status message. Event text is rendered with textContent to avoid treating API data as HTML.

Implementation Evidence

The live API passed 9 authenticated integration checks, including JSON reads, anonymous access denial, validation, create, fetch-by-ID, update, delete and missing-record handling. Live browser screenshots remain to be captured.

Database Results

Check	Observed result
API data source	MySQL events table through prepared queries.
Mutation audit	Practical 14 audit_logs and the existing event_audit table.
Credential handling	No API password, cookie or token is returned in JSON.

Testing Summary

Test input or action	Observed result
Database-free event validation	4 checks passed: valid data, date, category/status and length limits.
Practical 13 API integration suite	9 checks passed: JSON reads, access denial, validation, create, fetch, update, delete and missing record.

Screenshots of Portal Implementation

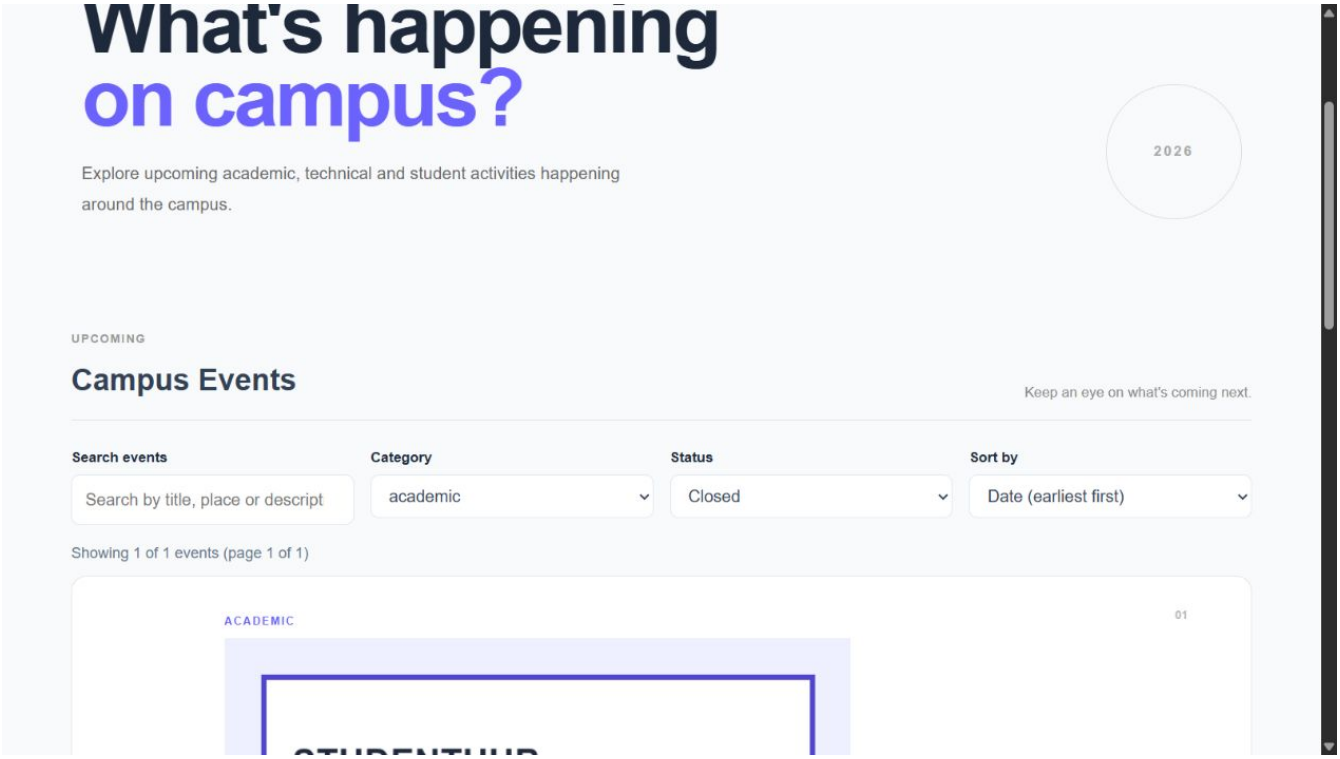


Figure 1: Existing campus events page, showing the public event listing used by the portal.

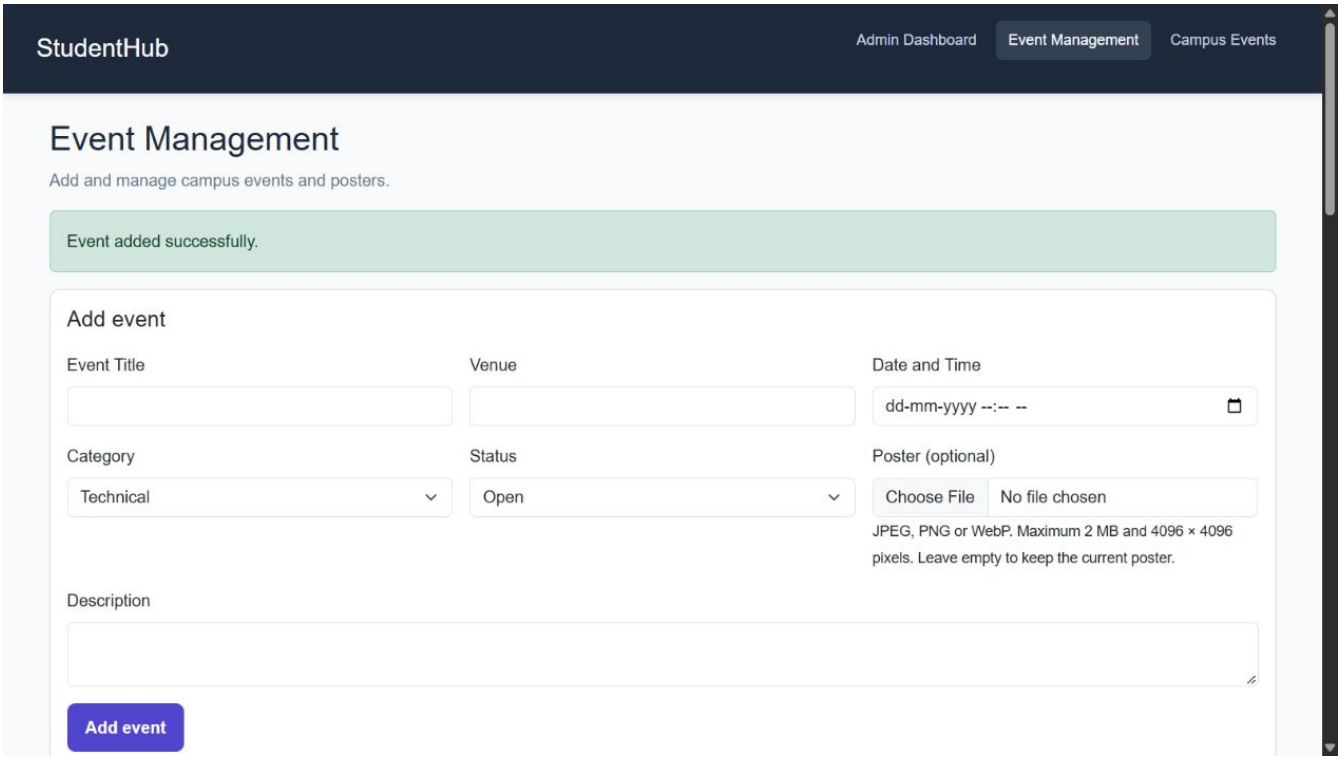


Figure 2: Existing event-management success state from Practical 12; this is supporting UI context, not a Fetch API screenshot.

Key Questions and Answers

Q1 Do PHP endpoints return valid JSON responses

The endpoint returns JSON for successful reads and mutations, and JSON error objects for rejected requests. The runtime checks are provided but need the configured local XAMPP database.

Q2 Are GET and POST requests handled properly

GET supports the event collection and an individual ID. POST creates an event. PUT updates and DELETE removes an eligible event. Unsupported methods return HTTP 405.

Q3 Are loading, success and error states shown on the frontend

Yes. The Fetch demo displays loading text, success messages and API errors through a role=status live region.

Q4 Is input validation performed before database operations

Yes. The existing shared event validator checks lengths, date, category and status before prepared INSERT or UPDATE statements execute.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 13–15 files and reports are committed and pushed to the main branch.

Conclusion

The live API and authenticated CRUD workflow passed all 9 integration checks. Screenshots from the running API demo and Network panel remain to be added if required by the instructor.

PRACTICAL 14

Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

AIM

To create a database-driven admin dashboard with user, student, event and registration statistics, role-appropriate navigation, recent registrations and an audit trail.

Introduction

The admin dashboard queries current counts and latest event registrations from MySQL. Admin navigation links expose student management, event management and the Practical 13 API demo. A general audit_logs table records important student and event management actions without storing passwords, session tokens or form contents.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, HTML, CSS, JavaScript Fetch API, browser, Git and GitHub.

Files Used

File	Purpose
admin-dashboard.php	Admin-only statistics, registration list and recent audit activity.
php/admin-audit.php	Shared helper for recording administrative changes.
php/event-functions.php	Adds event actions to the general audit trail.
students-manage.php	Records add, edit, soft-delete and restore actions.
sql/practical14.sql	Additive audit_logs table migration.
css/style.css	Responsive dashboard statistics grid.
scripts/test-practical14.py	Repeatable role, dashboard and menu checks.
docs/Practical14-setup.md	Migration and role verification steps.

Key Skills Used

Dashboard design, role-based access control, SQL aggregation, prepared queries, audit logging, role-based navigation and server-side authorization.

Implementation Overview

1 Database Statistics

Four live counts are displayed for users, active students, events and event registrations. Counts are calculated with SQL against the current database.

2 Latest Registrations

The dashboard joins registrations to student and event records and shows the five most recent entries with their registration time.

3 Admin-Only Access and Menu

The page calls requireRole(admin) before loading dashboard data. Management links are present on the admin dashboard; faculty and student dashboards remain separately protected.

4 General Audit Log

The migration creates audit_logs with actor, action, target type, target ID, summary and timestamp. Student add/edit/delete/restore and event add/edit/delete operations write entries.

5 Safe Audit Data

Audit summaries include record names or IDs for review. They deliberately exclude password values, CSRF tokens, remember-me validators and complete submitted forms.

Implementation Evidence

The additive audit_logs migration is installed. Anonymous dashboard access redirects to login. Signed-in admin checks pass for the dashboard, four statistic cards, latest registration section, audit section and management/API navigation. Student and faculty menu checks remain pending their existing passwords.

Database Results

Check	Observed result
Dashboard counts	Users, active students, events and registrations.
Recent registrations	Latest five registration rows joined to student and event titles.
Audit retention	Audit entries remain after a target event is deleted.
Migration	Additive CREATE TABLE IF NOT EXISTS; existing data is preserved.

Testing Summary

Test input or action	Observed result
Audit table migration	PASS: audit_logs created without changing existing tables.
Anonymous admin dashboard access	PASS: redirects to login.
Anonymous API demo access	PASS: redirects to login.
Admin dashboard, four cards, registrations, audit and menu	PASS: all 5 admin checks.
Student / faculty dashboard and role menus	Pending: their existing passwords were not changed.
Audit entries from student/event CRUD	Hooks are implemented; authenticated record-change checks remain pending.

Screenshots of Portal Implementation

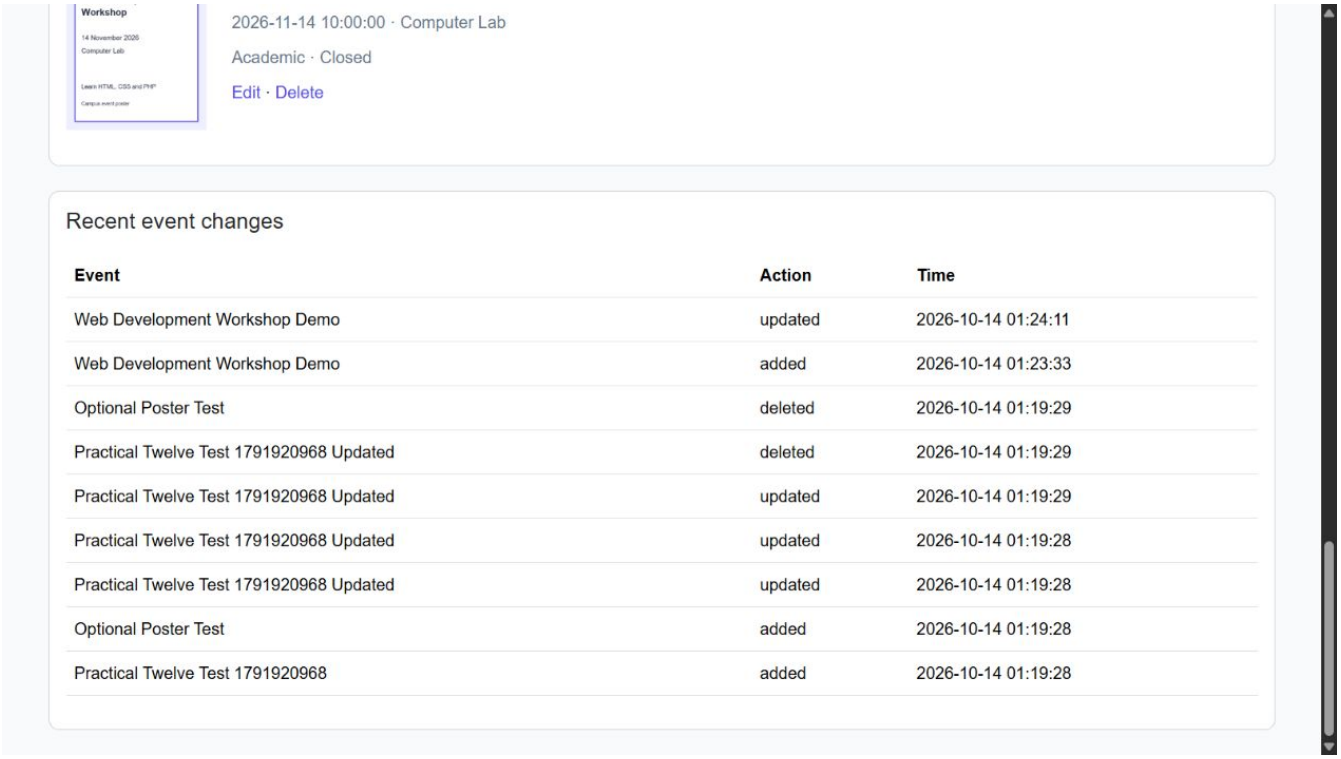


Figure 1: Existing Practical 12 event-change audit view; the new Practical 14 audit_logs dashboard view still needs a fresh capture.

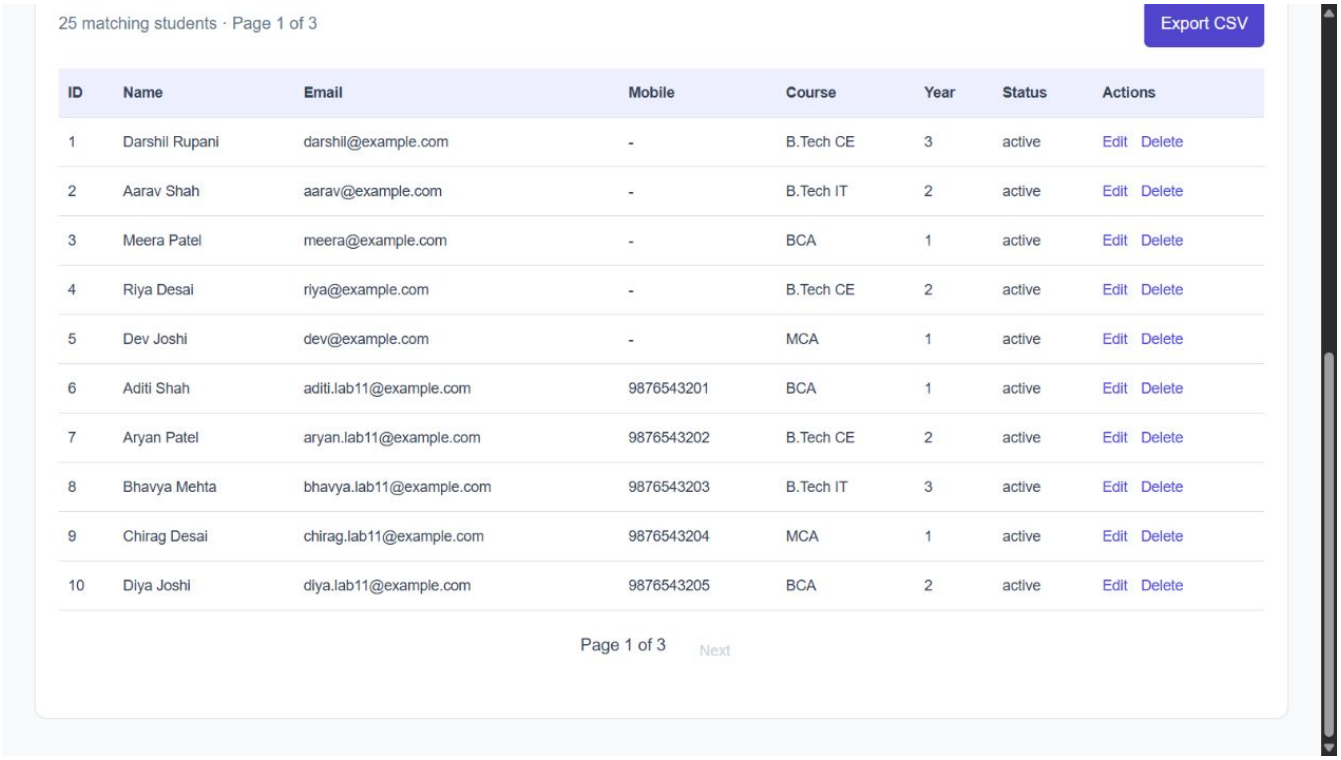


Figure 2: Existing protected student-management listing used as role-based admin UI context.

Key Questions and Answers

Q1 Are dashboard values loaded dynamically from the database

Yes. Counts and registration rows are queried for each admin dashboard request.

Q2 Is access restricted to admin users

Yes. requireRole checks the current database role before the page runs its dashboard queries.

Q3 Are role-based menus working

The management and API navigation is shown on the admin dashboard. Student and faculty pages remain separate role-protected destinations.

Q4 Are important admin actions recorded in audit_logs

Student record changes and event CRUD actions call the shared audit helper. The migration stores the actor, action, target and time.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 13–15 files and reports are committed and pushed to the main branch.

Conclusion

The audit migration, anonymous access gate and signed-in administrator dashboard checks pass. Student and faculty navigation checks still need their existing lab passwords; event/student audit-write behavior remains to be checked with authenticated CRUD.

PRACTICAL 15

Final Integration, Testing, Documentation, GitHub Submission, and Local Deployment

AIM

To integrate StudentHub modules, document the local deployment, provide repeatable integration checks, and prepare the repository for a final review.

Introduction

The portal includes authentication, student CRUD, event management, a JSON API and an admin dashboard with audit logging. Practical 15 adds a deployment guide and smoke-check script. The local XAMPP site and configured app database connection are currently available for public and unauthenticated checks.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, HTML, CSS, JavaScript Fetch API, browser, Git and GitHub.

Files Used

File	Purpose
docs/Practical15-deployment.md	Local setup, integration checklist and Apache virtual-host notes.
scripts/check-practical15.py	Public-page and admin-demo access smoke checks.
docs/Practical15-test-results.json	Records whether local smoke checks ran.
docs/Practical13-setup.md	API setup and test guidance.
docs/Practical14-setup.md	Dashboard audit migration and setup steps.
README.md	Project index updated for Practicals 13-15.

Key Skills Used

Full-stack integration, test planning, deployment documentation, accessibility review, local Apache configuration and repository hygiene.

Implementation Overview

1 Integrated Modules

Existing authentication and role checks connect to student management, events and registrations. The Practical 13 API integrates event CRUD, and Practical 14 adds dashboard statistics and audit activity.

2 Repeatable Verification

The API test script uses a temporary event and removes it after testing. The local smoke script checks public pages and unauthenticated access to the admin demo. Existing Practical 10-12 scripts cover their prior modules.

3 Deployment Documentation

The guide provides migration ordering, private database configuration, poster storage setup, lab account creation and an optional local Apache virtual host.

4 Security and Repository Review

The guide keeps credentials, database configuration and uploaded poster files out of Git. The Practical 13–15 implementation and reports are committed and pushed to the configured GitHub repository.

5 Limitations of Current Verification

Public API and local smoke checks pass. Student/faculty role checks, fresh Practical 13/14 screenshots and accessibility review remain to be completed.

Implementation Evidence

Practical 13 authenticated checks passed 9/9, Practical 14 admin dashboard checks passed, and Practical 15 smoke checks passed 8/8. The Practical 13–15 update is committed and pushed to the repository main branch. The screenshot pages use earlier portal captures; fresh API demo and admin audit-dashboard screenshots and student/faculty role checks remain outstanding.

Database Results

Check	Current status
PHP source syntax	Changed PHP files checked with PHP 8.2 lint.
Fetch client syntax	JavaScript syntax checked with Node.
Local XAMPP deployment	Apache and MySQL reachable; app database connection verified.
GitHub submission	Practical 13–15 update committed and pushed to main.

Testing Summary

Test input or action	Observed result
Changed PHP files	Syntax checks passed.
Fetch demo JavaScript	Node syntax check passed.
Practical 13 API integration	9/9 authenticated and public checks passed.
Practical 14 admin dashboard	Migration and admin checks passed; student/faculty menu checks pending their existing passwords.
Practical 15 smoke script	8/8 checks passed against the local site.
Accessibility review	Checklist and responsive checks documented; live review pending.

Screenshots of Portal Implementation

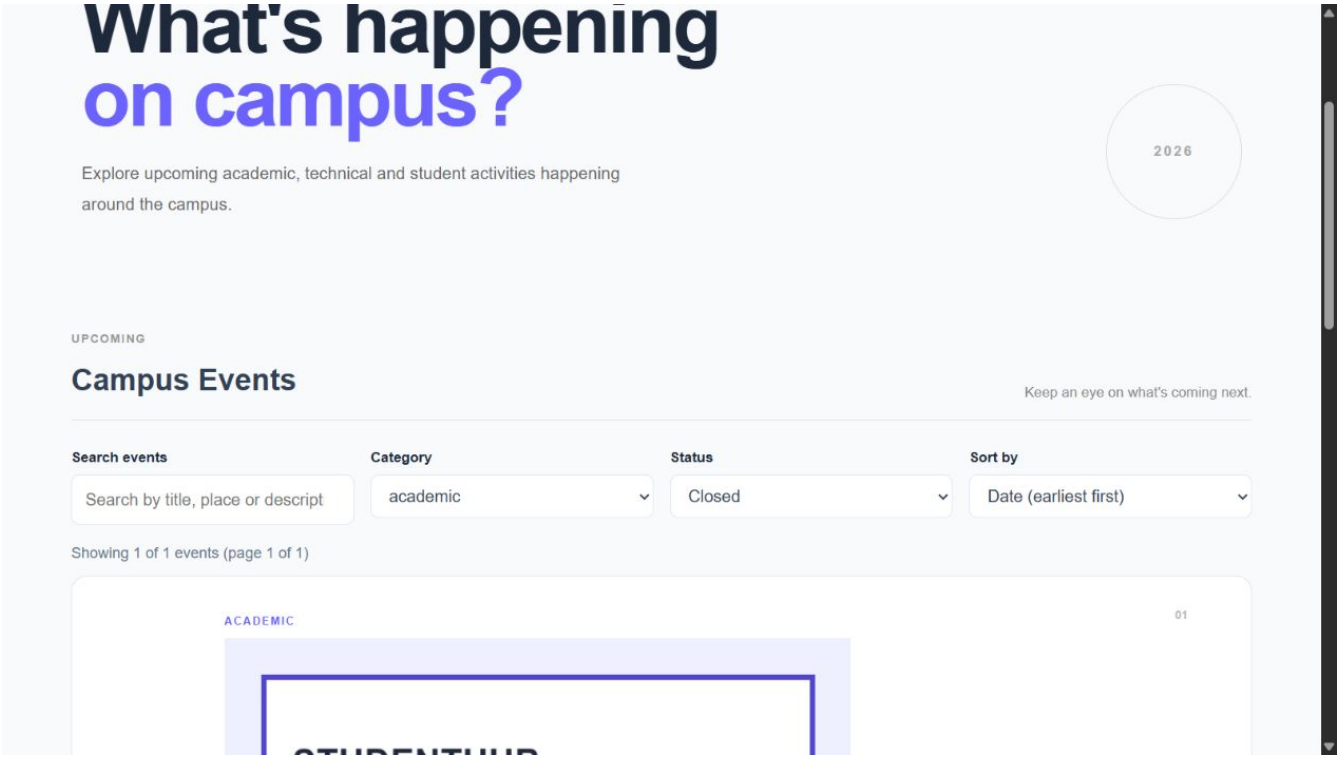


Figure 1: Existing public campus-events page used for the integrated portal.

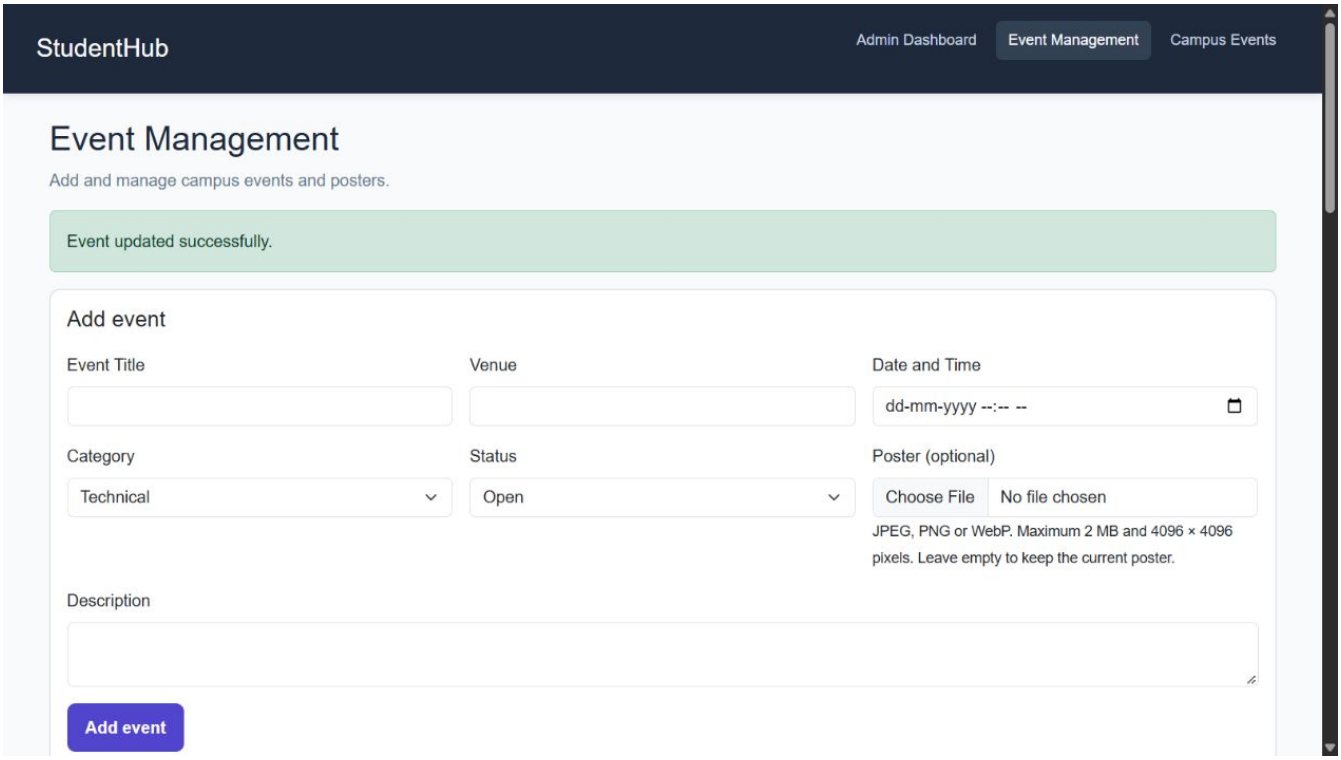


Figure 2: Existing event-management update state, showing the portal UI after an event change.

Key Questions and Answers

Q1 Are all pages properly linked and navigable

The new admin dashboard links to the student, event and API modules. The deployment guide includes a page and responsive review checklist. Live browser verification remains pending.

Q2 Are sessions, roles, validations and database operations integrated

Existing authentication, role checks and prepared-query CRUD remain in place. Practicals 13 and 14 connect their API and dashboard actions to the same database.

Q3 Are security and accessibility checklists completed

Security and accessibility review steps are documented. The interactive checklist cannot be marked complete without reviewing the running site.

Q4 Is project documentation sufficient for installation and testing

Practical 13, 14 and 15 setup guides describe the migration, API, local XAMPP and test steps.

Q5 Is the repository clean and committed

The Practical 13–15 update is committed and pushed to the configured GitHub repository main branch.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 13–15 files and reports are committed and pushed to the main branch.

Conclusion

The local smoke suite and Practical 13 API suite pass, and the Practical 14 admin dashboard checks are verified. The Practical 13–15 update is pushed to GitHub. Student/faculty checks, fresh Practical 13/14 screenshots and accessibility review remain outstanding.